BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

SINH VIÊN THỰC HIỆN

LÊ TRUNG KIÊN
MSSV: 223209
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ THIẾT BỊ IOT THEO DÕI SỨC KHỎE BỆNH NHÂN NGƯỜI CAO TUỔI

ĐỒ ÁN 2
NGÀNH KỸ THUẬT PHẦN MỀM
Mã ngành: 7480103

Cần Thơ, tháng 5 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

HỌ TÊN TÁC GIẢ

LÊ TRUNG KIÊN
MSSV: 223209
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ THIẾT BỊ IOT THEO DÕI SỨC KHỎE BỆNH NHÂN NGƯỜI CAO TUỔI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

NGÀNH KỸ THUẬT PHẦN MỀM

Mã ngành: 7480103

GIẢNG VIÊN HƯỚNG DẪN

TS.TRẦN VĂN THIỆN

LỜI CẢM TẠ

Chúng em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức quý báu trong suốt quá trình học tập và thực hiện đề tài. Nhờ sự hướng dẫn và chỉ bảo tận tâm của quý Thầy Cô, chúng em đã có được nền tảng kiến thức chuyên ngành cần thiết để hoàn thành đồ án này.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn **Thầy Trần Văn Thiện**, người đã tận tình hướng dẫn, hỗ trợ và giải đáp những khó khăn mà nhóm gặp phải trong suốt quá trình thực hiện **Đồ án 2**. Sự quan tâm, góp ý và định hướng của thầy là nguồn động viên rất lớn, giúp nhóm chúng em hoàn thành bài báo cáo một cách tốt nhất.

Do kiến thức chuyên môn và kinh nghiệm thực tế của chúng em còn hạn chế nên trong quá trình thực hiện đồ án khó tránh khỏi những thiếu sót. Vì vậy, chúng em rất mong nhận được sự thông cảm cùng những ý kiến đóng góp quý báu từ quý Thầy Cô để nhóm có thể rút kinh nghiệm, bổ sung kiến thức và hoàn thiện bản thân hơn trong quá trình học tập sau này.

Cuối lời, chúng em xin kính chúc quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ thật nhiều sức khỏe, hạnh phúc và thành công trong công tác giảng dạy.

Chúng em xin chân thành cảm ơn!

Cần Thơ, ngày tháng 5 năm 2026

Sinh viên thực hiện

LÊ TRUNG KIÊN

CAM KẾT

Chúng tôi xin cam kết đồ án tốt nghiệp này được hoàn thành dựa trên kết quả nghiên cứu của chúng tôi và các kết quả này chưa được sử dụng cho bất kỳ đồ án cùng cấp nào khác.

Cần Thơ, ngày tháng 5 năm 2026

Sinh viên thực hiện

LÊ TRUNG KIÊN

NHẬN XÉT ĐÁNH GIÁ CỦA GIÁO VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Cần Thơ, ngày….tháng…. năm 2026

GIẢNG VIÊN HƯỚNG DẪN

NHẬN XÉT ĐÁNH GIÁ CỦA GIÁO VIÊN PHẢN BIỆN

……
……
……

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU 1
1.1 Lý do chọn đề tài 1
1.2 Mục tiêu nghiên cứu 2
1.3 Đối tượng nghiên cứu 4
1.4 Phạm vi nghiên cứu 6
1.5 Phương pháp nghiên cứu 9
1.6 Ý nghĩa khoa học và thực tiễn của đề tài 11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 13
2.1 Tổng quan về hệ thống theo dõi sức khỏe từ xa 13
2.2 Internet of Things trong theo dõi sức khỏe 14
2.3 Các chỉ số sức khỏe được sử dụng trong hệ thống 14
2.4 Kiến trúc client-server và REST API 16
2.5 Xác thực và quản lý phiên bằng JWT, refresh token 17
2.6 Xác thực thiết bị bằng Device Token 19
2.7 Phân quyền RBAC trong hệ thống 20
2.8 FastAPI và lý do lựa chọn 20
2.9 MongoDB và lý do lựa chọn 21
2.10 Redis và vai trò trong hệ thống 23
2.11 Docker và Docker Compose 23
2.12 Web và lý do lựa chọn 25
2.13 Cơ chế command queue cho ECG on-demand 26
2.14 Các nguyên tắc bảo mật cơ bản được áp dụng 26
2.15 Logging, health check và giám sát hệ thống 27
CHƯƠNG 3: PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC HỆ THỐNG 28
3.1 Bối cảnh bài toán và yêu cầu tổng quát 28
3.2 Yêu cầu hệ thống 28
3.3 Tác nhân tham gia và phạm vi tương tác 30
3.4 Kiến trúc tổng thể của hệ thống 37
3.5 Kiến trúc phân lớp và phân rã theo thành phần 39
3.6 Thiết kế dữ liệu logic ở mức hệ thống 40
3.7 Các luồng nghiệp vụ chính 41
3.8 Kiến trúc giao tiếp và xác thực giữa các thành phần 46
3.9 Đánh giá kiến trúc hiện tại 56
3.10 Kết luận chương 56
CHƯƠNG 4: PHÂN TÍCH VÀ THIẾT KẾ BACKEND 58
4.1 Vai trò và mục tiêu thiết kế backend 58
4.2 Kiến trúc module backend 58
4.3 Thiết kế lớp giao tiếp API 60

4.4 Thiết kế xác thực và phân quyền 61
4.5 Thiết kế xử lý health readings và cảnh báo 63
4.6 Thiết kế quản lý thiết bị và liên kết người dùng 63
4.7 Thiết kế command queue cho ECG on-demand 64
4.8 Thiết kế lớp truy cập dữ liệu và mô hình lưu trữ 65
4.9 Thiết kế observability, rate limit và vận hành 65
4.10 Đánh giá backend hiện tại 66
4.11 Kết luận chương 67
CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU 68
5.1 Vai trò của cơ sở dữ liệu và định hướng thiết kế 68
5.2 Lý do lựa chọn MongoDB và mô hình tổng quát của database 68
5.3 Nhóm dữ liệu lõi sức khỏe và điều phối nghiệp vụ 70
5.4 Nhóm thực thể, liên kết và dữ liệu bảo mật 71
5.5 Chỉ mục, TTL, tối ưu truy vấn và an toàn dữ liệu 72
5.6 Đánh giá thiết kế cơ sở dữ liệu 74
CHƯƠNG 6: THIẾT KẾ ỨNG DỤNG WEB 75
6.1 Vai trò của ứng dụng Web và mục tiêu thiết kế 75
6.2 Kiến trúc tổng thể và cơ chế khởi tạo hệ thống frontend 77
6.3 Tầng giao tiếp API và quản lý phiên người dùng 78
6.4 Quản lý device scope và dữ liệu sức khỏe theo thời gian thực 79
6.5 Tổ chức giao diện và luồng sử dụng chính 80
6.6 Cache, xử lý lỗi, kiểm thử và đánh giá ứng dụng 86
CHƯƠNG 7: THIẾT KẾ VÀ MÔ PHỎNG THIẾT BỊ ESP32 87
7.1 Vai trò của thiết bị và phạm vi thiết kế chương 87
7.2 Kiến trúc phần cứng mức khối của thiết bị 87
7.3 Thiết kế firmware và luồng xử lý readings 89
7.4 Cơ chế ECG on-demand và sự phối hợp với backend 90
7.5 Độ tin cậy vận hành, nguồn và khả năng mở rộng 91
7.6 Đánh giá thiết kế phần cứng ESP32 92
CHƯƠNG 8: TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG 94
8.1 Môi trường triển khai hệ thống 94
8.2 Cấu hình và khởi tạo backend 95
8.3 Triển khai và kiểm tra lớp dữ liệu MongoDB – Redis 96
8.4 Triển khai và kiểm thử API backend 97
8.5 Triển khai và kiểm thử lớp thiết bị ESP32 97
8.6 Kiểm thử và đánh giá ứng dụng web 99
8.7 Đánh giá mức độ hoàn thiện của hệ thống 100
8.8 Hướng cải tiến sau triển khai 100
8.9 Kết luận chương 100
CHƯƠNG 9: KẾT LUẬN VÀ KIẾN NGHỊ 101
9.1 Kết quả đạt được của đề tài 101

9.2 Đánh giá mức độ hoàn thành mục tiêu đề tài 103
9.3 Những điểm mạnh nổi bật của hệ thống.....104
9.4 Hạn chế hiện tại của hệ thống.....105
9.5 Bài học kinh nghiệm rút ra từ đề tài 106
9.6 Kiến nghị và hướng phát triển 107
9.7 Kết luận chương.....107
TÀI LIỆU THAM KHẢO 108
PHỤ LỤC 110

DANH SÁCH BẢNG

Bảng 3.1. Ma trận yêu cầu chức năng theo thành phần.....28
Bảng 3.2. Yêu cầu phi chức năng của hệ thống 30
Bảng 3.3. Tác nhân chính và vai trò trong hệ thống.....31
Bảng 3.4. Ma trận giao tiếp và xác thực giữa các thành phần.....47
Bảng 4.1. Các mô-đun chính của backend 59
Bảng 4.2. Nhóm endpoint chính của backend.....61
Bảng 4.3. So sánh hai lớp xác thực trong backend.....62
Bảng 4.4. Các cơ chế quan sát và bảo vệ API 66
Bảng 5.1. Tổng quan vai trò của các collection chính trong hệ thống. 69
Bảng 5.2. Tóm tắt chiến lược chỉ mục và retention của các collection chính. ...73
Bảng 6.1. Vai trò của các module quản lý trạng thái trong ứng dụng Web. 77
Bảng 7.1. Các khối phần cứng chính và vai trò trong hệ thống thiết bị. 88

DANH SÁCH HÌNH

Hình 3.1. Sơ đồ Use Case tổng quan toàn hệ thống Eldercare.....32
Hình 3.2. Sơ đồ Use Case chi tiết phân hệ Tài khoản & Xác thực 33
Hình 3.3. Sơ đồ Use Case chi tiết phân hệ Chủ thiết bị 34
Hình 3.4. Sơ đồ Use Case chi tiết phân hệ Người xem 35
Hình 3.5. Sơ đồ Use Case chi tiết phân hệ Quản trị viên 36
Hình 3.6. Sơ đồ Use Case chi tiết phân hệ Thiết bị ESP32.....37
Hình 3.7. Kiến trúc tổng thể của hệ thống Eldercare 38
Hình 3.8. Component diagram tổng quan app-server.....38
Hình 3.9. Deployment diagram tổng quan Docker/Production.....39
Hình 3.10. Phân rã hệ thống theo lớp chức năng 39
Hình 3.11. Các nhóm dữ liệu logic của hệ thống 40
Hình 3.12. Vòng kín vận hành từ thiết bị, backend, cơ sở dữ liệu đến ứng dụng 42
Hình 3.13. Activity tổng quan nghiệp vụ 43
Hình 3.14. Activity đăng ký / đăng nhập.....44
Hình 3.15. Activity liên kết thiết bị.....44
Hình 3.16. Activity quản lý người xem.....45
Hình 3.17. Activity yêu cầu ECG.....45
Hình 3.18. Activity xử lý cảnh báo 46
Hình 3.19. Sơ đồ Sequence tổng quan App ↔ API ↔ DB.....47
Hình 3.20. Sequence đăng ký 48
Hình 3.21. Sequence đăng nhập 49
Hình 3.22. Sequence khôi phục phiên 50
Hình 3.23. Sequence liên kết thiết bị bằng mã ghép nối 51
Hình 3.24. Sequence tải dashboard thiết bị (latest/history/alerts).....51
Hình 3.25. Sequence quản lý người xem.....52
Hình 3.26. Sequence yêu cầu ECG theo lệnh từ app.....53
Hình 3.27. Sequence xác nhận xử lý cảnh báo.....54
Hình 3.28. Sequence admin đăng ký thiết bị / cấp lại pairing code 55
Hình 3.29. Sequence ESP gửi readings và backend sinh cảnh báo.....56
Hình 4.1. Kiến trúc backend ở mức mô-đun 59
Hình 4.2. Phân nhóm API và cơ chế xác thực.....60
Hình 4.3. Mô hình xác thực và phân quyền của backend.....62
Hình 4.4. Pipeline xử lý readings và sinh cảnh báo 63
Hình 4.5. Vòng đời command trong collection device_commands 64
Hình 4.6. Lớp observability, rate limit và vận hành.....65
Hình 5.1. Vai trò của MongoDB trong kiến trúc tổng thể hệ thống.....68
Hình 5.2. Mô hình tổng quát của database wearable và các nhóm collection chính. 69
Hình 5.3. Quan hệ logic giữa các collection lõi và nhóm thực thể. 71

Hình 5.4. Vòng đời command trong collection device_commands. 71
Hình 5.5. TTL và vòng đời dữ liệu chính trong MongoDB. 73
Hình 6.1. Kiến trúc nhiều lớp của ứng dụng web.....76
Hình 6.2. Sơ đồ quan hệ phụ thuộc khi khởi tạo ứng dụng Web. 77
Hình 6.3. Luồng bootstrap, đăng nhập và refresh token của ứng dụng.....79
Hình 6.4. Cơ chế đồng bộ phạm vi thiết bị trong ứng dụng Web. 79
Hình 6.5. Luồng dữ liệu từ giao diện web đến backend và ngược lại.....80
Hình 6.6. Cụm màn hình đầu vào của web: đăng nhập.....80
Hình 6.7. Màn hình giao diện đăng ký tài khoản. 81
Hình 6.8. Màn hình giao diện quản lý thiết bị.....82
Hình 6.9. Màn hình giao diện liên kết thiết bị.....82
Hình 6.10. Màn hình giao diện phân quyền người dùng.....83
Hình 6.11. Cụm màn hình theo dõi thiết bị, dashboard sức khỏe và lịch sử theo ngày.....84
Hình 6.12. Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng. 85
Hình 7.1. Kiến trúc phần cứng thiết bị ESP32 ở mức khối chức năng. 87
Hình 7.2. sơ đồ ESP32S3 88
Hình 7.2. Kiến trúc firmware theo mô-đun chức năng trên thiết bị ESP32. 89
Hình 7.3. Luồng đo lường, đóng gói và gửi readings từ ESP32 lên backend. ...90
Hình 7.4. Chuỗi tương tác giữa app, backend và ESP32 trong cơ chế ECG on-demand.....91
Hình 7.5. Gợi ý state machine cho độ tin cậy thiết bị: sampling, upload, polling, retry và offline buffer. 92
Hình 7.6. Vòng kín vận hành giữa thiết bị, backend, cơ sở dữ liệu và ứng dụng người dùng.....93

DANH MỤC TỪ VIẾT TẮT

Bảng dưới đây tổng hợp các từ viết tắt được sử dụng trong báo cáo.

Từ viết tắt	Tiếng Anh	Diễn giải
API	Application Programming Interface	Giao diện lập trình ứng dụng
ECG	Electrocardiogram	Điện tim
ESP32	Espressif 32-bit MCU	Vi điều khiển ESP32 dùng cho thiết bị IoT
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Phiên bản bảo mật của HTTP
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi dạng văn bản
JWT	JSON Web Token	Mã thông báo xác thực dạng JSON
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Phong cách kiến trúc cho web service/API
SpO2	Peripheral Capillary Oxygen Saturation	Độ bão hòa oxy ngoại vi trong máu
TTL	Time To Live	Thời gian tồn tại của dữ liệu/bản ghi
UI	User Interface	Giao diện người dùng

CHƯƠNG 1
MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh của Internet of Things, điện toán đám mây, thiết bị nhúng và ứng dụng web, các hệ thống theo dõi sức khỏe từ xa đang trở thành một hướng nghiên cứu và ứng dụng rất quan trọng. Nhu cầu này đặc biệt rõ ở nhóm người cao tuổi, người có bệnh nền, bệnh nhân cần theo dõi định kỳ hoặc người cần được giám sát liên tục các chỉ số sinh tồn mà không nhất thiết phải ở trong bệnh viện. Thay vì chỉ đo thủ công tại một thời điểm, hệ thống theo dõi sức khỏe từ xa cho phép thu thập dữ liệu liên tục, truyền dữ liệu về máy chủ, xử lý và hiển thị kết quả trên ứng dụng web để người dùng hoặc người chăm sóc có thể theo dõi nhanh chóng, thuận tiện. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024)

Thực tế cho thấy những chỉ số sức khỏe cơ bản như nhịp tim, nồng độ oxy trong máu, nhiệt độ cơ thể, nhịp thở và điện tim có giá trị lớn trong việc đánh giá tình trạng sức khỏe ban đầu. Nếu các dữ liệu này được thu thập liên tục và hiển thị kịp thời, người sử dụng có thể phát hiện sớm những bất thường để có hướng xử lý phù hợp. Do đó, việc xây dựng một hệ thống có thể thu thập, lưu trữ, hiển thị và cảnh báo dữ liệu sức khỏe là một bài toán có ý nghĩa thực tiễn cao.

Từ góc độ kỹ thuật, đề tài này hấp dẫn ở chỗ nó là một bài toán tích hợp nhiều thành phần khác nhau thành một hệ thống hoàn chỉnh. Theo mã nguồn hiện tại trong Git, hệ thống không chỉ gồm một ứng dụng web hay một API server riêng lẻ, mà là một kiến trúc tương đối đầy đủ gồm ba lớp chính. Lớp thứ nhất là thiết bị ESP32 đóng vai trò thiết bị đầu cuối thu thập dữ liệu. Lớp thứ hai là backend xây dựng bằng FastAPI đóng vai trò trung tâm xử lý, xác thực, lưu trữ dữ liệu vào MongoDB, quản lý command ECG, phân quyền người dùng, theo dõi tình trạng hệ thống và giới hạn truy cập bằng Redis-backed rate limit. Lớp thứ ba là ứng dụng web dùng để đăng nhập, đồng bộ phiên, lấy danh sách thiết bị liên kết, hiển thị dữ liệu sức khỏe và gửi yêu cầu ECG on-demand.

Một lý do quan trọng khác để chọn đề tài là tính thời sự và khả năng mở rộng thực tế. Hệ thống hiện tại trong Git đã vượt ra ngoài mức của một mô hình demo đơn giản. Backend không chỉ nhận readings rồi trả dữ liệu, mà còn có mô hình xác thực tách biệt giữa người dùng và thiết bị: app/admin sử dụng Authorization: Bearer <JWT>, còn thiết bị ESP dùng X-Device-Token; hệ thống có refresh_token được rotate qua endpoint /api/v1/auth/refresh, lưu dưới dạng hash trong auth_sessions, có phân quyền theo admin / caregiver / patient, có collection device_commands để hỗ trợ ECG on-demand, có các endpoint health check /live, /ready, /health, và có cả metrics để giám sát vận hành . Điều đó cho thấy hệ thống không chỉ phục vụ mục tiêu học tập mà còn mang tính ứng dụng cao, là một nền tảng tốt để phát triển thành sản phẩm hoàn thiện hơn trong tương lai.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng một hệ thống theo dõi sức khỏe từ xa hoạt động theo mô hình kết nối giữa thiết bị phần cứng ESP32, máy chủ xử lý trung tâm sử dụng FastAPI và ứng dụng Web. Hệ thống được định hướng nhằm tạo ra một nền tảng thống nhất cho phép thu thập dữ liệu sức khỏe từ thiết bị đo, truyền dữ liệu lên máy chủ, lưu trữ có tổ chức trong cơ sở dữ liệu, đồng thời hỗ trợ truy vấn và hiển thị thông tin trên giao diện Web một cách trực quan, chính xác và thuận tiện cho người sử dụng thông qua trình duyệt.

Bên cạnh chức năng thu thập và hiển thị dữ liệu, đề tài còn hướng đến việc bảo đảm sự phối hợp đồng bộ giữa các thành phần trong toàn bộ hệ thống, từ lớp thiết bị nhúng, lớp dịch vụ xử lý dữ liệu đến lớp giao diện người dùng trên nền tảng Web. Hệ thống cần có cơ chế xác thực và quản lý truy cập phù hợp nhằm bảo đảm tính an toàn, hạn chế truy cập trái phép và hỗ trợ quản lý người dùng theo vai trò hoặc phạm vi sử dụng. Đồng thời, hệ thống phải đáp ứng khả năng cập nhật dữ liệu gần thời gian thực ở mức cơ bản, giúp người dùng có thể theo dõi các chỉ số sức khỏe và trạng thái thiết bị trong thời gian ngắn sau khi dữ liệu được ghi nhận.

Ngoài ra, mục tiêu của đề tài không chỉ dừng lại ở việc xây dựng một mô hình thử nghiệm có khả năng vận hành, mà còn tạo tiền đề cho việc mở rộng trong tương lai. Cụ thể, kiến trúc hệ thống cần được thiết kế theo hướng linh hoạt, dễ nâng cấp và dễ tích hợp thêm các chức năng mới như cảnh báo bất thường, theo dõi lịch sử sức khỏe, quản lý nhiều thiết bị, hỗ trợ nhiều đối tượng người dùng hoặc kết nối với các dịch vụ xử lý dữ liệu nâng cao. Qua đó, đề tài góp phần xây dựng một giải pháp công nghệ có tính ứng dụng thực tiễn trong lĩnh vực chăm sóc và giám sát sức khỏe từ xa trên nền tảng Web.

1.2.2 Mục tiêu cụ thể

Trước hết, hệ thống cần xây dựng được khối backend đóng vai trò trung tâm trong việc tiếp nhận, xử lý và cung cấp dữ liệu. Backend phải có khả năng nhận dữ liệu đo từ thiết bị ESP32, thực hiện kiểm tra và chuẩn hóa dữ liệu đầu vào trước khi lưu trữ vào cơ sở dữ liệu MongoDB. Đồng thời, backend phải cung cấp các API phục vụ ứng dụng Web trong việc truy xuất dữ liệu sức khỏe mới nhất, dữ liệu lịch sử, dữ liệu ECG và các thông tin tổng hợp về tình trạng sức khỏe người dùng. Ngoài ra, backend cũng phải hỗ trợ các chức năng quản lý người dùng, quản lý thiết bị, lưu trữ và quản lý phiên xác thực, tạo và xử lý cảnh báo, đồng thời bảo đảm khả năng vận hành trong môi trường triển khai bằng Docker Compose.

Tiếp theo, hệ thống cần xây dựng được cơ chế xác thực, phân quyền và quản lý phiên phù hợp với đặc thù nhiều tác nhân tham gia. Cụ thể, ứng dụng Web và tài khoản quản trị sử dụng cơ chế xác thực dựa trên JWT Bearer khi

truy cập API, trong khi thiết bị ESP32 sử dụng mã định danh truy cập riêng thông qua X-Device-Token. Bên cạnh đó, hệ thống phải hỗ trợ cơ chế làm mới phiên đăng nhập bằng refresh_token, trong đó token được xoay vòng, băm và lưu trữ trong tập dữ liệu phiên xác thực để tăng mức độ an toàn. Hệ thống đồng thời phải bảo đảm khả năng phân quyền theo vai trò như quản trị viên, người chăm sóc và người được theo dõi, từ đó kiểm soát phạm vi truy cập và thao tác phù hợp với từng nhóm người dùng.

Một mục tiêu quan trọng khác là xây dựng ứng dụng Web phục vụ người dùng cuối. Ứng dụng phải hỗ trợ đầy đủ các chức năng cơ bản gồm đăng ký tài khoản, đăng nhập, khôi phục phiên làm việc, truy xuất thông tin người dùng hiện tại, đồng bộ danh sách thiết bị đã liên kết, hiển thị dữ liệu sức khỏe mới nhất, tra cứu dữ liệu lịch sử và gửi yêu cầu đo ECG. Về mặt kiến trúc, ứng dụng cần được tổ chức theo hướng tách biệt các tầng chức năng như tầng giao tiếp API, tầng lưu trữ phía client, tầng mô hình dữ liệu nghiệp vụ và tầng quản lý trạng thái giao diện Cách tổ chức này nhằm bảo đảm tính rõ ràng trong thiết kế, thuận lợi cho bảo trì, kiểm thử và mở rộng chức năng về sau.

Đề tài cũng đặt ra mục tiêu xây dựng và kiểm chứng luồng đo ECG theo yêu cầu như một chức năng đặc trưng của hệ thống. Khác với mô hình chỉ tiếp nhận dữ liệu đo định kỳ từ thiết bị, hệ thống phải hỗ trợ cơ chế tương tác hai chiều giữa ứng dụng Web và thiết bị. Theo đó, người dùng có thể gửi yêu cầu đo ECG thông qua giao diện Web đến backend; backend ghi nhận yêu cầu và tạo lệnh tương ứng trong hàng đợi xử lý của thiết bị; thiết bị ESP32 định kỳ kiểm tra lệnh, thực hiện phép đo và gửi tín hiệu phản hồi kèm dữ liệu kết quả về hệ thống. Cơ chế này cho phép hệ thống không chỉ giám sát thụ động mà còn hỗ trợ điều khiển tác vụ đo từ xa, qua đó nâng cao tính thực tiễn và giá trị ứng dụng của giải pháp.

Cuối cùng, hệ thống cần được xây dựng theo hướng có thể triển khai và vận hành thực tế chứ không chỉ dừng lại ở mức nguyên mẫu chức năng. Cụ thể, toàn bộ các thành phần chính của hệ thống phải được tổ chức để có thể chạy trong môi trường Docker Compose, bao gồm backend, cơ sở dữ liệu MongoDB, Redis và các dịch vụ hỗ trợ liên quan. Hệ thống cũng cần có các cơ chế phục vụ giám sát và bảo trì như kiểm tra tình trạng hoạt động, kiểm tra khả năng sẵn sàng, thu thập thông tin vận hành cơ bản, cũng như hỗ trợ sao lưu và khôi phục dữ liệu. Mục tiêu này nhằm bảo đảm hệ thống có tính ổn định, dễ triển khai, dễ kiểm tra và tạo nền tảng cho việc mở rộng trong các giai đoạn phát triển tiếp theo trên nền tảng Web.

1.2.3 Kết quả mong đợi

Kết quả mong đợi của đề tài là xây dựng được một nguyên mẫu hệ thống theo dõi sức khỏe từ xa có khả năng hoạt động thực tế ở mức cơ bản, bảo đảm sự phối hợp giữa thiết bị phần cứng, máy chủ xử lý trung tâm và ứng dụng Web. Nguyên mẫu này cần thể hiện được đầy đủ các luồng chức năng chính đã đặt ra trong phạm vi nghiên cứu và triển khai của đề tài.

Cụ thể, ở lớp thiết bị, hệ thống được kỳ vọng cho phép thiết bị ESP32 thực hiện việc thu thập dữ liệu sức khỏe, truyền dữ liệu về backend theo đúng định dạng quy định và tiếp nhận các lệnh điều khiển từ máy chủ khi có yêu cầu. Điều này thể hiện rằng thiết bị không chỉ đóng vai trò là nguồn phát sinh dữ liệu mà còn tham gia vào cơ chế tương tác hai chiều với hệ thống.

Ở lớp xử lý trung tâm, backend xây dựng bằng FastAPI được kỳ vọng có khả năng tiếp nhận dữ liệu gửi lên từ thiết bị, xử lý dữ liệu đầu vào, lưu trữ dữ liệu trong cơ sở dữ liệu và cung cấp các giao diện lập trình ứng dụng (API) phục vụ cho việc truy vấn thông tin từ phía ứng dụng Web. Các API này cần đáp ứng được nhu cầu lấy dữ liệu sức khỏe mới nhất, dữ liệu lịch sử, dữ liệu ECG và các thông tin liên quan đến người dùng hoặc thiết bị, từ đó bảo đảm vai trò trung tâm điều phối của backend trong toàn bộ hệ thống.

Ở lớp ứng dụng, ứng dụng Web được kỳ vọng hỗ trợ người dùng trong việc đăng nhập, duy trì và khôi phục phiên làm việc thông qua trình duyệt, đồng thời hiển thị thông tin sức khỏe dưới các góc nhìn phù hợp với mô hình dữ liệu của hệ thống. Cụ thể, ứng dụng cần có khả năng trình bày dữ liệu theo người dùng hoặc theo thiết bị, giúp người sử dụng dễ dàng theo dõi trạng thái sức khỏe, tra cứu lịch sử và tương tác với các chức năng giám sát từ xa.

Bên cạnh các chức năng theo dõi cơ bản, một kết quả quan trọng được mong đợi là hệ thống phải hỗ trợ được cơ chế ECG on-demand. Nghĩa là từ ứng dụng Web, người dùng có thể gửi yêu cầu đo ECG; backend tiếp nhận và phát sinh lệnh điều khiển tương ứng; thiết bị nhận lệnh, thực hiện tác vụ đo và phản hồi kết quả về hệ thống. Việc hiện thực thành công luồng chức năng này cho thấy hệ thống có khả năng hỗ trợ tương tác hai chiều giữa người dùng và thiết bị, vượt ra ngoài mô hình giám sát thụ động thông thường.

Ngoài ra, nguyên mẫu hệ thống cũng được kỳ vọng tích hợp các thành phần kỹ thuật cần thiết để phục vụ quá trình triển khai thử nghiệm. Các thành phần này bao gồm cơ chế bảo mật cơ bản trong xác thực và quản lý truy cập, cơ chế giới hạn tần suất yêu cầu nhằm giảm nguy cơ lạm dụng tài nguyên hệ thống, cũng như các chức năng giám sát vận hành ở mức cơ bản để hỗ trợ kiểm tra trạng thái và đánh giá khả năng hoạt động của toàn bộ hệ thống trong môi trường triển khai thực tế.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là một hệ thống theo dõi sức khỏe từ xa được xây dựng theo mô hình client–server–IoT, trong đó các thành phần phần cứng, phần mềm và hạ tầng truyền thông phối hợp với nhau để thực hiện chức năng thu thập, truyền nhận, xử lý, lưu trữ và hiển thị dữ liệu sức khỏe. Trong phạm vi nghiên cứu của đề tài, các đối tượng chính được xem xét bao gồm thiết bị nhúng, hệ thống backend, ứng dụng Web và cơ chế giao tiếp giữa các thành phần trong toàn bộ hệ thống.

Trước hết, một đối tượng nghiên cứu quan trọng là thiết bị ESP32 đóng vai trò là nút IoT đầu cuối. Thiết bị này có nhiệm vụ thu thập dữ liệu sức khỏe từ cảm biến hoặc từ mô-đun đo liên kết, đóng gói dữ liệu theo định dạng quy định và gửi dữ liệu về backend thông qua kết nối mạng. Bên cạnh chức năng gửi dữ liệu, thiết bị còn là đối tượng nghiên cứu trong cơ chế tiếp nhận lệnh điều khiển từ máy chủ, đặc biệt trong các tình huống yêu cầu đo dữ liệu theo tác vụ, chẳng hạn như ECG on-demand. Vì vậy, thiết bị không chỉ được xem xét ở góc độ phát sinh dữ liệu mà còn ở khả năng tương tác hai chiều với hệ thống.

Đối tượng nghiên cứu tiếp theo là backend của hệ thống, được xây dựng theo kiến trúc dịch vụ sử dụng FastAPI. Backend giữ vai trò trung tâm trong việc tiếp nhận dữ liệu từ thiết bị, xử lý và chuẩn hóa dữ liệu đầu vào, tổ chức lưu trữ trong cơ sở dữ liệu, đồng thời cung cấp các API phục vụ cho ứng dụng Web và các tác nhân quản trị. Ngoài chức năng xử lý dữ liệu, backend còn là đối tượng nghiên cứu ở các khía cạnh xác thực, phân quyền, quản lý phiên đăng nhập, quản lý thiết bị, xử lý cảnh báo và điều phối các luồng nghiệp vụ giữa ứng dụng Web và thiết bị IoT.

Một đối tượng nghiên cứu khác là ứng dụng Web, đóng vai trò là giao diện tương tác chính của người dùng cuối với hệ thống thông qua trình duyệt. Ứng dụng được nghiên cứu ở phương diện tổ chức chức năng và hiển thị dữ liệu, bao gồm các khả năng như đăng ký, đăng nhập, duy trì phiên làm việc, đồng bộ thiết bị liên kết, hiển thị dữ liệu sức khỏe mới nhất, tra cứu lịch sử đo và gửi yêu cầu đo ECG. Đồng thời, ứng dụng cũng là đối tượng được xem xét về mặt kiến trúc phần mềm, cụ thể là cách tổ chức các lớp giao tiếp API, lưu trữ phía client (local storage hoặc session storage), mô hình nghiệp vụ và cơ chế quản lý trạng thái giao diện.

Ngoài các thành phần cốt lõi nêu trên, đề tài còn nghiên cứu cơ chế giao tiếp và tích hợp giữa các thành phần trong hệ thống. Nội dung này bao gồm giao tiếp giữa ESP32 và backend thông qua API hoặc token thiết bị (X-Device-Token), giao tiếp giữa ứng dụng Web và backend thông qua cơ chế xác thực JWT Bearer, cũng như các luồng truyền dữ liệu và phản hồi lệnh trong quá trình vận hành. Việc nghiên cứu cơ chế tích hợp này nhằm bảo đảm rằng hệ thống hoạt động đồng bộ, dữ liệu được trao đổi đúng ngữ nghĩa và các thành phần có thể phối hợp ổn định trong môi trường thực tế.

Bên cạnh đó, đề tài cũng xem xét môi trường triển khai và vận hành hệ thống như một phần của đối tượng nghiên cứu. Cụ thể, việc tổ chức các dịch vụ backend, cơ sở dữ liệu MongoDB, Redis và các thành phần hỗ trợ bằng Docker Compose là một nội dung cần được khảo sát nhằm đánh giá khả năng triển khai, giám sát, kiểm tra trạng thái hoạt động và mở rộng hệ thống trong tương lai trên nền tảng Web.

1.4 Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài được giới hạn nhằm phù hợp với thời gian thực hiện, nguồn lực triển khai và mục tiêu của đồ án, đồng thời vẫn bảo đảm xây dựng được một nguyên mẫu hệ thống có tính hoàn chỉnh về mặt chức năng và có khả năng vận hành ở mức cơ bản. Trong phạm vi đó, đề tài tập trung vào việc phân tích, thiết kế và xây dựng một hệ thống theo dõi sức khỏe từ xa theo mô hình client–server–IoT, bao gồm thiết bị ESP32, backend FastAPI, ứng dụng Web và môi trường triển khai hỗ trợ vận hành hệ thống.

Về mặt chức năng, đề tài tập trung nghiên cứu và hiện thực các chức năng cốt lõi của hệ thống, bao gồm thu thập dữ liệu sức khỏe từ thiết bị, truyền dữ liệu về máy chủ, lưu trữ dữ liệu trong cơ sở dữ liệu, cung cấp API cho ứng dụng Web truy vấn dữ liệu mới nhất và dữ liệu lịch sử, đồng thời hỗ trợ hiển thị dữ liệu sức khỏe trên giao diện người dùng thông qua trình duyệt. Bên cạnh đó, đề tài cũng giới hạn trong việc xây dựng một số chức năng quản lý cơ bản như quản lý người dùng, quản lý thiết bị liên kết, quản lý phiên đăng nhập, tiếp nhận và xử lý yêu cầu đo ECG theo yêu cầu.

Về mặt kỹ thuật, phạm vi nghiên cứu tập trung vào việc xây dựng cơ chế giao tiếp giữa các thành phần chính của hệ thống. Cụ thể, đề tài xem xét luồng trao đổi dữ liệu giữa thiết bị ESP32 và backend, luồng truy cập dịch vụ giữa ứng dụng Web và backend, cũng như cơ chế xác thực và phân quyền tương ứng cho từng loại tác nhân tham gia. Ngoài ra, đề tài chỉ hướng đến mức cập nhật dữ liệu gần thời gian thực ở mức cơ bản, phục vụ mục tiêu theo dõi và kiểm tra trạng thái hệ thống, chưa đi sâu vào các cơ chế truyền thông thời gian thực có yêu cầu cao về độ trễ hoặc khả năng chịu tải lớn.

Về mặt triển khai, đề tài giới hạn trong môi trường thử nghiệm có tổ chức bằng Docker Compose, bao gồm các thành phần cần thiết như backend, cơ sở dữ liệu MongoDB, Redis và các dịch vụ hỗ trợ liên quan. Việc triển khai trong phạm vi này nhằm phục vụ mục đích kiểm thử, đánh giá khả năng vận hành và kiểm chứng tính khả thi của giải pháp, chứ chưa đặt mục tiêu triển khai ở quy mô sản xuất hoặc thương mại hóa thực tế.

Bên cạnh những nội dung nằm trong phạm vi nghiên cứu, đề tài không đi sâu vào việc phát triển hoặc đánh giá độ chính xác y khoa của các thuật toán chẩn đoán, không nghiên cứu chuyên sâu về phần cứng cảm biến ở mức thiết kế mạch điện tử, không xử lý các bài toán tối ưu hiệu năng trên quy mô lớn và cũng chưa xem xét đầy đủ các yêu cầu pháp lý, tiêu chuẩn y tế hay bảo mật ở mức cao dành cho hệ thống triển khai trong môi trường thực tế. Các chức năng nâng cao như phân tích chuyên sâu dữ liệu sức khỏe, cảnh báo thông minh dựa trên mô hình học máy, đồng bộ đa nền tảng quy mô lớn hoặc tích hợp với các hệ thống y tế bên ngoài cũng nằm ngoài phạm vi của đồ án này trên nền tảng Web.

1.4.1 Phạm vi chức năng

Trong phạm vi của đề tài, hệ thống được giới hạn triển khai ở các chức năng cốt lõi, đủ để hình thành một nguyên mẫu theo dõi sức khỏe từ xa có khả năng vận hành tương đối hoàn chỉnh. Các chức năng này được xác định dựa trên kiến trúc và mã nguồn hiện tại của hệ thống, bao gồm cả phía backend và ứng dụng Web.

Trước hết, hệ thống hỗ trợ các chức năng liên quan đến tài khoản và phiên người dùng, bao gồm đăng ký tài khoản, đăng nhập, khôi phục phiên làm việc và quản lý trạng thái xác thực trong quá trình sử dụng. Đây là nhóm chức năng nền tảng, bảo đảm người dùng có thể truy cập hệ thống một cách hợp lệ và duy trì phiên làm việc khi tương tác với ứng dụng Web.

Bên cạnh đó, hệ thống hỗ trợ quản lý thiết bị liên kết với người dùng. Chức năng này cho phép người dùng đồng bộ danh sách thiết bị đã được liên kết, lựa chọn thiết bị đang theo dõi và làm cơ sở để truy vấn dữ liệu sức khỏe theo từng thiết bị cụ thể. Đây là một điểm quan trọng vì hệ thống hiện tại được tổ chức theo hướng device-centric, trong đó phần lớn dữ liệu và quyền truy cập đều gắn với thiết bị.

Ở lớp tiếp nhận dữ liệu, hệ thống hỗ trợ nhận dữ liệu sức khỏe từ thiết bị ESP32, sau đó thực hiện chuẩn hóa và lưu trữ dữ liệu vào **MongoDB**. Các dữ liệu được xử lý trong phạm vi chức năng của đề tài bao gồm các chỉ số sức khỏe cơ bản và dữ liệu ECG. Việc lưu trữ này là cơ sở cho các chức năng theo dõi thời gian thực, xem lại lịch sử và tạo cảnh báo tự động.

Sau khi dữ liệu được lưu trữ, hệ thống cho phép truy vấn dữ liệu sức khỏe mới nhất và dữ liệu lịch sử, phục vụ nhu cầu theo dõi trạng thái hiện tại cũng như xem lại diễn biến chỉ số theo thời gian. Ngoài ra, hệ thống còn hỗ trợ truy vấn dữ liệu ECG và bản tóm tắt sức khỏe, giúp người dùng có thể tiếp cận thông tin sức khỏe ở cả mức chi tiết và mức tổng hợp.

Một chức năng đặc trưng khác trong phạm vi của đề tài là tạo và xử lý yêu cầu **ECG on-demand**. Thay vì chỉ tiếp nhận dữ liệu đo định kỳ, hệ thống còn cho phép ứng dụng Web gửi yêu cầu đo ECG, backend tạo command tương ứng và thiết bị thực hiện tác vụ khi nhận được lệnh. Cơ chế này làm cho hệ thống có khả năng tương tác hai chiều giữa ứng dụng Web và thiết bị.

Bên cạnh đó, hệ thống còn hỗ trợ sinh cảnh báo tự động theo ngưỡng khi dữ liệu sức khỏe vượt quá các giá trị được cấu hình. Chức năng này giúp tăng giá trị sử dụng thực tế của hệ thống, vì dữ liệu không chỉ được lưu trữ và hiển thị mà còn được dùng để phát hiện các trạng thái bất thường ở mức cơ bản.

Cuối cùng, ở góc độ vận hành, hệ thống cung cấp các chức năng hỗ trợ kiểm tra trạng thái hoạt động như health check và cơ chế rate limit cơ bản để bảo vệ API. Đây là các chức năng không trực tiếp phục vụ nghiệp vụ người

dùng, nhưng có ý nghĩa quan trọng trong việc bảo đảm hệ thống có thể vận hành ổn định trong môi trường thử nghiệm.

1.4.2 Phạm vi công nghệ

Phạm vi công nghệ của đề tài được giới hạn theo đúng các công nghệ đang sử dụng trong hệ thống hiện thực, nhằm bảo đảm báo cáo bám sát kiến trúc triển khai thực tế và phản ánh đúng giải pháp đã xây dựng. Việc giới hạn này giúp tránh mở rộng sang các công nghệ không xuất hiện trong mã nguồn hoặc không tham gia trực tiếp vào vận hành hệ thống.

Cụ thể, ở lớp thiết bị đầu cuối, đề tài sử dụng **ESP32** làm nền tảng phần cứng cho khối thu thập và truyền dữ liệu sức khỏe. Ở lớp xử lý trung tâm, **FastAPI** được dùng để xây dựng backend, tổ chức các dịch vụ API và xử lý các nghiệp vụ chính của hệ thống. Về lưu trữ dữ liệu, hệ thống sử dụng **MongoDB** làm cơ sở dữ liệu chính để quản lý thông tin người dùng, thiết bị và dữ liệu sức khỏe. Bên cạnh đó, **Redis** được sử dụng cho các chức năng hỗ trợ hạ tầng, đặc biệt là cơ chế rate limiting nhằm kiểm soát tần suất truy cập API.

Ở lớp ứng dụng người dùng, đề tài sử dụng công nghệ Web (ví dụ: **ReactJS**) để phát triển giao diện người dùng, phục vụ các chức năng tương tác và kết nối với backend thông qua trình duyệt. Về môi trường triển khai, **Docker Compose** được dùng để tổ chức và vận hành các thành phần như backend, MongoDB và Redis trong cùng một môi trường chạy.

1.4.3 Phạm vi người dùng

Phạm vi người dùng của đề tài được giới hạn ở các tác nhân đang tham gia trực tiếp vào quá trình vận hành hệ thống trong phiên bản hiện tại, nhằm bảo đảm việc phân tích, thiết kế và triển khai bám sát mô hình sử dụng thực tế.

Trong phạm vi này, hệ thống gồm ba nhóm tác nhân chính. Thứ nhất là quản trị viên hệ thống, có vai trò quản lý tài khoản, thiết bị, cấu hình và hỗ trợ duy trì hoạt động chung của hệ thống. Thứ hai là người dùng ứng dụng Web, bao gồm người được theo dõi hoặc người chăm sóc, với các chức năng như đăng nhập, đồng bộ thiết bị, xem dữ liệu sức khỏe, tra cứu lịch sử và gửi một số yêu cầu nghiệp vụ thông qua trình duyệt. Thứ ba là thiết bị **ESP32**, được xem như một tác nhân kỹ thuật, tham gia vào hệ thống thông qua việc gửi dữ liệu sức khỏe, nhận lệnh điều khiển và phản hồi kết quả xử lý.

Ngoài các nhóm trên, đề tài chưa mở rộng đến các đối tượng như bác sĩ, cơ sở y tế, đơn vị giám sát tổ chức hoặc các hệ thống bên thứ ba. Việc xác định phạm vi người dùng như vậy giúp báo cáo bám sát hiện trạng triển khai, đồng thời làm rõ cơ sở xây dựng các mô hình chức năng và cơ chế phân quyền của hệ thống.

1.4.4 Phạm vi dữ liệu

Phạm vi dữ liệu của đề tài được giới hạn ở các loại dữ liệu sức khỏe cơ bản mà hệ thống hiện tại có khả năng thu thập, truyền nhận, lưu trữ, truy vấn và hiển thị. Cách xác định này giúp đề tài bám sát chức năng thực tế của hệ thống và phù hợp với mục tiêu xây dựng nguyên mẫu theo dõi sức khỏe từ xa.

Cụ thể, hệ thống tập trung xử lý các chỉ số chính gồm nhịp tim, SpO2, nhiệt độ, nhịp thở và dữ liệu ECG. Các dữ liệu này được thiết bị thu thập, gửi về backend và phục vụ cho việc theo dõi, tra cứu lịch sử, hiển thị trên web di động và cảnh báo theo ngưỡng ở mức cơ bản.

Tuy nhiên, đề tài không đi sâu vào phân tích y khoa chuyên sâu và không nhằm thay thế các thiết bị chẩn đoán chuyên dụng. Hệ thống cũng chưa mở rộng sang các hướng xử lý nâng cao như dự đoán bệnh lý, phân loại tình trạng sức khỏe bằng AI hay hỗ trợ chẩn đoán lâm sàng.

1.4.5 Những nội dung ngoài phạm vi đề tài

Để bảo đảm phù hợp với thời gian, nguồn lực và mục tiêu của đồ án, đề tài không mở rộng sang một số nội dung ngoài phạm vi hiện thực chính của hệ thống. Việc giới hạn này giúp tập trung vào các chức năng cốt lõi của nguyên mẫu và bảo đảm tính khả thi trong quá trình triển khai.

Cụ thể, đề tài không đi sâu vào **thiết kế phần cứng cảm biến y tế chuyên sâu**, không bao gồm **kiểm định hay chứng nhận độ chính xác y khoa** của thiết bị. Bên cạnh đó, đề tài cũng không hướng đến **phân tích dữ liệu ECG ở mức lâm sàng** hoặc hỗ trợ **chẩn đoán bệnh**.

Về hạ tầng, đề tài chưa mở rộng đến **hệ thống thông báo đẩy ở quy mô production**, **triển khai cloud hoàn chỉnh với autoscaling**, **message broker chuyên dụng** hay **dashboard giám sát production đầy đủ**. Các cơ chế hiện tại chủ yếu phục vụ vận hành thử nghiệm và kiểm tra nguyên mẫu.

1.5 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu để bảo đảm vừa có cơ sở lý thuyết, vừa có triển khai thực nghiệm.

1.5.1 Phương pháp nghiên cứu lý thuyết

Đề tài tiến hành nghiên cứu các cơ sở lý thuyết liên quan trực tiếp đến bài toán xây dựng hệ thống theo dõi sức khỏe từ xa, làm nền tảng cho quá trình phân tích, thiết kế và triển khai hệ thống.

Nội dung nghiên cứu tập trung vào mô hình theo dõi sức khỏe từ xa, nguyên lý IoT và thiết bị nhúng trong y tế, kiến trúc client-server, mô hình REST

API, cùng với các cơ chế xác thực và quản lý phiên như **JWT** và **refresh token**. Bên cạnh đó, đề tài cũng khảo sát các công nghệ hiện thực đang sử dụng như **FastAPI, ReactJS, MongoDB, Redis** và **Docker Compose** để làm rõ vai trò và phạm vi ứng dụng của từng thành phần trong hệ thống.

Nhìn chung, phương pháp nghiên cứu lý thuyết giúp đề tài xây dựng cơ sở khoa học cho việc lựa chọn mô hình, công nghệ và cách tổ chức hệ thống phù hợp với mục tiêu của đồ án.

1.5.2 Phương pháp phân tích yêu cầu

Từ bài toán xây dựng hệ thống theo dõi sức khỏe từ xa, đề tài tiến hành phân tích yêu cầu theo hai nhóm chính là yêu cầu chức năng và yêu cầu phi chức năng. Quá trình này nhằm xác định rõ các chức năng hệ thống cần cung cấp, các tác nhân tham gia và các điều kiện kỹ thuật cần đáp ứng trong quá trình triển khai.

Trước hết, đề tài xác định các tác nhân chính của hệ thống gồm người dùng ứng dụng Web, người chăm sóc, quản trị viên và thiết bị ESP32. Trên cơ sở đó, các yêu cầu chức năng được xác định như đăng nhập, quản lý phiên, theo dõi dữ liệu sức khỏe, quản lý thiết bị, gửi yêu cầu đo ECG, tiếp nhận dữ liệu từ thiết bị và thực hiện các chức năng quản trị.

Bên cạnh đó, đề tài cũng phân tích các yêu cầu phi chức năng như xác thực, phân quyền, quản lý phiên, tính nhất quán dữ liệu, bảo mật, giới hạn truy cập và khả năng triển khai bằng Docker Compose. Sau khi phân tích, các yêu cầu này được đối chiếu với các thành phần hiện thực trong mã nguồn backend và ứng dụng Web nhằm bảo đảm sự thống nhất giữa phần phân tích và phần triển khai.

1.5.3 Phương pháp thiết kế hệ thống

Sau khi xác định và phân tích yêu cầu, đề tài tiến hành thiết kế hệ thống theo **kiến trúc nhiều lớp** nhằm bảo đảm tính phân tách chức năng, tính rõ ràng trong tổ chức thành phần và thuận lợi cho việc mở rộng. Hệ thống gồm bốn lớp chính: **lớp thiết bị IoT, lớp backend API, lớp cơ sở dữ liệu và**, **lớp ứng dụng Web.**

Trong đó, thiết bị IoT đảm nhiệm việc thu thập và gửi dữ liệu sức khỏe, backend thực hiện xử lý nghiệp vụ và cung cấp API, cơ sở dữ liệu lưu trữ thông tin hệ thống, còn ứng dụng Web hỗ trợ người dùng theo dõi dữ liệu và thực hiện các thao tác cần thiết. Bên cạnh cấu trúc nhiều lớp, đề tài cũng thiết kế các **luồng dữ liệu** và **luồng xác thực** bám sát cách tổ chức thực tế của hệ thống.

Cụ thể, ứng dụng sử dụng **JWT Bearer kết hợp refresh token** để xác thực và duy trì phiên làm việc, trong khi thiết bị sử dụng **X-Device-Token** khi giao tiếp với backend. Về lưu trữ, dữ liệu sức khỏe được tổ chức trong **health_readings**, lệnh điều khiển ECG trong **device_commands**, và thông tin

phiên trong **auth_sessions**. Cách thiết kế này giúp hệ thống vừa phản ánh đúng hiện trạng triển khai, vừa tạo cơ sở rõ ràng cho các bước hiện thực và kiểm thử tiếp theo.

1.5.4 Phương pháp triển khai thực nghiệm

Phương pháp triển khai thực nghiệm của đề tài được thực hiện trực tiếp trên các thành phần hệ thống đã được xây dựng thực tế, nhằm bảo đảm nội dung báo cáo phản ánh đúng hiện trạng triển khai và cho phép đối chiếu giữa phân tích, thiết kế và kết quả hiện thực.

Cụ thể, backend được triển khai bằng **FastAPI** và vận hành cùng **MongoDB, Redis** và **Docker Compose** để kiểm chứng khả năng tiếp nhận dữ liệu, xử lý nghiệp vụ, lưu trữ, quản lý phiên xác thực và cung cấp API cho ứng dụng Web. Ở phía ứng dụng, hệ thống được phát triển bằng **ReactJS** với cách tổ chức tách biệt giữa các thành phần xử lý nghiệp vụ và quản lý trạng thái, qua đó hỗ trợ kiểm chứng các chức năng như đăng nhập, đồng bộ thiết bị, xem dữ liệu sức khỏe, lịch sử và xử lý yêu cầu ECG.

Bên cạnh đó, đề tài cũng tiến hành kiểm thử giao diện và trạng thái hoạt động của ứng dụng nhằm đánh giá các màn hình chính, luồng chuyển trạng thái và khả năng phản hồi trong các tình huống sử dụng cơ bản. Thông qua phương pháp này, đề tài có cơ sở xác nhận tính khả thi, mức độ hoàn chỉnh và giá trị thực tiễn của hệ thống đã xây dựng.

1.5.5 Phương pháp kiểm thử và đánh giá

Sau khi hoàn thành triển khai, hệ thống được kiểm thử theo các luồng chức năng chính nhằm xác định mức độ hoạt động đúng đắn và sự phù hợp với các yêu cầu đã đặt ra. Nội dung kiểm thử tập trung vào các chức năng như đăng nhập và quản lý phiên, tải danh sách thiết bị, xem dữ liệu sức khỏe, xử lý các trạng thái biên và gửi yêu cầu đo ECG.

Bên cạnh đó, hệ thống còn được kiểm tra ở góc độ vận hành thông qua các nội dung như readiness, health check và log hệ thống, nhằm đánh giá khả năng hoạt động ổn định trong môi trường triển khai thử nghiệm. Trên cơ sở kết quả kiểm thử, hệ thống được đánh giá theo các tiêu chí chính gồm mức độ đáp ứng yêu cầu chức năng, tính đúng đắn của luồng dữ liệu, khả năng vận hành ổn định và tiềm năng mở rộng.

1.6 Ý nghĩa khoa học và thực tiễn của đề tài

1.6.1 Ý nghĩa khoa học

Về mặt khoa học, đề tài có ý nghĩa trong việc nghiên cứu và minh họa mô hình **hệ thống theo dõi sức khỏe từ xa** theo hướng tích hợp nhiều lớp công nghệ, bao gồm **thiết bị IoT, backend, cơ sở dữ liệu và ứng dụng web**. Cách

tiếp cận này giúp làm rõ mối quan hệ giữa các thành phần trong một hệ thống chăm sóc sức khỏe hiện đại.

Bên cạnh đó, đề tài còn có giá trị ở khía cạnh phân tích và hiện thực một số vấn đề kỹ thuật như **xác thực nhiều tác nhân, quản lý phiên bằng access token và refresh token, tổ chức dữ liệu sức khỏe trên cơ sở dữ liệu tài liệu, cơ chế polling queue cho yêu cầu ECG, rate limiting và health check**. Những nội dung này góp phần minh họa cách xây dựng một hệ thống IoT có khả năng vận hành ổn định và phù hợp với yêu cầu thực tế.

Nhìn chung, ý nghĩa khoa học của đề tài nằm ở việc cung cấp một mô hình tham khảo tương đối hoàn chỉnh cho bài toán theo dõi sức khỏe từ xa trên nền tảng IoT, làm cơ sở cho các nghiên cứu và phát triển tiếp theo trong cùng lĩnh vực.

1.6.2 Ý nghĩa thực tiễn

Về mặt thực tiễn, đề tài có ý nghĩa trong việc xây dựng một **nguyên mẫu hệ thống theo dõi sức khỏe từ xa** có khả năng đáp ứng các nhu cầu giám sát sức khỏe cơ bản. Hệ thống cho phép thu thập dữ liệu từ thiết bị, lưu trữ tập trung, truy vấn theo người dùng hoặc thiết bị và hiển thị thông tin trên ứng dụng web, đồng thời hỗ trợ yêu cầu đo ECG và cảnh báo theo ngưỡng cấu hình.

Đề tài đặc biệt phù hợp với các nhu cầu như **theo dõi sức khỏe tại nhà, hỗ trợ người cao tuổi, theo dõi sau điều trị và chăm sóc từ xa trong gia đình**. Bên cạnh giá trị ở mức nguyên mẫu, hệ thống còn tạo ra một nền tảng ban đầu để tiếp tục mở rộng thêm các chức năng như giao diện quản trị, thông báo đẩy, hạ tầng xử lý chuyên dụng và công cụ giám sát production.

Ý nghĩa thực tiễn của đề tài nằm ở khả năng xây dựng một hệ thống có thể vận hành, đáp ứng nhu cầu giám sát sức khỏe cơ bản và có tiềm năng phát triển thành giải pháp ứng dụng thực tế trong tương lai.

1.6.3 Ý nghĩa đối với học tập và phát triển kỹ năng

Đối với người thực hiện, đề tài có ý nghĩa trong việc rèn luyện năng lực chuyên môn theo hướng tổng hợp, thông qua toàn bộ quá trình **phân tích yêu cầu, thiết kế, triển khai, kiểm thử và đánh giá hệ thống**. Qua đó, người thực hiện không chỉ tiếp cận một kỹ thuật riêng lẻ mà còn có cái nhìn toàn diện về quy trình xây dựng một hệ thống phần mềm hoàn chỉnh.

Bên cạnh đó, đề tài giúp phát triển các kỹ năng quan trọng như **phân tích yêu cầu, thiết kế kiến trúc backend, xây dựng cơ sở dữ liệu, lập trình API, phát triển ứng dụng Web, quản lý xác thực và phiên làm việc, triển khai bằng Docker Compose, kiểm thử và đánh giá hệ thống**. Đây đều là những kỹ năng có giá trị thực tiễn cao trong học tập và công việc.

Nhìn chung, đề tài không chỉ mang lại kết quả ở mức sản phẩm, mà còn góp phần hình thành **tư duy kỹ thuật toàn diện**, giúp người thực hiện tích lũy

kiến thức, kinh nghiệm và kỹ năng phục vụ cho học tập, nghiên cứu và định hướng nghề nghiệp sau này.

CHƯƠNG 2
CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan về hệ thống theo dõi sức khỏe từ xa

Theo dõi sức khỏe từ xa là mô hình thu thập, truyền, lưu trữ và hiển thị các chỉ số sức khỏe của người dùng thông qua các thiết bị điện tử có kết nối mạng. Trong mô hình này, dữ liệu không còn bị giới hạn ở việc đo thủ công tại từng thời điểm, mà có thể được ghi nhận liên tục hoặc theo chu kỳ, sau đó được gửi về máy chủ để xử lý và cung cấp lại cho ứng dụng giám sát. Đây là một hướng phát triển quan trọng trong bối cảnh nhu cầu chăm sóc sức khỏe cá nhân, chăm sóc người cao tuổi và giám sát bệnh nhân tại nhà ngày càng tăng. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024)

Về bản chất, hệ thống theo dõi sức khỏe từ xa là một hệ thống tích hợp đa thành phần. Nó thường gồm thiết bị cảm biến hoặc thiết bị nhúng để thu nhận dữ liệu, một máy chủ trung tâm để tiếp nhận và xử lý dữ liệu, và một ứng dụng phía người dùng để hiển thị và theo dõi. Trong đề tài này, kiến trúc đó được hiện thực rõ ràng trong Git: thiết bị ESP32 gửi dữ liệu sức khỏe và nhận command ECG; backend FastAPI tiếp nhận dữ liệu, lưu MongoDB, kiểm tra ngưỡng cảnh báo và cung cấp API; còn ứng dụng Web lấy dữ liệu latest, history, summary và ECG để hiển thị cho người dùng.

Theo source backend hiện tại, hệ thống server có các mục tiêu rất cụ thể: nhận dữ liệu sức khỏe từ thiết bị đeo như nhịp tim, SpO2, nhiệt độ, nhịp thở và ECG; chuẩn hóa và lưu trữ dữ liệu vào MongoDB; phân tích ngưỡng để tạo cảnh báo tự động; đồng thời cung cấp API cho app/web truy vấn dữ liệu realtime và dữ liệu lịch sử. Ngoài luồng từ thiết bị lên server, hệ thống còn có luồng điều khiển ngược từ app đến thiết bị thông qua command ECG on-demand, cho thấy đây không phải hệ thống một chiều đơn thuần mà là hệ thống tương tác hai chiều giữa phần mềm và phần cứng.

So với các mô hình IoT cơ bản chỉ dừng ở việc gửi số liệu cảm biến lên cloud, hệ thống của đề tài có mức hoàn thiện cao hơn ở vài điểm. Thứ nhất, backend đã có tách biệt rõ giữa app/admin và thiết bị, mỗi bên dùng một hình thức xác thực riêng. Thứ hai, hệ thống có quản lý phiên người dùng bằng JWT và refresh token, thay vì chỉ cấp một access token đơn giản. Thứ ba, hệ thống có command queue cho ECG và background recovery loop cho command bị timeout. Thứ tư, hệ thống có các endpoint vận hành như /live, /ready, /health, /metrics, đồng thời có Redis-backed rate limit để kiểm soát truy cập. Những đặc điểm đó cho thấy đề tài không chỉ có ý nghĩa mô phỏng mà còn có giá trị như một nguyên mẫu hệ thống thực tế.

Từ góc độ học thuật, hệ thống theo dõi sức khỏe từ xa là một bài toán phù hợp để nghiên cứu nhiều chủ đề cùng lúc, bao gồm: IoT, giao tiếp REST, xác thực nhiều tác nhân, lưu trữ dữ liệu thời gian thực, quản lý phiên người dùng, xử lý dữ liệu sức khỏe và thiết kế ứng dụng web. Đây cũng là lý do chương cơ sở lý thuyết của báo cáo cần không chỉ trình bày kiến thức nền chung mà còn phải gắn với cách hệ thống đang được hiện thực trong Git.

2.2 Internet of Things trong theo dõi sức khỏe

Internet of Things là mô hình kết nối các đối tượng vật lý với Internet để chúng có thể thu thập, truyền nhận và trao đổi dữ liệu. Trong lĩnh vực chăm sóc sức khỏe, IoT giúp tạo ra các thiết bị đeo hoặc thiết bị giám sát có khả năng ghi nhận liên tục các chỉ số sinh học của cơ thể và truyền dữ liệu về hệ thống xử lý trung tâm. (World Health Organization, 2025; U.S. Department of Health and Human Services, 2024; Espressif Systems, 2026)

Việc áp dụng IoT vào theo dõi sức khỏe mang lại nhiều lợi ích. Trước hết, nó cho phép đo lường liên tục hoặc định kỳ mà không phụ thuộc hoàn toàn vào thao tác thủ công của con người. Thứ hai, dữ liệu có thể được lưu trữ tập trung để phục vụ truy vấn, so sánh theo thời gian và phát hiện bất thường. Thứ ba, người thân hoặc người chăm sóc có thể theo dõi tình trạng của bệnh nhân từ xa thông qua ứng dụng.

Về mặt nguyên lý, mô hình IoT trong hệ thống gồm các bước chính:

- Thiết bị đo hoặc mô phỏng dữ liệu sức khỏe.
- Thiết bị đóng gói dữ liệu theo JSON schema chuẩn của backend.
- Thiết bị gửi dữ liệu lên server qua HTTP/HTTPS.
- Backend lưu dữ liệu, phân tích ngưỡng, tạo cảnh báo hoặc phục vụ truy vấn.
- Ứng dụng lấy dữ liệu qua API để hiển thị.
- Khi có yêu cầu đặc biệt như ECG on-demand, app gửi yêu cầu đến server, server tạo command, thiết bị poll và phản hồi kết quả.

Cách tổ chức này phù hợp với hệ thống IoT quy mô nhỏ đến vừa vì đơn giản trong triển khai, ít phụ thuộc vào broker phức tạp và dễ kiểm thử tích hợp. Trong repo backend cũng nêu rõ queue ECG đang dùng collection MongoDB device_commands thay vì Redis hoặc RabbitMQ, phù hợp với mục tiêu nguyên mẫu nhưng vẫn đủ rõ ràng về mặt kiến trúc.

2.3 Các chỉ số sức khỏe được sử dụng trong hệ thống

Trong hệ thống theo dõi sức khỏe từ xa, các chỉ số sức khỏe đóng vai trò trung tâm vì phản ánh trực tiếp tình trạng cơ bản của người dùng. Theo phạm vi hiện tại, hệ thống xử lý các dữ liệu chính gồm **nhịp tim, SpO2, nhiệt độ, nhịp thở và ECG**. Các dữ liệu này được thiết bị gửi về backend, lưu trữ trong hệ thống và hiển thị trên ứng dụng web theo thời gian thực, lịch sử hoặc theo các tác vụ chuyên biệt như đo ECG.

Trong đó, **nhịp tim, SpO2, nhiệt độ và nhịp thở** là các chỉ số sức khỏe cơ bản, hỗ trợ theo dõi trạng thái sinh lý và phát hiện bất thường ở mức nền tảng. Bên cạnh đó, **ECG** là dữ liệu đặc trưng hơn, phản ánh hoạt động điện của tim theo thời gian và được hệ thống hỗ trợ thu thập, lưu trữ, truy vấn và hiển thị, đặc biệt trong cơ chế đo theo yêu cầu.

Về mặt xử lý, backend thực hiện tiếp nhận, lưu trữ và hỗ trợ cảnh báo theo ngưỡng đối với các chỉ số sức khỏe, trong khi ứng dụng web đảm nhiệm việc hiển thị dữ liệu thời gian thực, dữ liệu lịch sử và kết quả ECG. Việc lựa chọn các chỉ số này giúp hệ thống phản ánh tình trạng sức khỏe ở mức tương đối toàn diện trong phạm vi đề tài, đồng thời tạo cơ sở cho việc mở rộng các chức năng phân tích sâu hơn trong tương lai.

2.3.1 Nhịp tim

Nhịp tim là số lần tim đập trong một phút. Đây là một trong những chỉ số sinh tồn cơ bản, thường được dùng để đánh giá tình trạng hoạt động của hệ tim mạch và phản ứng cơ thể đối với vận động, căng thẳng hoặc bệnh lý.

2.3.2 Nồng độ oxy trong máu

SpO2 là chỉ số phản ánh mức độ bão hòa oxy trong máu ngoại vi. Đây là thông số đặc biệt quan trọng đối với người có bệnh lý hô hấp hoặc tuần hoàn.

2.3.3 Nhiệt độ cơ thể

Nhiệt độ là một chỉ số cơ bản giúp phát hiện sốt, rối loạn thân nhiệt hoặc những biến đổi trạng thái cơ thể.

2.3.4 Nhịp thở

Nhịp thở phản ánh số lần hít thở trong một phút. Đây là chỉ số quan trọng trong đánh giá chức năng hô hấp.

2.3.5 Điện tim

ECG là tín hiệu điện sinh học phản ánh hoạt động điện của tim. So với các chỉ số: app gửi yêu cầu, backend tạo command, thiết bị thực hiện và gửi lại dữ liệu. Vì vậy, ECG là phần dữ liệu nổi bật nhất của đề tài về mặt kiến trúc và luồng xử lý.

2.3.6 Vai trò của các chỉ số trong hệ thống

Trong hệ thống theo dõi sức khỏe từ xa, các chỉ số sức khỏe không chỉ được thu thập và lưu trữ mà còn tham gia trực tiếp vào nhiều chức năng nghiệp vụ quan trọng. Theo thiết kế hiện tại, dữ liệu sức khỏe là đầu vào cho các chức năng **hiển thị dữ liệu mới nhất, truy vấn lịch sử, tổng hợp thống kê, sinh cảnh báo theo ngưỡng** và hỗ trợ các tác vụ như **đo ECG theo yêu cầu**.

Trước hết, các chỉ số sức khỏe được sử dụng để hiển thị **trạng thái hiện tại** của người dùng thông qua dữ liệu mới nhất từ thiết bị. Bên cạnh đó, hệ thống còn hỗ trợ truy vấn **dữ liệu lịch sử** theo thời gian, giúp người dùng theo dõi sự thay đổi của các chỉ số trong từng giai đoạn khác nhau. Ngoài ra, dữ liệu sức khỏe còn được dùng để tạo **summary sức khỏe**, bao gồm các giá trị trung bình, nhỏ nhất và lớn nhất của một số chỉ số quan trọng.

Một vai trò quan trọng khác là hỗ trợ **cảnh báo theo ngưỡng cấu hình**. Sau khi dữ liệu được tiếp nhận và lưu trữ, hệ thống sẽ kiểm tra các ngưỡng cảnh báo liên quan đến SpO2, nhiệt độ, nhịp tim, nhịp thở và chất lượng ECG để phát hiện bất thường ở mức cơ bản. Đối với **ECG**, dữ liệu này vừa được lưu trữ và hiển thị như một bản ghi sức khỏe đặc biệt, vừa tham gia vào luồng xử lý đo theo yêu cầu giữa ứng dụng, backend và thiết bị.

2.4 Kiến trúc client-server và REST API

2.4.1 Khái niệm client-server

Kiến trúc client-server là mô hình trong đó client gửi yêu cầu hoặc dữ liệu đến server, còn server đảm nhiệm việc tiếp nhận, xác thực, xử lý nghiệp vụ, truy xuất dữ liệu và trả về kết quả. Mô hình này giúp tách biệt rõ giữa nơi phát sinh yêu cầu và nơi xử lý trung tâm, từ đó tăng tính nhất quán dữ liệu, thuận lợi cho bảo trì, mở rộng và kiểm soát bảo mật. (Fielding, 2000)

Trong các hệ thống web và IoT, kiến trúc client-server đặc biệt phù hợp vì cho phép nhiều loại tác nhân cùng giao tiếp với một trung tâm xử lý thống nhất. Thay vì mỗi thành phần tự xử lý độc lập, toàn bộ yêu cầu được đưa về server để kiểm tra, chuẩn hóa, lưu trữ và cung cấp lại thông qua các giao diện chung.

Trong phạm vi đề tài, mô hình này được thể hiện rõ qua việc **ESP32**, **ứng dụng Web** và **công cụ admin** đóng vai trò client, trong khi **backend FastAPI** là server trung tâm. Thiết bị ESP32 gửi dữ liệu sức khỏe và nhận lệnh điều khiển như đo ECG; ứng dụng web gửi các yêu cầu như đăng nhập, truy vấn thiết bị, dữ liệu mới nhất, lịch sử, cảnh báo và ECG; công cụ admin phục vụ quản trị người dùng, thiết bị và vận hành hệ thống.

Ở phía server, **FastAPI** tiếp nhận yêu cầu từ các client, thực hiện xác thực theo từng loại tác nhân, xử lý nghiệp vụ, truy cập cơ sở dữ liệu và trả kết quả phù hợp. Bên cạnh đó, **MongoDB** và **Redis** là các thành phần hạ tầng hỗ trợ cho server, lần lượt phục vụ lưu trữ dữ liệu và kiểm soát truy cập.

2.4.2 REST API

REST API là phong cách thiết kế giao diện lập trình ứng dụng dựa trên giao thức HTTP, trong đó các tài nguyên được truy cập thông qua các URL định danh rõ ràng và được thao tác bằng các phương thức chuẩn như GET, POST, PATCH, DELETE. Trong mô hình này, client gửi yêu cầu đến đúng endpoint với dữ liệu phù hợp, còn server thực hiện xác thực, xử lý nghiệp vụ, truy xuất dữ liệu và trả về phản hồi chuẩn hóa. Cách tiếp cận này phù hợp với các hệ thống hiện đại vì rõ ràng, dễ mở rộng và thuận lợi cho tích hợp nhiều loại client. (Fielding, 2000)

Trong đề tài này, REST API là cơ chế giao tiếp thống nhất giữa các thành phần của hệ thống. **ESP32** sử dụng REST qua **HTTPS** kết hợp **X-Device-Token** để gửi dữ liệu sức khỏe và nhận lệnh điều khiển, trong khi **ứng dụng Web /web/admin** sử dụng REST qua **HTTPS** kết hợp **Authorization: Bearer**

<JWT> để thực hiện đăng nhập, truy xuất dữ liệu, quản lý phiên và gửi các yêu cầu nghiệp vụ như đo ECG.

Các endpoint REST của hệ thống được tổ chức thành nhiều nhóm rõ ràng. Thứ nhất là nhóm **API cho app và admin** dưới tiền tố **/api/v1/**, bao gồm các chức năng xác thực, quản lý người dùng, quản lý thiết bị, truy vấn dữ liệu sức khỏe, xem cảnh báo và yêu cầu đo ECG. Thứ hai là nhóm **API dành cho thiết bị ESP32** dưới tiền tố **/api/v1/esp/**, phục vụ các tác vụ gửi readings, nhận command tiếp theo và gửi ACK sau khi thực thi. Việc tách riêng nhóm endpoint cho thiết bị giúp kiểm soát xác thực, rate limit và logic xử lý phù hợp với đặc thù của client IoT.

Ngoài các API nghiệp vụ, hệ thống còn cung cấp các endpoint như **/live, /ready, /health** để kiểm tra trạng thái hoạt động của backend và các phụ thuộc liên quan. Bên cạnh đó, hệ thống cũng hỗ trợ **/metrics** để phục vụ giám sát theo kiểu Prometheus-style khi được cấu hình cho phép. Điều này cho thấy REST API không chỉ phục vụ trao đổi dữ liệu nghiệp vụ mà còn hỗ trợ quan sát và vận hành hệ thống.

Về nội dung chức năng, REST API hiện hỗ trợ các nhóm chính gồm **xác thực** (đăng ký, đăng nhập, refresh, logout, lấy thông tin người dùng), **quản lý thiết bị và quyền truy cập** (đăng ký thiết bị, liên kết thiết bị, quản lý viewers, cập nhật ngưỡng, xoay token ESP), **dữ liệu sức khỏe** (dữ liệu mới nhất, lịch sử, ECG, summary, alerts) và **điều khiển tác vụ** như yêu cầu đo ECG hoặc hủy lệnh. Với riêng thiết bị ESP32, các API chính gồm gửi readings, lấy command tiếp theo và gửi ACK, tạo thành một vòng giao tiếp REST khép kín giữa thiết bị và backend.

2.5 Xác thực và quản lý phiên bằng JWT, refresh token

2.5.1 JWT trong hệ thống

JWT là một loại token xác thực thường được sử dụng trong các hệ thống client-server để cho phép người dùng truy cập API. Trong hệ thống này, JWT được áp dụng cho ứng dụng Web, web app và admin tool, trong đó client gửi access token qua header Authorization: Bearer <access_token> khi gọi các endpoint nghiệp vụ của backend. (Jones et al., 2015)

Sau khi người dùng đăng nhập thành công, backend cấp **access token** để client sử dụng trong các request tiếp theo. Token này đóng vai trò là bằng chứng xác thực tạm thời, giúp backend nhận biết người dùng và kiểm soát quyền truy cập vào các tài nguyên của hệ thống. Cách tổ chức này phù hợp với mô hình **REST API**, vì client chỉ cần đính kèm token trong mỗi request mà không phải duy trì phiên theo cách truyền thống.

Việc sử dụng JWT mang lại ưu điểm như **gọn nhẹ, thuận tiện cho giao tiếp HTTP, dễ tích hợp với ứng dụng web và các route API hiện có**. Tuy nhiên, do access token thường có **thời gian sống ngắn** để tăng tính an toàn, hệ

thống không sử dụng JWT một cách độc lập mà kết hợp thêm **refresh token** để duy trì phiên đăng nhập.

2.5.2 Refresh token

Để khắc phục hạn chế của access token có thời gian sống ngắn, hệ thống sử dụng thêm refresh token nhằm duy trì phiên làm việc của người dùng theo hướng an toàn và thuận tiện hơn. Sau khi đăng nhập thành công, server trả về cả access token và refresh token; khi access token hết hạn, ứng dụng sẽ gọi API refresh để xin cấp lại cặp token mới. (Hardt, 2012; Sheffer et al., 2020)

Trong hệ thống hiện tại, refresh token là một **token dạng opaque** được sinh ngẫu nhiên, không phải JWT, và được lưu dưới dạng **hash** trong collection **auth_sessions**. Mỗi refresh token luôn gắn với một **phiên đăng nhập cụ thể**, giúp backend có thể quản lý, kiểm tra hiệu lực và kiểm soát vòng đời phiên làm việc.

Khi client gửi yêu cầu làm mới phiên, backend sẽ thực hiện **xoay vòng refresh token** bằng cách kiểm tra token cũ, tạo token mới và cập nhật lại thông tin phiên. Cơ chế này giúp tăng tính bảo mật, hạn chế rủi ro khi token cũ bị lộ hoặc bị tái sử dụng. Trong ứng dụng Web, luồng refresh cũng được tích hợp vào quá trình vận hành: nếu access token hết hạn, app sẽ tự động thử refresh; nếu thành công thì tiếp tục gửi lại request, còn nếu thất bại thì người dùng phải đăng nhập lại.

2.5.3 Quản lý session ở phía web

Ở phía Web, **session** không chỉ là trạng thái "đã có token" mà được tổ chức như một **trạng thái xác thực đầy đủ**, bao gồm **access token, refresh token, thông tin người dùng hiện tại, vai trò truy cập** và các trạng thái phục vụ giao diện. Trong thiết kế hiện tại, Cơ chế quản lý **session** ở phía frontend là thành phần trung tâm chịu trách nhiệm quản lý trạng thái phiên làm việc của người dùng. Thành phần này phối hợp với lớp dịch vụ API xác thực để giao tiếp với backend, sử dụng cơ chế lưu trữ cục bộ để lưu dữ liệu phiên và sử dụng cấu trúc dữ liệu phù hợp để biểu diễn cặp token xác thực.

Khi ứng dụng web khởi động, session được khôi phục thông qua cơ chế **bootstrap**. Ứng dụng web đọc token đã lưu, tải lại thông tin người dùng và đồng bộ dữ liệu hiện tại từ backend nếu phiên còn hiệu lực. Khi người dùng đăng nhập, web không chỉ nhận token mà còn lấy thêm thông tin người dùng để cập nhật đầy đủ trạng thái xác thực.

Ở tầng lưu trữ, session được duy trì qua các lần mở ứng dụng nhờ việc lưu **access token** và **refresh token** vào bộ nhớ cục bộ. Khi access token hết hạn, ứng dụng web sẽ tự động dùng **refresh token** để xin cấp token mới; nếu thành công thì cập nhật lại phiên và tiếp tục request, còn nếu thất bại thì xóa toàn bộ session và đưa người dùng về trạng thái chưa đăng nhập.

Ngoài chức năng xác thực, session còn ảnh hưởng trực tiếp đến **giao diện và quyền truy cập** trong ứng dụng, chẳng hạn quyết định hiển thị màn hình đăng nhập, danh sách thiết bị hay các chức năng dành riêng cho quản trị viên. Như vậy, quản lý session ở phía web được tổ chức khá đầy đủ, bảo đảm cả **tính liên tục của đăng nhập, khả năng đồng bộ với backend và hỗ trợ điều khiển giao diện theo trạng thái người dùng**.

2.5.4 Ý nghĩa của mô hình xác thực hiện tại

Bên cạnh cơ chế xác thực người dùng, hệ thống còn sử dụng **Device Token** để xác thực riêng cho thiết bị **ESP32**. Đây là cách tiếp cận phù hợp với bài toán IoT, vì thiết bị nhúng không thể áp dụng mô hình đăng nhập tương tác như người dùng trên ứng dụng. Trong hệ thống hiện tại, ESP32 gửi các request đến backend kèm header **X-Device-Token**, trong khi ứng dụng người dùng sử dụng **JWT Bearer**, cho thấy hai luồng xác thực được tách biệt rõ ràng.

Device Token được dùng khi thiết bị gửi dữ liệu sức khỏe, nhận lệnh điều khiển hoặc gửi phản hồi thực thi lệnh. Backend sẽ kiểm tra token này trước khi cho phép thiết bị truy cập các API dành riêng cho lớp IoT. Để tăng tính an toàn, token thiết bị không được lưu ở dạng thô trong cơ sở dữ liệu mà được **băm (hash)** trước khi lưu và đối chiếu khi xác thực. Cơ chế này giúp giảm rủi ro nếu dữ liệu lưu trữ bị lộ.

Việc xác thực bằng Device Token giúp bảo đảm **chỉ các thiết bị hợp lệ mới có thể gửi dữ liệu hoặc nhận lệnh từ hệ thống**, đồng thời ngăn chặn các client lạ giả mạo thiết bị. Bên cạnh đó, cơ chế này cũng thuận lợi cho việc **xoay token theo từng thiết bị** khi cần thay đổi cấu hình hoặc xử lý rủi ro bảo mật.

Nhìn chung, xác thực thiết bị bằng Device Token là một thành phần quan trọng trong kiến trúc bảo mật của hệ thống. Cách tổ chức này vừa phù hợp với đặc thù của thiết bị nhúng, vừa giúp backend kiểm soát đúng nguồn dữ liệu từ lớp IoT trong hệ thống theo dõi sức khỏe từ xa.

2.6 Xác thực thiết bị bằng Device Token

Ngoài xác thực người dùng, hệ thống còn phải xác thực thiết bị ESP32. Đây là một yêu cầu đặc thù của hệ thống IoT vì thiết bị không thể đăng nhập theo cùng mô hình như người dùng trên app.

Theo backend repo, API cho ESP bắt buộc dùng header X-Device-Token, và token thiết bị được hash theo công thức sha256(secret:token) trước khi lưu vào cơ sở dữ liệu. Việc không lưu token thô là một điểm bảo mật tốt, giúp hạn chế rủi ro nếu database bị lộ.

Cơ chế xác thực thiết bị có vài ý nghĩa quan trọng:

- bảo đảm chỉ thiết bị hợp lệ mới được gửi readings,
- ngăn chặn client lạ giả mạo thiết bị,
- tách riêng luồng bảo mật của thiết bị với luồng user/admin,

- thuận lợi cho việc rotate token theo từng thiết bị.

So với JWT, device token đơn giản hơn, dễ triển khai cho firmware ESP32 và đủ phù hợp trong môi trường thiết bị nhúng. Đây là một quyết định thiết kế hợp lý với bài toán của đề tài.

2.7 Phân quyền RBAC trong hệ thống

RBAC là mô hình phân quyền dựa trên vai trò, trong đó quyền truy cập của người dùng được xác định theo vai trò được gán. Trong các hệ thống theo dõi sức khỏe từ xa, cơ chế này giúp kiểm soát truy cập phù hợp giữa các nhóm người dùng khác nhau. (Sandhu et al., 2000)

Tuy nhiên, trong hệ thống hiện tại của đề tài, cơ chế phân quyền không còn thuần túy dựa trên các vai trò như **admin, caregiver, patient**, mà đã chuyển sang mô hình **device-centric**. Theo đó, quyền truy cập dữ liệu hằng ngày được xác định chủ yếu dựa trên **liên kết thiết bị (device_links)**, với hai mức quyền chính là **owner** và **viewer**. Trong khi đó, vai trò **admin** vẫn được giữ lại ở mức quản trị hệ thống, phục vụ các chức năng như khởi tạo người dùng, đăng ký thiết bị, xoay token ESP và vận hành hệ thống.

Cụ thể, **owner** là tài khoản sở hữu thiết bị, có quyền đọc dữ liệu, quản lý người được chia sẻ, chỉnh ngưỡng cảnh báo, gửi yêu cầu ECG, hủy lệnh và xoay token thiết bị. **Viewer** là tài khoản được cấp quyền theo dõi dữ liệu của thiết bị nhưng không có các quyền quản trị như owner. Cách tổ chức này phù hợp hơn với thực tế sử dụng, vì quyền được gắn theo từng thiết bị thay vì chỉ dựa trên vai trò chung của tài khoản.

Hệ thống có yếu tố **RBAC** ở mức quản trị với vai trò **admin**, nhưng quyền truy cập dữ liệu sức khỏe trong vận hành hằng ngày được triển khai chủ yếu theo mô hình **phân quyền dựa trên liên kết thiết bị**. Cách tiếp cận này phản ánh đúng kiến trúc thực tế của hệ thống và bảo đảm kiểm soát truy cập linh hoạt hơn.

2.8 FastAPI và lý do lựa chọn

2.8.1 Tổng quan về FastAPI

FastAPI là một framework xây dựng web API bằng Python, phù hợp với các hệ thống REST có nhiều endpoint, nhiều loại client và yêu cầu kiểm soát chặt chẽ dữ liệu đầu vào. Trong đề tài này, FastAPI được lựa chọn vì backend phải đồng thời phục vụ ứng dụng web, công cụ quản trị và thiết bị ESP32, đồng thời xử lý các chức năng như xác thực, quản lý thiết bị, dữ liệu sức khỏe, cảnh báo và ECG theo yêu cầu. (FastAPI, 2026)

Một ưu điểm quan trọng của FastAPI là hỗ trợ tốt cho mô hình **bất đồng bộ với async/await**, phù hợp với hệ thống có nhiều luồng giao tiếp đồng thời giữa app, thiết bị và backend. Bên cạnh đó, FastAPI tích hợp chặt chẽ với **Pydantic**, giúp kiểm tra, chuẩn hóa dữ liệu đầu vào và xử lý lỗi validation rõ ràng trước khi đưa vào xử lý nghiệp vụ hoặc lưu trữ.

Ngoài ra, FastAPI còn thuận lợi cho việc tổ chức backend theo hướng **mô-đun**, thông qua việc tách các nhóm route như xác thực, thiết bị, dữ liệu sức khỏe, cảnh báo và API cho ESP32 thành các router riêng. Với nền tảng kỹ thuật gồm **Python 3.11, FastAPI, Uvicorn, Pydantic v2 và pydantic-settings**, backend của hệ thống có cấu trúc rõ ràng, dễ mở rộng và phù hợp với bài toán theo dõi sức khỏe từ xa theo mô hình client-server-IoT.

2.8.2 Vai trò của service layer

Trong kiến trúc backend của hệ thống, **service layer** giữ vai trò tách riêng **logic nghiệp vụ** khỏi lớp **route API**, giúp mã nguồn rõ ràng, dễ bảo trì và thuận lợi cho mở rộng. Theo cách tổ chức này, route chủ yếu nhận request, kiểm tra đầu vào ban đầu, gọi service phù hợp và trả response, còn phần xử lý nghiệp vụ cốt lõi được thực hiện trong service.

Trong hệ thống hiện tại, vai trò này thể hiện rõ ở luồng **ingest dữ liệu sức khỏe**. Route nhận dữ liệu từ thiết bị chỉ thực hiện validate ban đầu rồi chuyển sang **health_service** để xử lý. Tại đây, dữ liệu được chuẩn hóa, tổ chức lại theo mô hình lưu trữ, ghi vào MongoDB, cập nhật trạng thái thiết bị và tiếp tục chuyển sang **alert_service** để kiểm tra ngưỡng và sinh cảnh báo khi cần.

Cách tách này giúp backend giảm độ phức tạp ở lớp API, đồng thời dễ bảo trì hơn khi cần thay đổi quy tắc xử lý dữ liệu hoặc cảnh báo. Bên cạnh đó, service layer cũng thuận lợi cho **kiểm thử độc lập**, vì có thể kiểm tra trực tiếp logic nghiệp vụ mà không cần đi qua toàn bộ lớp HTTP.

2.9 MongoDB và lý do lựa chọn

2.9.1 Tổng quan về MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL theo mô hình document, trong đó dữ liệu được lưu dưới dạng BSON với cấu trúc gần giống JSON. So với cơ sở dữ liệu quan hệ, MongoDB cho phép tổ chức dữ liệu linh hoạt hơn, phù hợp với các hệ thống có nhiều trường lồng nhau và cấu trúc dữ liệu thay đổi theo ngữ cảnh. (MongoDB, 2026)

Trong đề tài này, MongoDB là lựa chọn phù hợp vì hệ thống phải xử lý nhiều loại dữ liệu khác nhau như **người dùng, thiết bị, dữ liệu sức khỏe, cảnh báo, lệnh điều khiển và phiên xác thực**. MongoDB đóng vai trò là **kho lưu trữ trung tâm** cho hầu hết các thành phần dữ liệu của backend.

Ưu điểm nổi bật của MongoDB trong hệ thống là khả năng lưu trữ tốt các dữ liệu có cấu trúc lồng nhau như **vitals, ECG, metadata, recorded_at, received_at** trong cùng một document. Điều này giúp backend thuận tiện hơn trong việc lưu trữ, truy vấn dữ liệu mới nhất, dữ liệu lịch sử và các bản ghi sức khỏe đặc thù.

Bên cạnh đó, MongoDB còn hỗ trợ **index** và **TTL index**, giúp tối ưu truy vấn, quản lý vòng đời dữ liệu và hạn chế lưu trùng bản ghi. Nhờ vậy, MongoDB

không chỉ là nơi lưu dữ liệu mà còn là nền tảng quan trọng hỗ trợ backend hiện thực hóa các luồng nghiệp vụ của hệ thống theo dõi sức khỏe từ xa.

2.9.2 Tổ chức dữ liệu trong MongoDB

Trong hệ thống hiện tại, dữ liệu trong MongoDB được phân tách thành nhiều **collection chuyên biệt**, mỗi collection đảm nhiệm một vai trò riêng trong kiến trúc backend. Các collection chính gồm **health_readings, alerts, devices, users, device_commands, device_links** và **auth_sessions**. Cách tổ chức này giúp hệ thống dễ quản lý, truy vấn, bảo trì và mở rộng theo từng nhóm nghiệp vụ.

Trong đó, **health_readings** là collection trung tâm dùng để lưu **dữ liệu sức khỏe và dữ liệu ECG**, hỗ trợ truy vấn dữ liệu mới nhất, dữ liệu lịch sử và quản lý vòng đời dữ liệu. **device_commands** đóng vai trò như **hàng đợi lệnh** cho thiết bị ESP32, phục vụ các tác vụ như đo ECG theo yêu cầu và theo dõi trạng thái xử lý lệnh. **devices** lưu thông tin hồ sơ thiết bị, cấu hình, ngưỡng cảnh báo, trạng thái hoạt động và token xác thực của thiết bị.

Bên cạnh đó, **users** lưu thông tin người dùng và dữ liệu xác thực; **alerts** lưu các cảnh báo sinh ra từ dữ liệu sức khỏe; **auth_sessions** quản lý các phiên đăng nhập và refresh token; còn **device_links** lưu liên kết giữa người dùng và thiết bị, đồng thời là cơ sở chính để xác định **quyền truy cập dữ liệu** trong hệ thống hiện tại.

MongoDB trong hệ thống được tổ chức theo hướng **mỗi collection phụ trách một trách nhiệm nghiệp vụ riêng**, giúp cấu trúc dữ liệu rõ ràng, bám sát các luồng xử lý thực tế và phù hợp với kiến trúc backend của hệ thống theo dõi sức khỏe từ xa.

2.9.3 Index và TTL

Một điểm đáng chú ý trong thiết kế dữ liệu của hệ thống là việc sử dụng index và TTL cho các collection chính như health_readings, alerts và device_commands. Cách tổ chức này cho thấy MongoDB không chỉ được dùng để lưu trữ dữ liệu, mà còn được tối ưu cho hiệu năng truy vấn, chống trùng lặp và quản lý vòng đời dữ liệu. (MongoDB, 2026)

Đối với **health_readings**, hệ thống tạo index theo **device_id, timestamp** và **device_uid, timestamp** để hỗ trợ truy vấn dữ liệu mới nhất và dữ liệu lịch sử theo thiết bị. Collection này còn có **unique partial index trên (device_id, seq)** nhằm hạn chế lưu trùng bản ghi khi thiết bị retransmit dữ liệu. Bên cạnh đó, **TTL trên recorded_at** giúp tự động xóa các bản ghi sức khỏe quá cũ sau một thời gian nhất định.

Với **alerts**, hệ thống tạo index theo **severity, acknowledged, device_id, alert_type, timestamp** để hỗ trợ truy vấn cảnh báo theo mức độ, trạng thái hoặc thiết bị. Collection này cũng có cơ chế chống trùng cảnh báo và sử dụng **TTL** để tự động dọn các cảnh báo cũ.

Đối với **device_commands**, các index được tổ chức để phục vụ luồng **polling queue** của thiết bị ESP32, bao gồm truy vấn lệnh theo **device_id, status, created_at**, phát hiện lệnh trùng và bảo đảm không enqueue lặp ngoài ý muốn. Đồng thời, **TTL trên expires_at** giúp tự động xóa các lệnh đã hết hạn, giữ cho hàng đợi luôn gọn gàng.

Việc sử dụng index và TTL giúp hệ thống **truy vấn nhanh hơn, hạn chế dữ liệu trùng lặp, tự động dọn dữ liệu cũ** và hỗ trợ vận hành ổn định hơn trong quá trình triển khai thực tế.

2.10 Redis và vai trò trong hệ thống

Redis là hệ quản trị dữ liệu in-memory, thường được dùng cho các tác vụ cần truy xuất nhanh như cache, lưu trạng thái tạm thời hoặc giới hạn tần suất truy cập. Trong hệ thống của đề tài, Redis không phải là hàng đợi chính cho luồng ECG, mà chỉ giữ vai trò hỗ trợ ở tầng kiểm soát truy cập và vận hành. (Redis, 2026)

Cụ thể, hàng đợi command cho ECG và các tác vụ on-demand được tổ chức trong **MongoDB**, tại collection **device_commands**. Trong khi đó, Redis được backend sử dụng chủ yếu cho cơ chế **rate limiting**, thông qua việc lưu trạng thái ngắn hạn và theo dõi số lượng request trong từng khoảng thời gian cấu hình.

Nhờ Redis, hệ thống có thể kiểm soát tần suất truy cập của nhiều loại tác nhân khác nhau như **người dùng, admin và thiết bị ESP32**, từ đó giới hạn request khi vượt ngưỡng và trả về các phản hồi phù hợp như **429 Too Many Requests**. Việc dùng Redis cho mục đích này là hợp lý vì Redis có tốc độ xử lý cao và rất phù hợp với các phép toán đếm, cập nhật trạng thái ngắn hạn theo từng request.

Trong hệ thống hiện tại, **Redis được dùng cho rate limiting và kiểm soát truy cập ngắn hạn**, còn **MongoDB mới là nơi lưu và điều phối command on-demand**. Cách phân tách này phản ánh đúng kiến trúc thực tế của backend.

2.11 Docker và Docker Compose

2.11.1 Docker

Docker là nền tảng cho phép đóng gói ứng dụng và các thành phần phụ thuộc vào **container**, giúp môi trường chạy được đồng nhất giữa các máy phát triển, kiểm thử và triển khai. Trong đề tài này, Docker đặc biệt phù hợp vì hệ thống gồm nhiều thành phần như **backend, MongoDB, Redis** và các dịch vụ hỗ trợ khác.

Trong hệ thống hiện tại, Docker được dùng để đóng gói **backend FastAPI** thành một image chạy độc lập. Việc sử dụng Docker giúp đồng nhất môi trường, giảm lỗi do khác biệt cấu hình giữa các máy, đồng thời thuận lợi hơn cho kiểm thử, triển khai và quản lý vòng đời ứng dụng.

2.11.2 Docker Compose

Nếu Docker dùng để đóng gói từng thành phần, thì Docker Compose được dùng để khai báo và điều phối nhiều container của toàn bộ hệ thống trong một file cấu hình thống nhất. Đối với đề tài này, Docker Compose giữ vai trò quan trọng vì hệ thống không chỉ có backend mà còn gồm MongoDB, Redis và các dịch vụ hỗ trợ khác. (Docker, 2026)

Thông qua Docker Compose, các service được tổ chức và khởi chạy đồng thời, đồng thời xác định rõ mối quan hệ phụ thuộc giữa chúng. Hệ thống hiện cũng có các cấu hình khác nhau cho **môi trường mặc định, môi trường gần production và cấu hình mạng đặc thù**, cho thấy Docker Compose không chỉ hỗ trợ chạy thử mà còn phục vụ triển khai thực tế linh hoạt hơn.

2.11.3 Vai trò trong đề tài

Trong đề tài, **Docker Compose** giữ vai trò là công cụ tổ chức và triển khai môi trường chạy cho toàn bộ hệ thống. Nhờ đó, các thành phần như **backend FastAPI, MongoDB, Redis** và các dịch vụ hỗ trợ có thể được khởi chạy đồng thời trong một cấu hình thống nhất, giúp hệ thống nhất quán hơn trong quá trình phát triển, kiểm thử và vận hành thử nghiệm.

Docker Compose còn cho phép mô tả rõ **mối quan hệ phụ thuộc giữa các dịch vụ**, cấu hình mạng, biến môi trường, cổng truy cập và cơ chế health check. Điều này giúp toàn bộ hệ thống được dựng lên theo đúng kiến trúc triển khai thực tế, thay vì phải cài đặt và chạy thủ công từng thành phần riêng lẻ.

Bên cạnh đó, Docker Compose cũng giúp **đơn giản hóa việc quản lý vòng đời hệ thống**, cho phép dễ dàng khởi động, dừng, khởi động lại hoặc hủy môi trường chạy khi cần. Nhờ vậy, hệ thống không chỉ tồn tại ở mức mã nguồn mà còn có thể được triển khai, kiểm tra và đánh giá trong môi trường thực tế.

2.11.4 Cloudflare Tunnel và Nginx

Trong kiến trúc triển khai hiện tại, hệ thống không chỉ chạy trong **Docker Compose** mà còn có phương án công bố dịch vụ ra bên ngoài để **ứng dụng Web** và **thiết bị ESP32** có thể truy cập qua Internet. Hiện tại, hệ thống **ưu tiên sử dụng Cloudflare Tunnel** làm giải pháp public endpoint, trong đó container **cloudflared** được dùng để kết nối backend ra ngoài một cách an toàn và thuận tiện.

Bên cạnh đó, hệ thống cũng chuẩn bị thêm **quick tunnel** để phục vụ các tình huống kiểm thử hoặc demo nhanh. Điều này cho thấy Cloudflare Tunnel là thành phần chính trong phương án triển khai public service của hệ thống.

Ngoài Cloudflare Tunnel, repo còn cung cấp cấu hình mẫu cho **Nginx** như một phương án **reverse proxy** riêng. Nginx được cấu hình để chuyển tiếp các route API và các endpoint kiểm tra trạng thái về backend, đồng thời hỗ trợ

truyền các header cần thiết và có sẵn khung cấu hình HTTPS cùng một số thiết lập bảo mật cơ bản.

2.12 Web và lý do lựa chọn

2.12.1 Tổng quan về Web

Ứng dụng Web là mô hình phần mềm hoạt động trên trình duyệt, cho phép người dùng truy cập trực tiếp mà không cần cài đặt. Với sự phát triển của các framework JavaScript hiện đại (ví dụ: ReactJS), ứng dụng Web có thể cung cấp giao diện trực quan, khả năng tương tác cao và dễ dàng kết nối với backend thông qua API.

Trong đề tài này, ứng dụng Web giữ vai trò là lớp giao diện và tương tác với người dùng, đồng thời là cầu nối giữa người sử dụng và backend FastAPI. Việc lựa chọn Web giúp hệ thống dễ triển khai, dễ truy cập trên nhiều thiết bị và thuận tiện trong quá trình kiểm thử cũng như vận hành.

2.12.2 Vai trò của Web trong hệ thống

Ứng dụng Web đóng vai trò là lớp giao diện người dùng, cho phép người dùng truy cập hệ thống thông qua trình duyệt và tương tác với các chức năng chính. Thông qua các API do backend cung cấp, ứng dụng Web đảm nhiệm các chức năng như đăng nhập, đăng ký, quản lý session, tải danh sách thiết bị liên kết, truy vấn và hiển thị dữ liệu latest, history và ECG.

Bên cạnh đó, ứng dụng còn xử lý các trạng thái như chưa đăng nhập, chưa có thiết bị hoặc chưa có dữ liệu, giúp bảo đảm trải nghiệm sử dụng thực tế. Nhờ đó, người dùng có thể dễ dàng theo dõi tình trạng sức khỏe và tương tác với hệ thống một cách thuận tiện.

Ứng dụng Web không chỉ là giao diện hiển thị mà còn là thành phần chức năng quan trọng, giúp kết nối người dùng với backend và trình bày dữ liệu sức khỏe theo cách trực quan, dễ sử dụng.

2.12.3 Kiến trúc ứng dụng Web, quản lý trạng thái

Ứng dụng Web được tổ chức theo hướng phân tách nhiều tầng chức năng rõ ràng, bao gồm tầng giao tiếp API, tầng lưu trữ cục bộ, tầng mô hình dữ liệu và tầng quản lý trạng thái giao diện. Cách tổ chức này giúp mã nguồn rõ ràng, dễ bảo trì và thuận lợi cho việc mở rộng.

Ở tầng giao tiếp dữ liệu, ứng dụng sử dụng một lớp API client để gửi các request HTTP đến backend, kết hợp với các service chức năng như xác thực người dùng, quản lý thiết bị và truy vấn dữ liệu sức khỏe. Ở tầng lưu trữ cục bộ, trình duyệt sử dụng localStorage hoặc sessionStorage để lưu token và thông tin phiên.

Về quản lý trạng thái, ứng dụng sử dụng các cơ chế quản lý state ở frontend (ví dụ: Context API hoặc thư viện quản lý state) để điều phối trạng thái đăng nhập, dữ liệu thiết bị, dữ liệu thời gian thực, lịch sử và ECG. Các trạng thái này phụ thuộc vào phiên xác thực hiện tại, bảo đảm tính nhất quán trong toàn bộ ứng dụng.

Nhìn chung, kiến trúc này giúp tách biệt rõ ràng giữa các thành phần, hỗ trợ kiểm thử tốt hơn và tạo điều kiện thuận lợi cho việc mở rộng chức năng trong tương lai.

2.13 Cơ chế command queue cho ECG on-demand

Một điểm nổi bật của hệ thống là cơ chế hỗ trợ **ECG on-demand**, cho phép người dùng chủ động gửi yêu cầu đo ECG từ ứng dụng. Trong kiến trúc hiện tại, cơ chế này không dùng **Redis** hay **RabbitMQ**, mà sử dụng collection **device_commands** trong **MongoDB** làm hàng đợi lệnh, kết hợp với **recovery loop** chạy nền trong backend để phục hồi và đồng bộ trạng thái command.

Luồng hoạt động của cơ chế này gồm các bước chính: ứng dụng gửi yêu cầu đo ECG đến backend, backend tạo command trong **device_commands**, thiết bị ESP32 poll lệnh từ server, thực hiện đo ECG, gửi **ACK** về backend và backend cập nhật trạng thái command. Nhờ vậy, hệ thống không chỉ thu thập dữ liệu một chiều từ thiết bị mà còn hỗ trợ **tương tác hai chiều** giữa ứng dụng, backend và thiết bị.

Ưu điểm của mô hình này là **không cần bổ sung message broker chuyên dụng**, giúp kiến trúc triển khai gọn hơn và dễ tích hợp với MongoDB vốn đã là cơ sở dữ liệu trung tâm của hệ thống. Đồng thời, queue command vẫn được kiểm soát tương đối đầy đủ thông qua các trạng thái như **pending, completed, failed, expired, cancelled**, cùng với cơ chế recovery và theo dõi trạng thái trong backend.

Tuy nhiên, cơ chế **MongoDB polling queue** phù hợp hơn với **nguyên mẫu hoặc hệ thống quy mô nhỏ đến vừa**. Nếu mở rộng hệ thống ở mức thông lượng lớn hoặc yêu cầu độ trễ thấp, giải pháp này sẽ khó tối ưu bằng các **message broker chuyên dụng**. Như vậy, đây là một lựa chọn thực tế, phù hợp với phạm vi và mức độ hoàn thiện hiện tại của đề tài.

2.14 Các nguyên tắc bảo mật cơ bản được áp dụng

Ngoài cơ chế xác thực và phân quyền, hệ thống còn áp dụng một số **nguyên tắc bảo mật cơ bản** phù hợp với đặc thù dữ liệu sức khỏe. Trước hết, hệ thống **tách riêng cơ chế xác thực** giữa người dùng và thiết bị: app/admin dùng **JWT và refresh token**, còn ESP32 dùng **X-Device-Token**. Cách tổ chức này giúp áp dụng phương thức bảo mật phù hợp cho từng loại tác nhân.

Bên cạnh đó, hệ thống **không lưu token ở dạng thô** trong cơ sở dữ liệu. Refresh token và device token đều được **băm (hash)** trước khi lưu, đồng thời các loại secret cũng được tách riêng trong cấu hình triển khai. Hệ thống còn áp dụng **CORS whitelist**,

Redis-backed rate limiting để hạn chế lạm dụng API, và **tắt tài liệu API theo mặc định** nhằm giảm bề mặt lộ thông tin kỹ thuật ra bên ngoài.

2.15 Logging, health check và giám sát hệ thống

Đối với một hệ thống thực tế, khả năng vận hành không chỉ phụ thuộc vào chức năng nghiệp vụ mà còn ở khả năng quan sát và kiểm tra trạng thái hoạt động. Backend hiện tại đã hỗ trợ khá rõ các cơ chế này thông qua các endpoint như /live, /ready, /health và /metrics. Trong đó, /live dùng để kiểm tra tiến trình còn hoạt động, /ready kiểm tra trạng thái sẵn sàng của hệ thống và kết nối database, còn /metrics cung cấp các chỉ số phục vụ giám sát.

Bên cạnh đó, hệ thống còn tích hợp health check ngay trong môi trường Docker Compose, giúp theo dõi trạng thái của backend, MongoDB, Redis và các dịch vụ liên quan. Ở mức metrics, backend cũng theo dõi các chỉ số như số lượng request, độ trễ xử lý, số lần bị rate limit và trạng thái hàng đợi command. Điều này cho thấy hệ thống không chỉ được xây dựng để chạy chức năng mà còn đã được tổ chức theo hướng có thể kiểm tra, giám sát và vận hành thử nghiệm tương đối đầy đủ.

CHƯƠNG 3
PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC HỆ THỐNG

3.1 Bối cảnh bài toán và yêu cầu tổng quát

Bài toán theo dõi sức khỏe từ xa không dừng ở việc đo được một vài chỉ số sinh hiệu từ thiết bị đeo. Giá trị thật sự của hệ thống chỉ xuất hiện khi dữ liệu đó đi qua được toàn bộ chuỗi xử lý gồm xác thực, truyền tải, lưu trữ, truy vấn, hiển thị, cảnh báo và phản hồi điều khiển. Nếu chỉ có thiết bị cảm biến mà không có backend và ứng dụng tiêu thụ dữ liệu, readings chỉ là những mẫu số liệu rời rạc. Ngược lại, nếu chỉ có ứng dụng web mà không có nguồn dữ liệu đầu vào hoặc không có cơ chế điều phối với thiết bị, hệ thống cũng không thể giải quyết nhu cầu thực tế của người dùng.

Từ mã nguồn hiện tại, có thể xác định hệ thống của đề tài được xây dựng để giải quyết chính bài toán đó theo mô hình nhiều thành phần: thiết bị ESP32 làm nhiệm vụ đo dữ liệu và nhận lệnh, backend FastAPI đóng vai trò trung tâm nghiệp vụ, MongoDB lưu toàn bộ trạng thái dữ liệu, Redis hỗ trợ bảo vệ API, còn ứng dụng web cung cấp giao diện cho người dùng cuối. Hệ thống không chỉ hỗ trợ luồng ingest một chiều từ thiết bị lên server mà còn hỗ trợ tương tác hai chiều qua cơ chế ECG on-demand, trong đó app có thể chủ động yêu cầu thiết bị thực hiện một tác vụ mới.

Từ góc nhìn phân tích yêu cầu, hệ thống cần đồng thời đáp ứng ba mục tiêu. Thứ nhất, phải thu thập và lưu dữ liệu sức khỏe một cách ổn định theo thời gian. Thứ hai, phải cho phép người dùng truy cập dữ liệu theo đúng phạm vi được cấp quyền, đồng thời duy trì trải nghiệm sử dụng đủ mượt cho ứng dụng web . Thứ ba, phải đủ mở để tích hợp thêm các chức năng vận hành và mở rộng về sau như thay đổi chính sách queue, bổ sung loại readings mới hoặc tăng cường monitoring khi triển khai thực tế.

3.2 Yêu cầu hệ thống

Yêu cầu của hệ thống có thể nhìn từ hai nhóm lớn là yêu cầu chức năng và yêu cầu phi chức năng. Ở nhóm chức năng, đề tài cần hỗ trợ đầy đủ ba luồng chính: luồng người dùng trên ứng dụng Web, luồng ingest và điều khiển thiết bị ESP32, và luồng xử lý nghiệp vụ tập trung tại backend. Ở nhóm phi chức năng, hệ thống phải quan tâm đến tính đúng đắn dữ liệu, bảo mật, khả năng quan sát vận hành, khả năng mở rộng và tính thân thiện ở phía ứng dụng.

Bảng ***3.1. Ma trận yêu cầu chức năng theo thành phần***

Thành phần	Chức năng chính	Điểm nhấn trong thiết kế
Backend	Xác thực người dùng, quản lý refresh session, xác thực thiết bị, ingest readings, sinh cảnh báo, phục vụ latest/history/ECG, điều phối command ECG, health check, metrics và rate limit.	Backend là trung tâm nghiệp vụ, không chỉ trả dữ liệu mà còn điều phối luồng hai chiều giữa app và thiết bị.
Ứng dụng Web	Đăng nhập, đăng ký, khôi phục phiên, đồng bộ thiết bị, chọn device hiện tại, hiển thị realtime/history, xem cảnh báo và gửi yêu cầu ECG.	App được tổ chức theo hướng device-centric, tức mọi truy vấn sau đăng nhập đều bám theo thiết bị đang được chọn.
ESP32	Đo chỉ số sức khỏe, gửi readings, poll command, ACK command và thực thi tác vụ ECG on-demand.	Thiết bị không chỉ là nguồn phát dữ liệu mà còn là tác nhân thi hành lệnh từ backend.

Ở mức chức năng chi tiết, backend phải hỗ trợ luồng xác thực người dùng bằng access token và refresh token, đồng thời duy trì một cơ chế xác thực riêng cho thiết bị thông qua `X-Device-Token`. Điều này đặc biệt quan trọng vì ứng dụng người dùng và thiết bị vật lý là hai loại client khác nhau hoàn toàn. Ứng dụng cần cơ chế phiên hiện đại để có trải nghiệm liền mạch, trong khi thiết bị cần một giao thức xác thực đơn giản, gọn và phù hợp với firmware nhúng.

Ứng dụng Web phải hỗ trợ toàn bộ chuỗi tương tác thực tế của người dùng, từ khi chưa đăng nhập, đã đăng nhập nhưng chưa có thiết bị, đã chọn thiết bị nhưng chưa có dữ liệu đo, cho đến khi dữ liệu được đồng bộ đầy đủ và người dùng có thể xem dữ liệu mới nhất, xem lịch sử, xem cảnh báo hoặc gửi yêu cầu đo ECG.

Một điểm đáng chú ý là ứng dụng không vận hành theo mô hình một người dùng gắn với một thiết bị cố định, mà cho phép lựa chọn và thay đổi thiết bị đang theo dõi. Việc thay đổi thiết bị này dẫn đến sự thay đổi phạm vi dữ liệu được hiển thị và xử lý trong toàn bộ hệ thống, bao gồm dữ liệu thời gian thực, dữ liệu lịch sử và các thông tin liên quan khác. Do đó, cơ chế quản lý trạng thái ở phía frontend cần được tổ chức phù hợp để bảo đảm tính nhất quán của dữ liệu khi người dùng chuyển đổi giữa các thiết bị.

Về phía thiết bị, hợp đồng API hiện tại cho thấy ESP32 phải thực hiện ít nhất ba nhiệm vụ: gửi readings theo đúng schema, polling command tiếp theo từ backend và gửi ACK sau khi hoàn tất tác vụ. Vì vậy, ngay cả khi repo firmware chưa được

tách thành một project độc lập trong ngữ cảnh hiện tại, backend contract đã thể hiện tương đối rõ các trách nhiệm mà firmware buộc phải đáp ứng.

Bảng *3.2. Yêu cầu phi chức năng của hệ thống*

Yêu cầu	Biểu hiện trong hệ thống	Ý nghĩa đối với đề tài
Tính đúng đắn dữ liệu	Dedupe readings bằng (device_id, seq), phân tách dữ liệu theo user-device scope, hỗ trợ latest và history theo thời gian.	Giúp dữ liệu sức khỏe có thể được truy vấn ổn định và không bị sai lệch do retransmission.
Tính bảo mật	JWT Bearer cho app, refresh session trong `auth_sessions`, `X-Device-Token` cho ESP, ownership check theo `device_links`.	Giảm rủi ro lộ phiên, giới hạn quyền truy cập và bảo vệ dữ liệu sức khỏe nhạy cảm.
Tính sẵn sàng	Có `/live`, `/ready`, `/health`, `/metrics`, logging qua Docker Compose và monitor script.	Cho phép quan sát và kiểm tra hệ thống khi triển khai bằng container.
Khả năng mở rộng	MongoDB với mô hình document, cơ chế command queue dựa trên collection, kiến trúc backend được module hóa theo các lớp API – service – database – utility và cơ chế quản lý trạng thái ở phía frontend.	Tạo nền tảng mở rộng thêm loại dữ liệu, route hoặc policy queue trong tương lai.
Tính thân thiện với người dùng	App xử lý rõ các trạng thái chưa đăng nhập, chưa có device, chưa có reading, lỗi quyền và session lỗi.	Làm cho hệ thống không chỉ đúng về kỹ thuật mà còn khả dụng ở góc nhìn người dùng cuối.

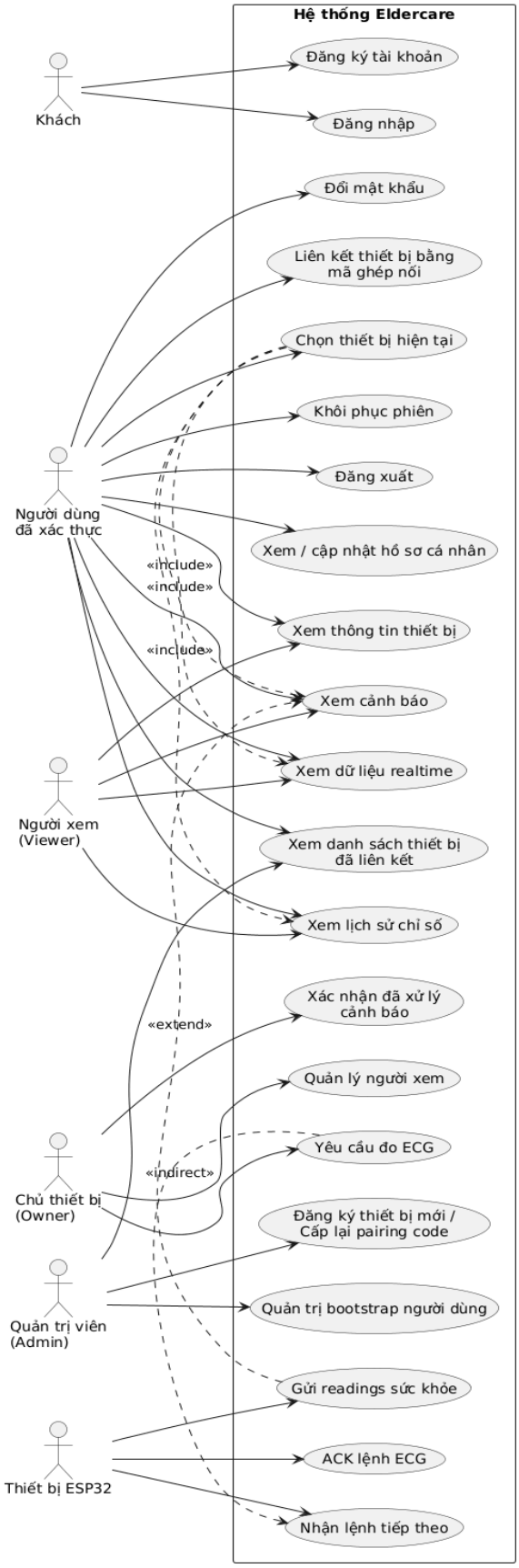
3.3 Tác nhân tham gia và phạm vi tương tác

Nếu xét theo tác nhân tham gia, hệ thống hiện tại có ít nhất năm thực thể đáng chú ý. Người dùng cuối là tác nhân trực tiếp thao tác trên ứng dụng, chủ yếu quan tâm đến đăng nhập, đồng bộ thiết bị, xem dữ liệu và gửi yêu cầu ECG. Người quản trị hoặc người theo dõi có quyền rộng hơn trong một số tình huống nghiệp vụ, nhưng trong mã nguồn hiện tại quyền dữ liệu hằng ngày chủ yếu được suy ra từ liên kết user-device hơn là từ một role tĩnh duy nhất.

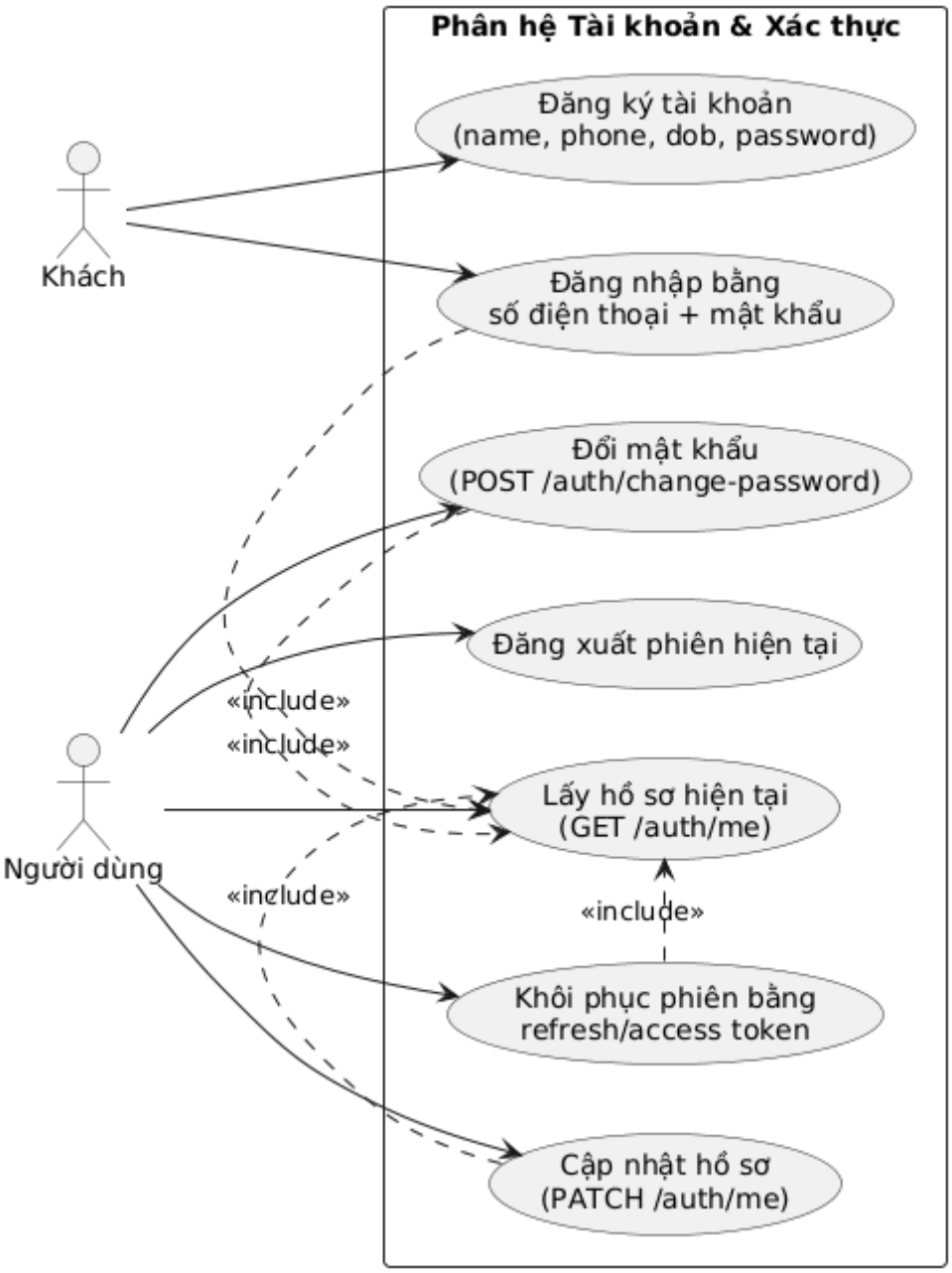
Thiết bị ESP32 là tác nhân vật lý của hệ thống, đóng vai trò vừa là nguồn readings vừa là nơi thi hành command. Backend là tác nhân điều phối trung tâm; nó tiếp nhận request từ cả app lẫn ESP, xác thực, phân quyền, lưu trữ dữ liệu và trả kết quả về cho client phù hợp. MongoDB và Redis không phải là tác nhân sử dụng theo nghĩa người dùng, nhưng là hai thành phần hạ tầng bắt buộc để backend hoạt động đúng và ổn định.

Bảng *3.3. Tác nhân chính và vai trò trong hệ thống*

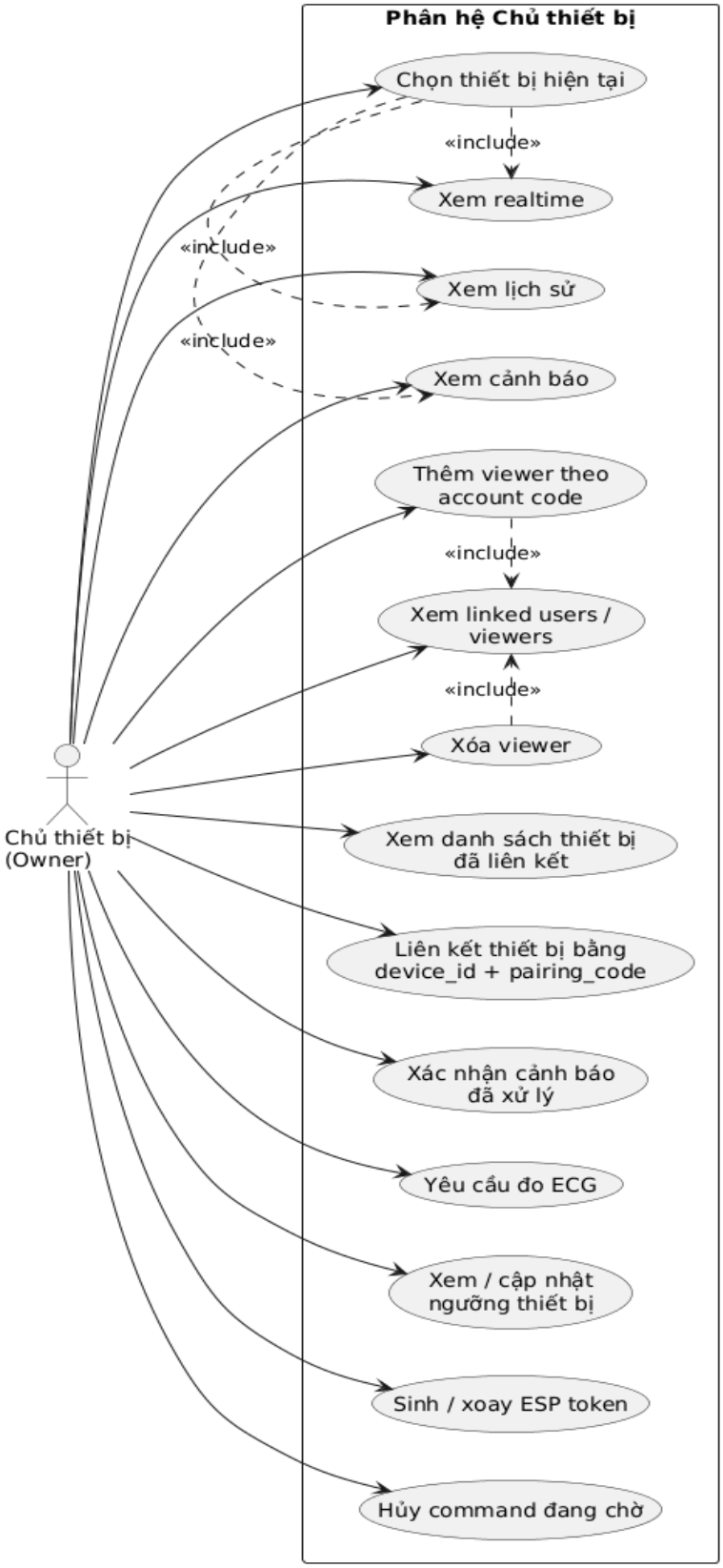
Tác nhân	Vai trò	Tương tác chủ yếu
Người dùng cuối	Sử dụng ứng dụng để xem dữ liệu và gửi yêu cầu ECG.	Đăng nhập, chọn thiết bị, xem latest/history/alerts, refresh dữ liệu, yêu cầu ECG.
Người quản trị / người xem được cấp quyền	Theo dõi hoặc quản trị ở mức cao hơn tùy bối cảnh.	Quản lý người dùng, thiết bị, token, quyền chia sẻ hoặc giám sát tình trạng vận hành.
ESP32	Thiết bị đầu cuối phát sinh readings và thực thi lệnh.	POST readings, GET commands/next, POST ack, thực thi đo ECG khi được yêu cầu.
Backend FastAPI	Hạt nhân nghiệp vụ và lớp điều phối hệ thống.	Xác thực, ingest, alerting, truy vấn, command queue, session management, rate limit.
MongoDB và Redis	Hạ tầng lưu trữ và bảo vệ API.	MongoDB lưu dữ liệu nghiệp vụ; Redis giữ state rate limit.



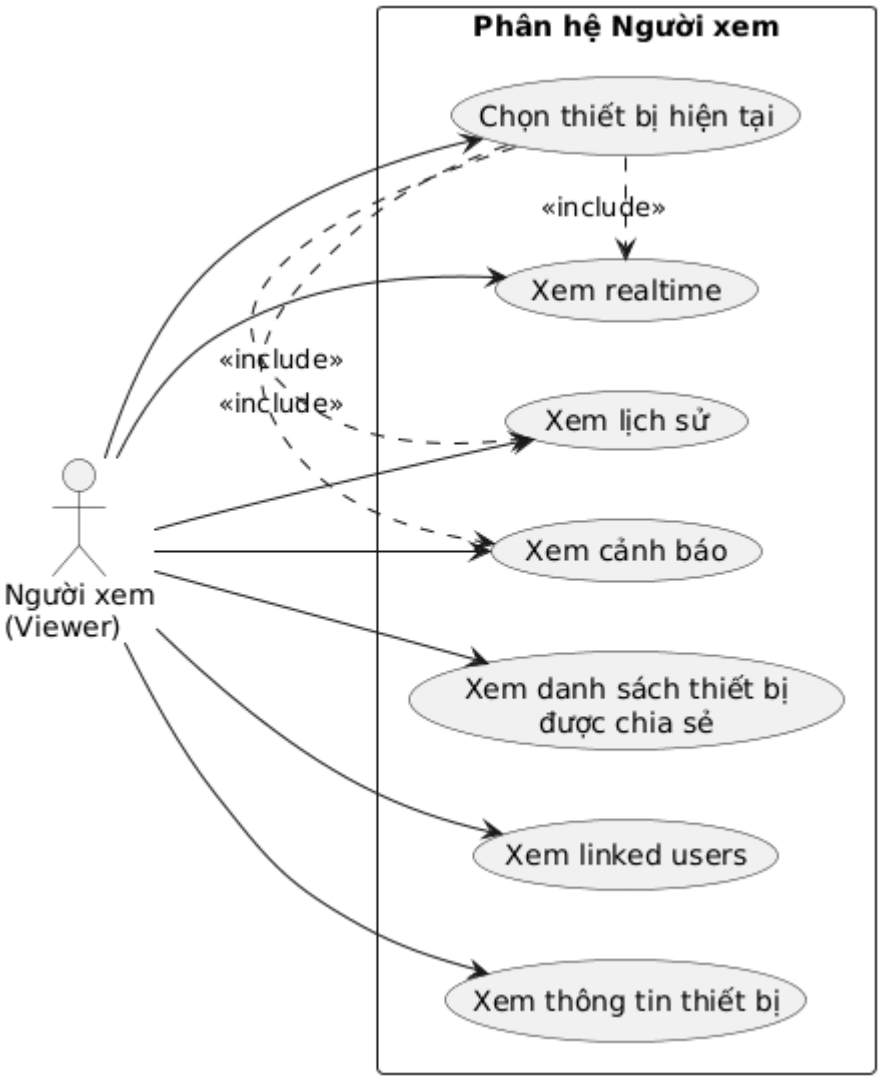
***Hình** 3.1. Sơ đồ Use Case tổng quan toàn hệ thống Eldercare*



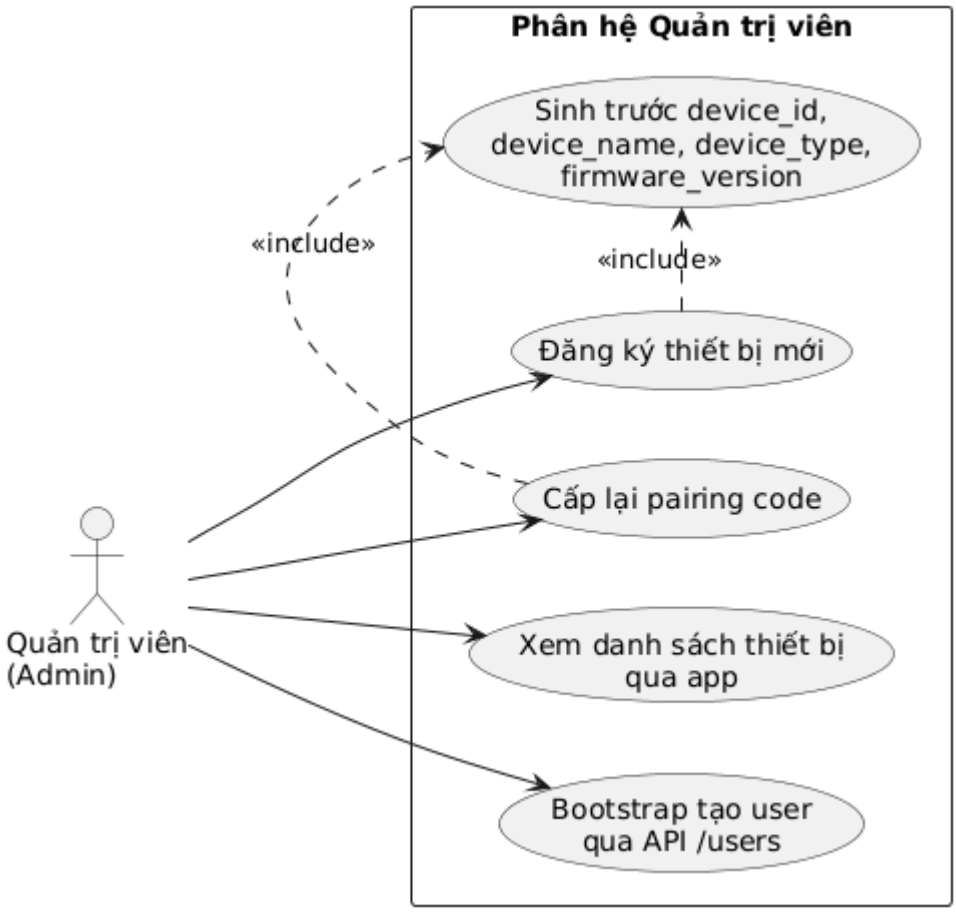
***Hình** 3.2. Sơ đồ Use Case chi tiết phân hệ Tài khoản & Xác thực*



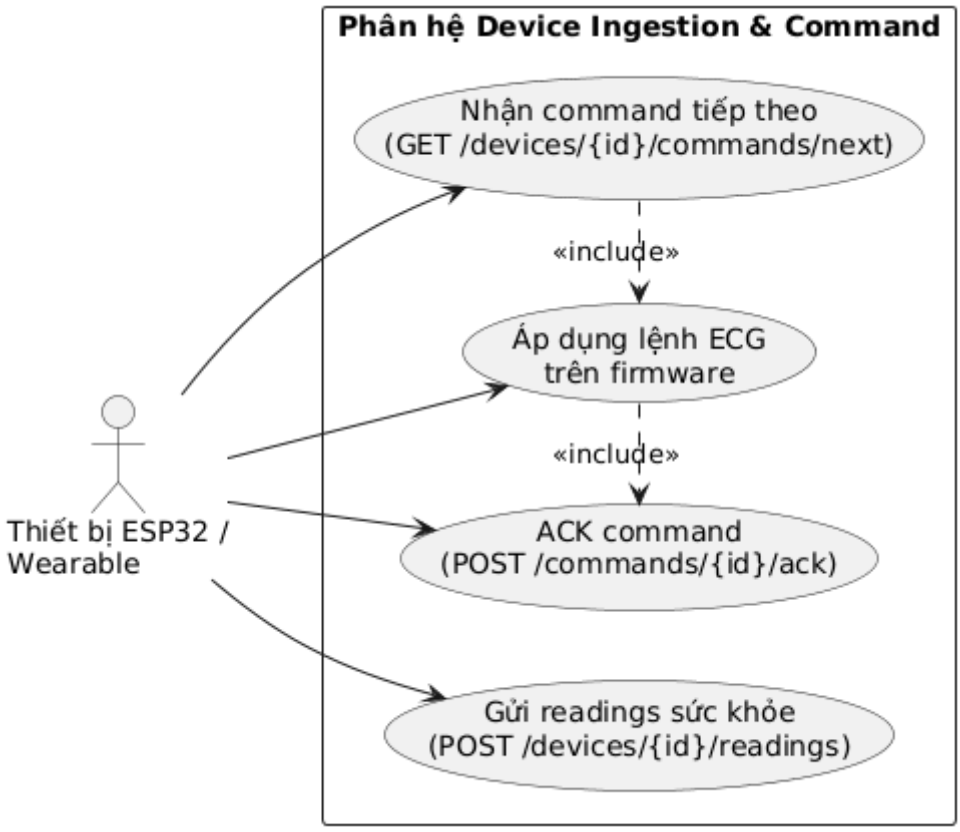
***Hình** 3.3. Sơ đồ Use Case chi tiết phân hệ Chủ thiết bị*



***Hình** 3.4. Sơ đồ Use Case chi tiết phân hệ Người xem*



***Hình** 3.5. Sơ đồ Use Case chi tiết phân hệ Quản trị viên*

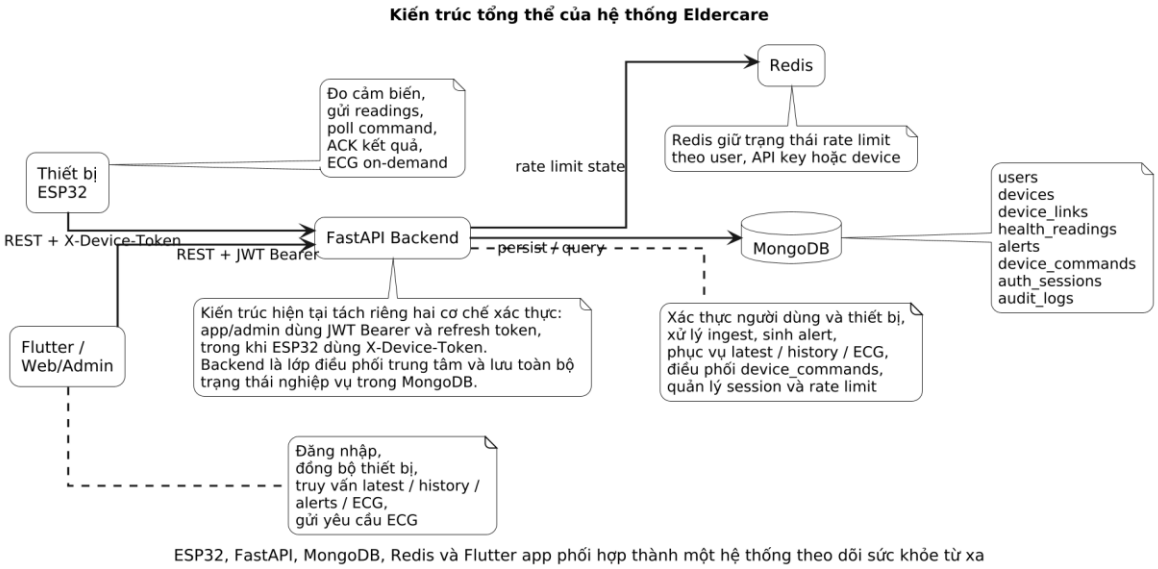


***Hình** 3.6. Sơ đồ Use Case chi tiết phân hệ Thiết bị ESP32*

3.4 Kiến trúc tổng thể của hệ thống

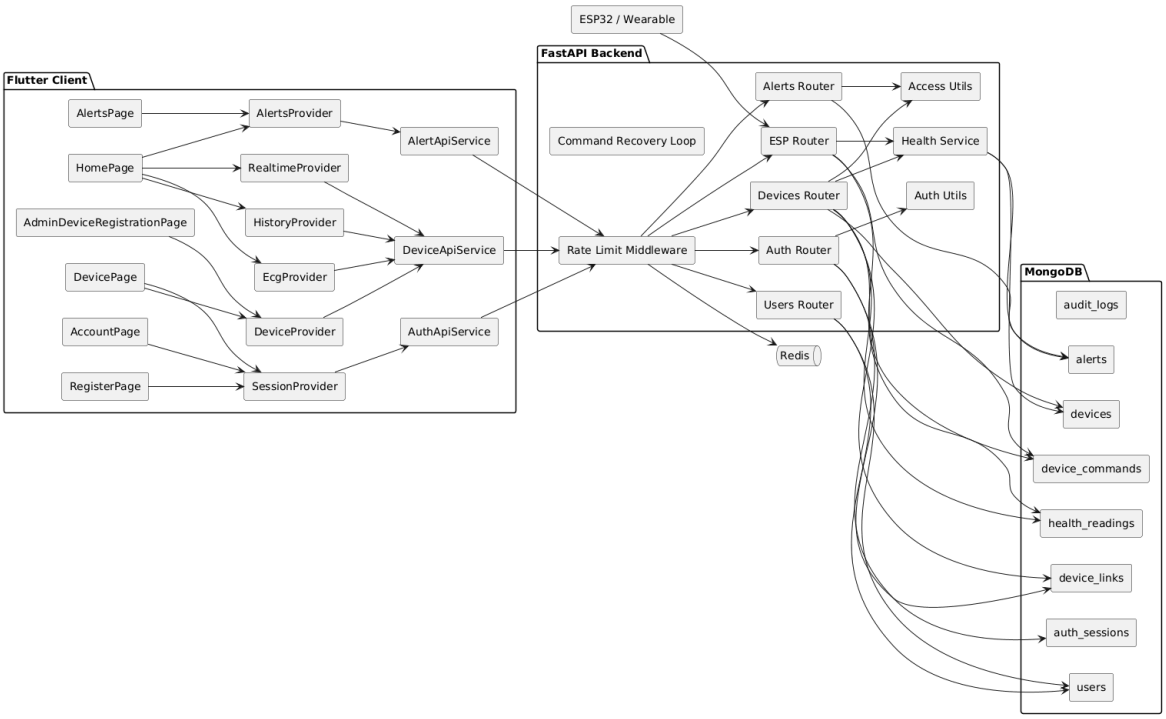
Ở mức cao, hệ thống hiện tại được xây dựng theo mô hình trung tâm hóa tại backend. ESP32 giao tiếp với backend qua nhóm route dành riêng cho thiết bị, ứng dụng web và các công cụ admin giao tiếp với backend qua nhóm route dành cho người dùng, còn backend tiếp tục kết nối xuống MongoDB và Redis để lưu trữ và bảo vệ hệ thống. Điểm nổi bật trong kiến trúc này là backend không chỉ làm API trung chuyển mà thực sự là nơi hội tụ mọi quyết định nghiệp vụ.

Cách bố trí hiện tại tạo ra ba hướng giao tiếp rõ ràng. Hướng thứ nhất là ESP32 đến backend, sử dụng HTTPS REST và `X-Device-Token`. Hướng thứ hai là app hoặc web-admin đến backend, sử dụng HTTPS REST và JWT Bearer kết hợp refresh token. Hướng thứ ba là backend đến MongoDB và Redis, trong đó MongoDB lưu dữ liệu cốt lõi còn Redis phục vụ chủ yếu cho rate limit.

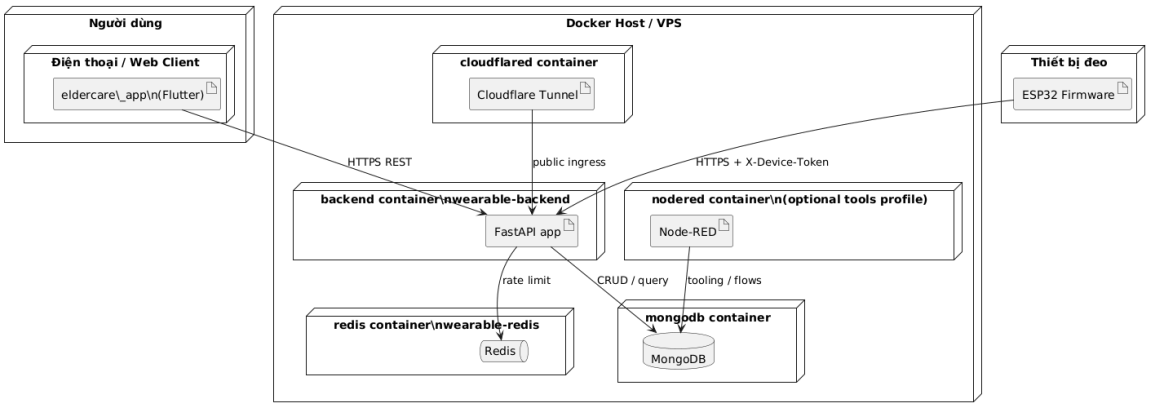


***Hình** 3.7. Kiến trúc tổng thể của hệ thống Eldercare*

Một hệ quả quan trọng của kiến trúc này là mọi luồng nghiệp vụ quan trọng đều phải đi qua backend. Người dùng không giao tiếp trực tiếp với thiết bị, thiết bị cũng không giao tiếp trực tiếp với cơ sở dữ liệu, và ngay cả các chức năng vận hành như health check, metrics hay rate limit cũng đều được đặt quanh backend. Điều này làm cho chương sau có thể tập trung phân tích backend như thành phần quan trọng nhất của toàn hệ thống.



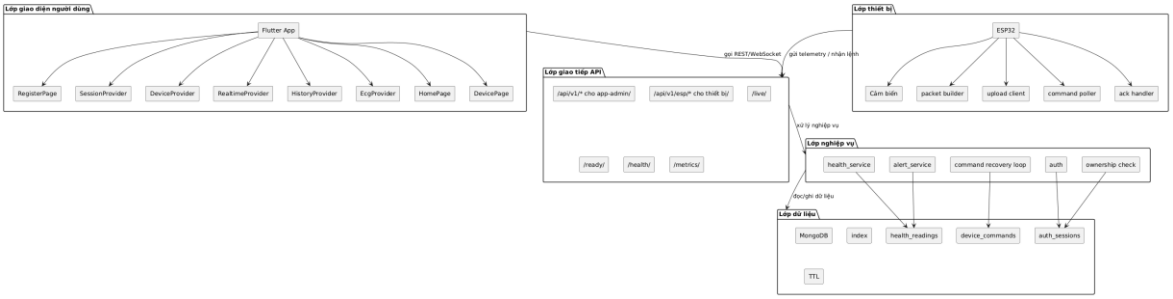
***Hình** 3.8. Component diagram tổng quan app-server*



***Hình** 3.9. Deployment diagram tổng quan Docker/Production*

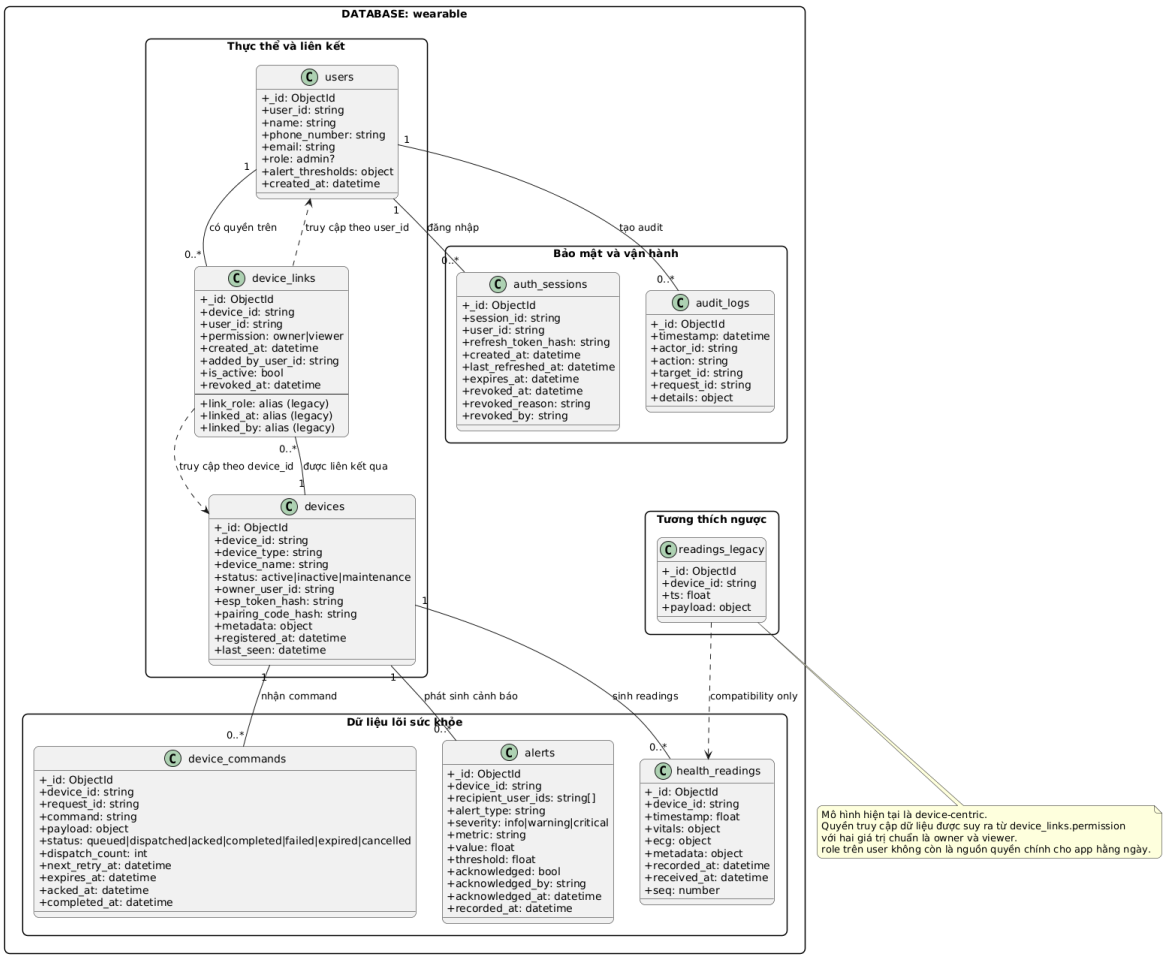
3.5 Kiến trúc phân lớp và phân rã theo thành phần

Nếu nhìn theo lớp, hệ thống hiện tại có thể được phân thành lớp thiết bị, lớp giao tiếp API, lớp nghiệp vụ, lớp dữ liệu và lớp giao diện người dùng. Cách phân lớp này không chỉ mang tính mô tả lý thuyết mà còn phản ánh khá đúng cấu trúc thực tế trong mã nguồn hiện tại. Ở phía dưới, firmware và cảm biến tạo ra dữ liệu đo. Ở lớp trung gian, backend tiếp nhận request, xử lý nghiệp vụ, thao tác dữ liệu và áp dụng các cơ chế bảo mật. Ở phía trên, ứng dụng Web duy trì session, phạm vi thiết bị đang theo dõi và quản lý trạng thái để hiển thị dữ liệu cho người dùng.



***Hình** 3.10. Phân rã hệ thống theo lớp chức năng*

Điểm đáng chú ý là ứng dụng Web hiện không chỉ đóng vai trò là lớp hiển thị thuần túy. Thông qua cơ chế quản lý trạng thái ở phía frontend, ứng dụng còn đảm nhiệm vai trò điều phối trạng thái giữa các thành phần dữ liệu như session, thiết bị, dữ liệu thời gian thực, dữ liệu lịch sử và dữ liệu ECG. Trong khi đó, backend được tổ chức theo hướng phân lớp rõ ràng gồm API, lớp xử lý nghiệp vụ, lớp truy cập dữ liệu và các tiện ích hỗ trợ. MongoDB đóng vai trò là lớp lưu trữ trạng thái lâu dài của toàn bộ hệ thống.



***Hình** 3.11. Các nhóm dữ liệu logic của hệ thống*

3.6 Thiết kế dữ liệu logic ở mức hệ thống

Ở mức logic, hệ thống vận hành trên một nhóm đối tượng dữ liệu tương đối rõ ràng. Người dùng được quản lý trong `users`, thiết bị được lưu trong `devices`, liên kết nhiều-nhiều giữa user và device được quản lý qua `device_links`, readings sức khỏe được lưu trong `health_readings`, cảnh báo được lưu trong `alerts`, lệnh ECG on-demand được đặt trong `device_commands`, còn refresh session được duy trì trong `auth_sessions`.

Việc tồn tại `device_links` là một chi tiết đặc biệt quan trọng. Nó cho thấy hệ thống hiện tại đã vượt qua mô hình một người dùng chỉ gắn với một thiết bị. Nhờ collection này, một thiết bị có thể được chia sẻ cho nhiều user theo các quyền khác nhau như owner hoặc viewer. Ở góc độ nghiệp vụ, điều đó tương đương với việc hệ thống có thể mô hình hóa quan hệ người sở hữu thiết bị và người theo dõi, thay vì chỉ giới hạn trong một cặp patient-device cố định.

Bên cạnh nhóm dữ liệu nghiệp vụ, backend còn duy trì `audit_logs` và một collection legacy `readings` để phục vụ quản trị và tương thích ngược. Điều này phản

ánh một hệ thống đang đi theo hướng sản phẩm hóa dần, chứ không chỉ là một bài demo học thuật tách rời các nhu cầu vận hành thực tế.

3.7 Các luồng nghiệp vụ chính

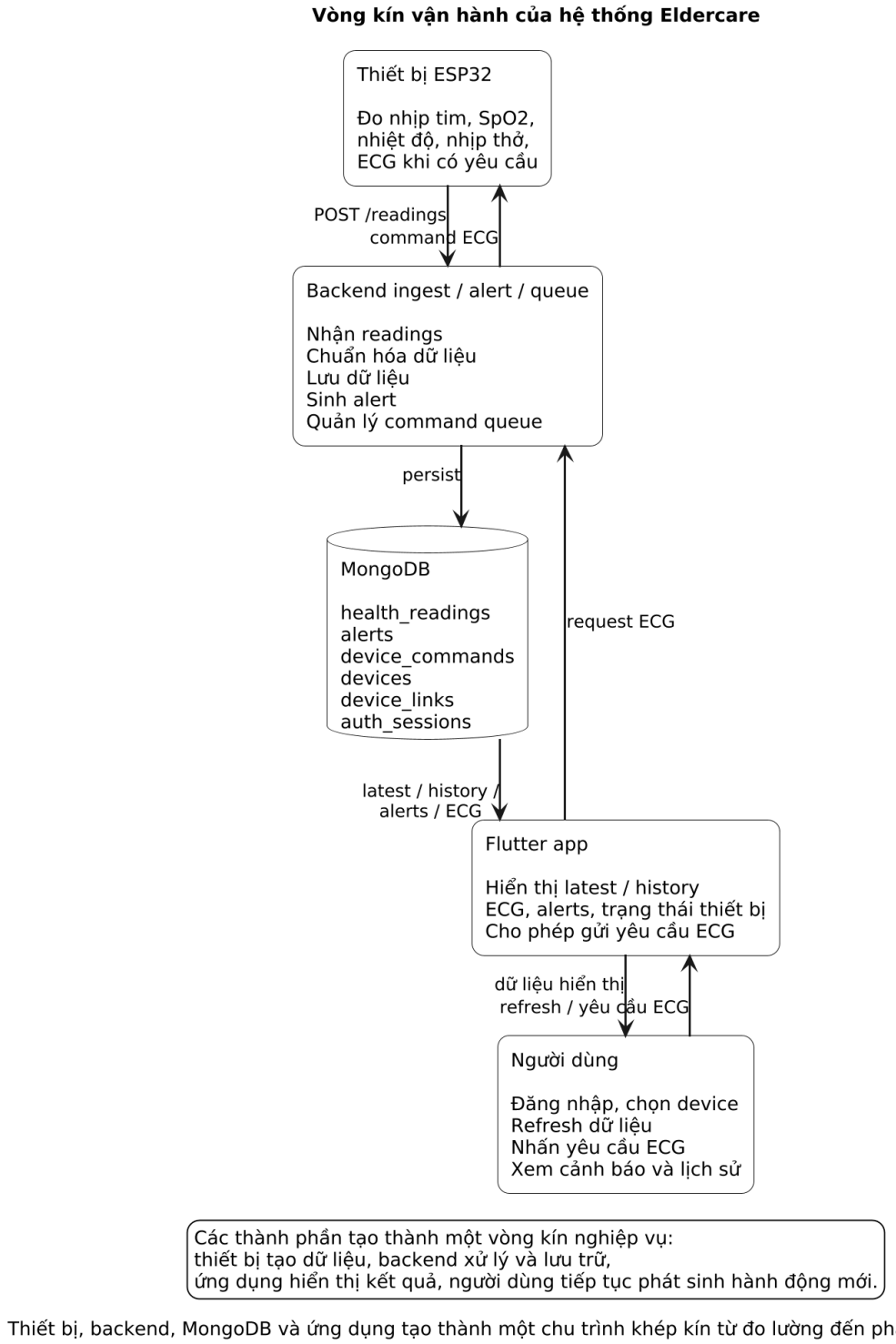
Từ góc nhìn vận hành, có năm luồng nghiệp vụ nổi bật quyết định giá trị của hệ thống hiện tại. Thứ nhất là luồng đăng nhập và đồng bộ phiên, trong đó app hiển thị form đăng nhập, gọi `login`, nhận `access_token` và `refresh_token`, tiếp tục gọi `me`, sau đó tải danh sách thiết bị đã liên kết. Thứ hai là luồng đồng bộ thiết bị và chọn thiết bị hiện tại; đây là bước chuyển hệ thống sang chế độ device-centric, vì mọi truy vấn về sau đều bám theo `currentDeviceId`.

Thứ ba là luồng ingest readings từ ESP32 lên backend. Thiết bị gửi readings đến route `/api/v1/esp/devices/{device_id}/readings`; backend xác thực thiết bị, validate payload, gọi `health_service.process_health_reading`, lưu document vào `health_readings` và gọi `alert_service.check_health_reading` để sinh cảnh báo khi cần. Luồng này là trục xương sống của toàn bộ bài toán theo dõi sức khỏe từ xa.

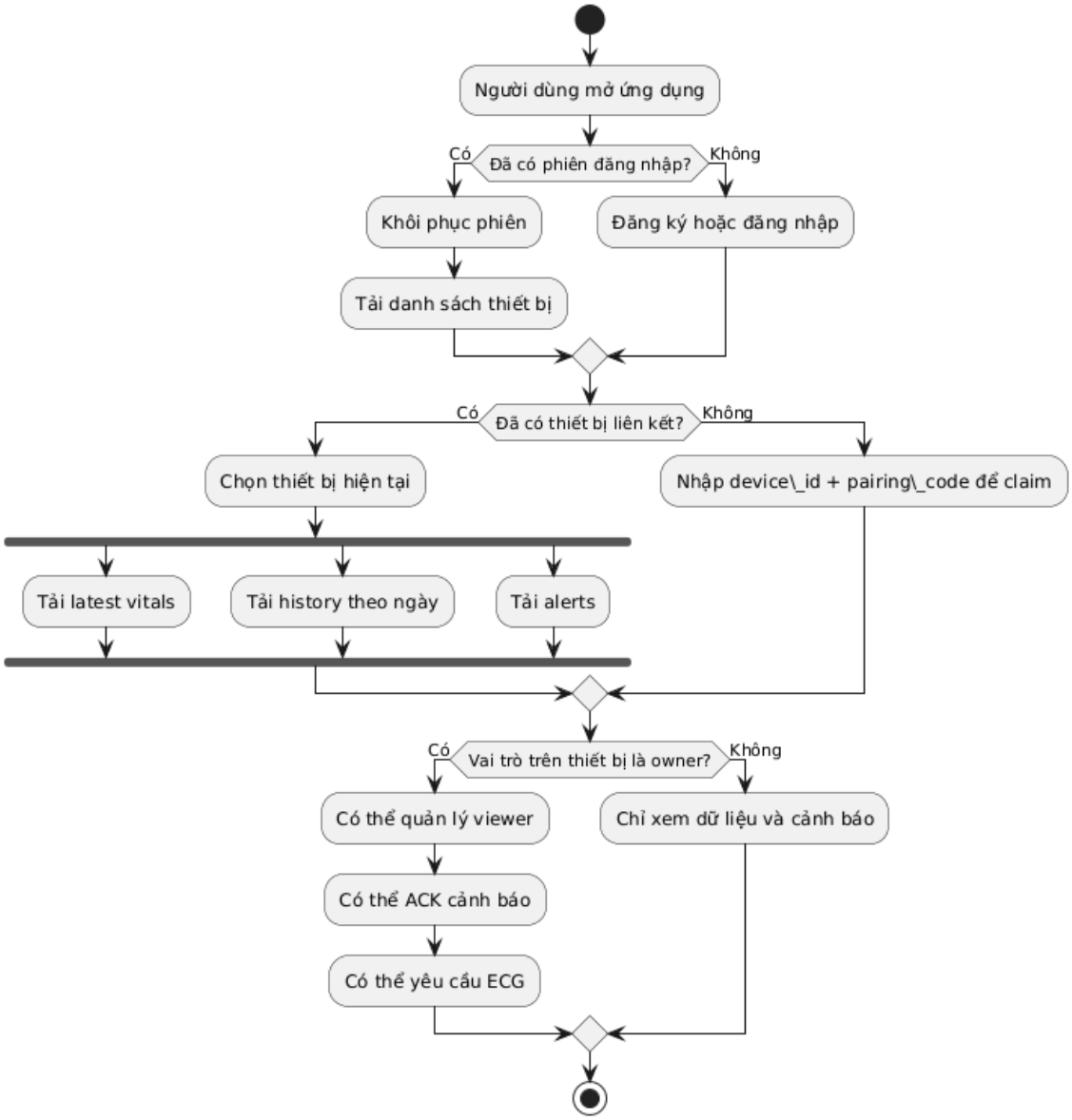
Thứ tư là luồng truy vấn dữ liệu mới nhất, dữ liệu lịch sử, cảnh báo và ECG ở phía ứng dụng Web. Sau khi người dùng đã chọn thiết bị, hệ thống frontend sẽ gửi các yêu cầu tương ứng đến các API như latest, history hoặc ECG dựa trên phạm vi của thiết bị đó; dữ liệu sau đó được xử lý và hiển thị trên các trang hoặc thành phần giao diện phù hợp.

Việc ứng dụng được thiết kế với nhiều trạng thái như không có dữ liệu (empty state) và lỗi (error state) cho thấy luồng xử lý này đã được xây dựng hướng đến các tình huống vận hành thực tế, thay vì chỉ phục vụ cho dữ liệu trong điều kiện lý tưởng.

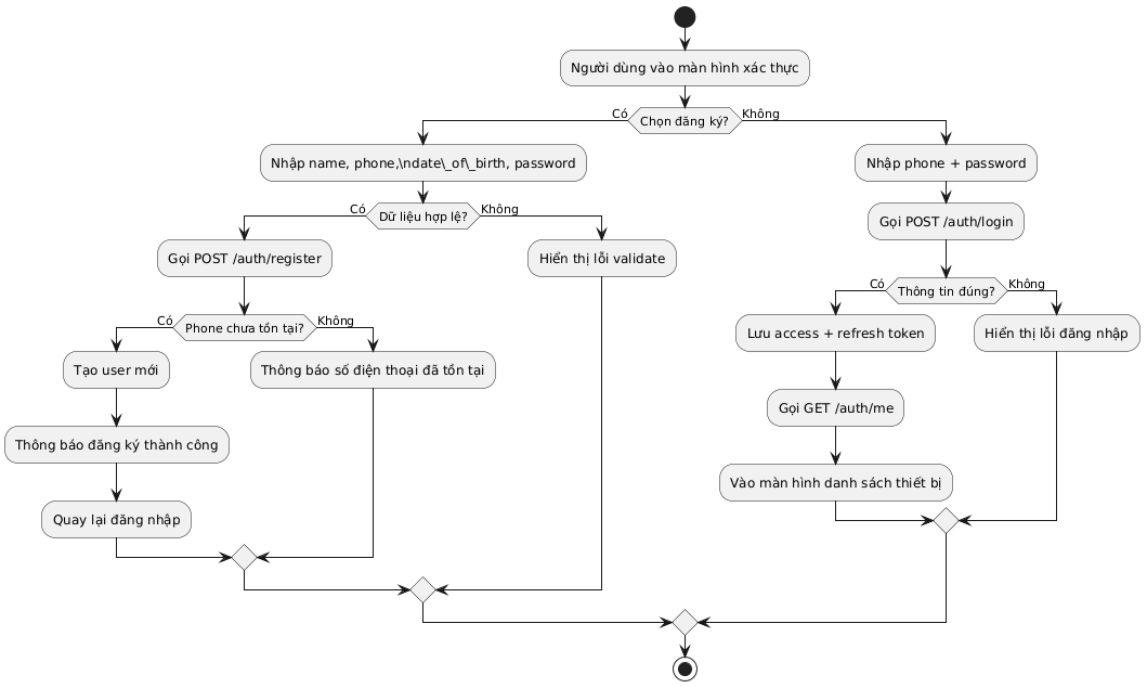
Thứ năm và cũng là luồng nổi bật nhất là ECG on-demand. Người dùng nhấn yêu cầu ECG trên app, backend tạo command trong `device_commands`, ESP32 poll command mới, nhận lệnh, thực thi đo ECG, gửi ACK và app sau đó refresh lại dữ liệu để xem kết quả mới. Luồng này là bằng chứng cho thấy hệ thống không còn là một mô hình one-way đơn giản mà đã có tương tác điều khiển hai chiều giữa người dùng và thiết bị.



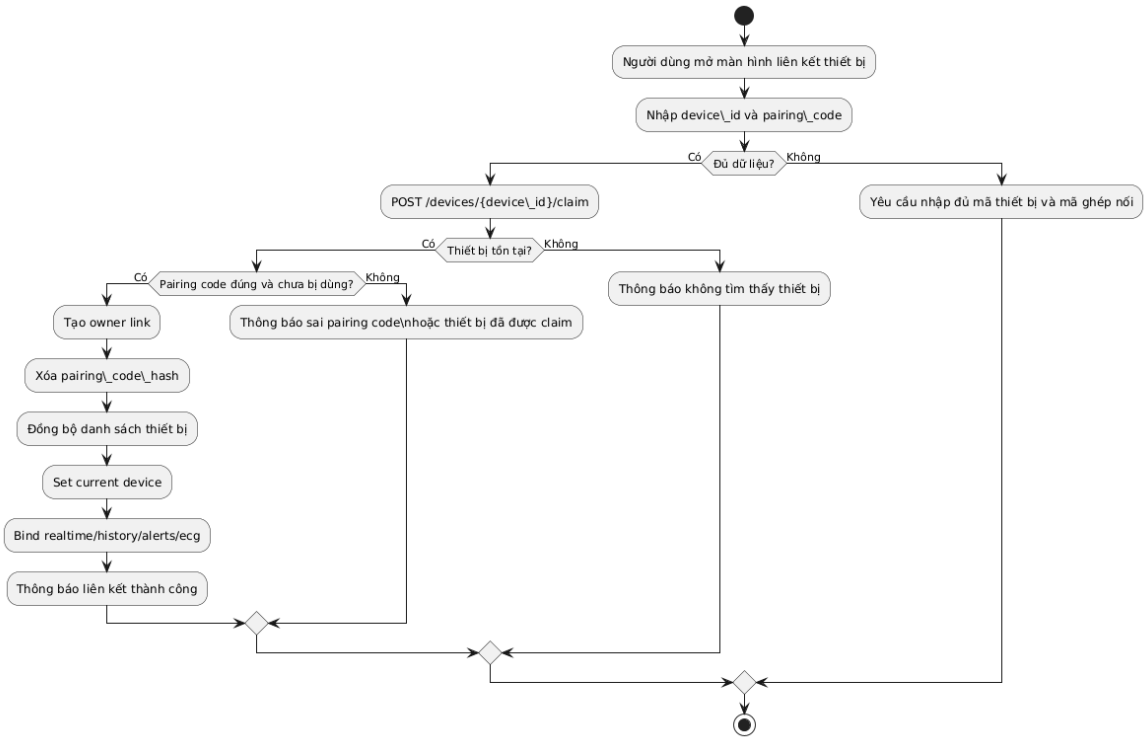
***Hình** 3.12. Vòng kín vận hành từ thiết bị, backend, cơ sở dữ liệu đến ứng dụng*



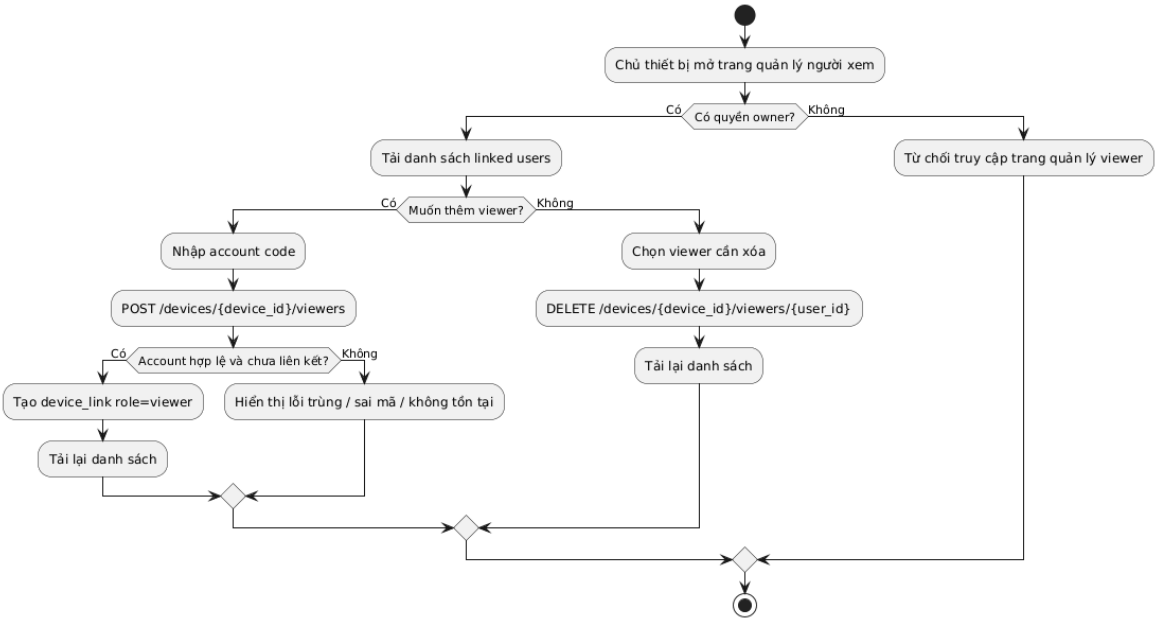
***Hình** 3.13. Activity tổng quan nghiệp vụ*



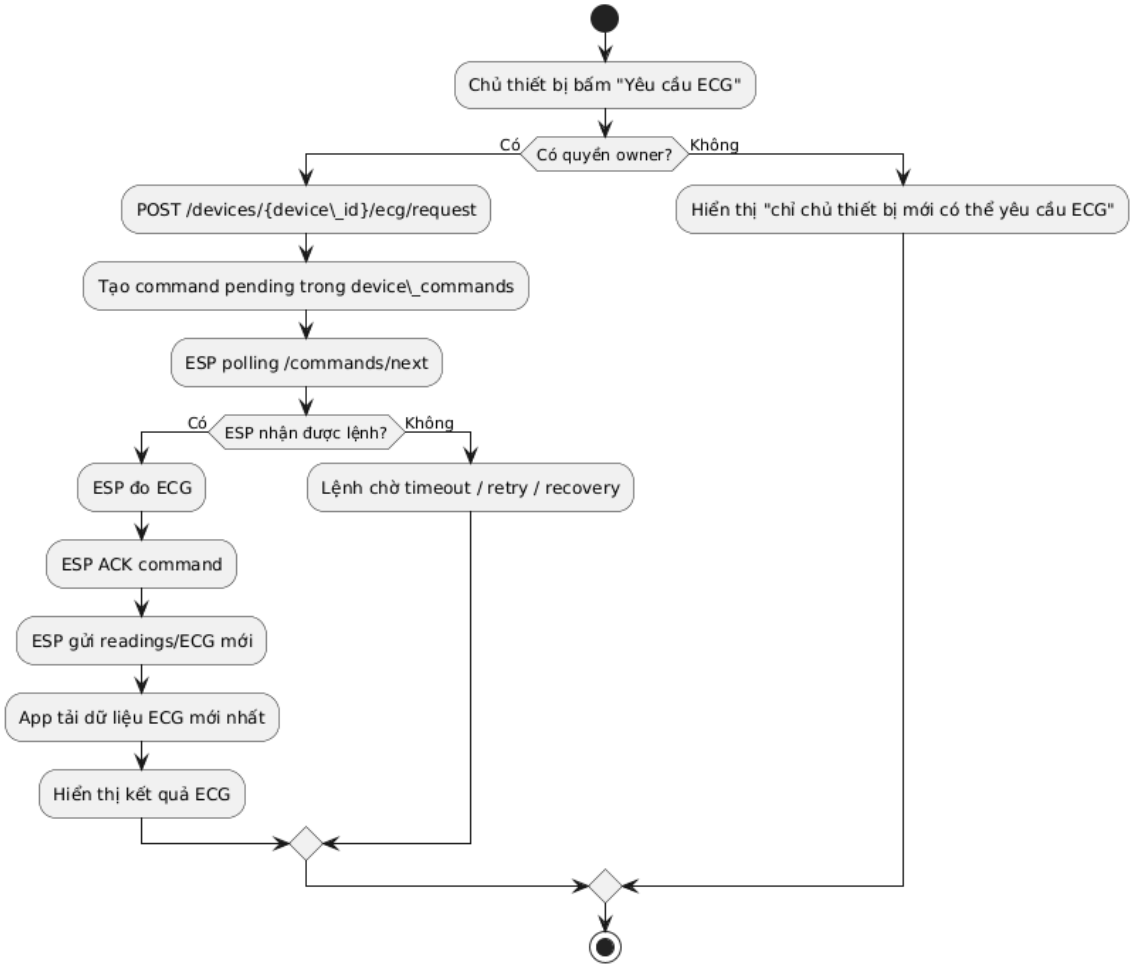
***Hình** 3.14. Activity đăng ký / đăng nhập*



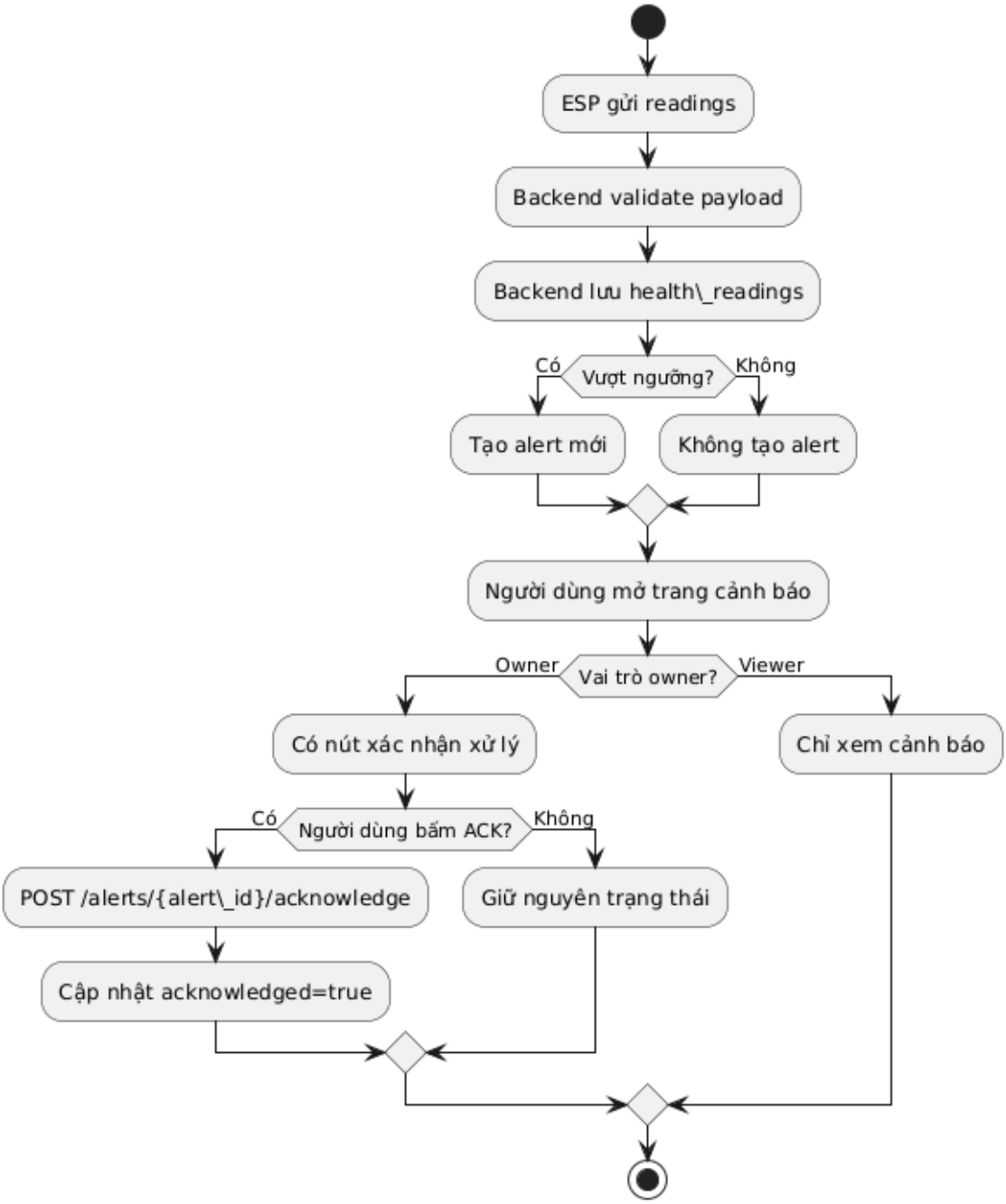
***Hình** 3.15. Activity liên kết thiết bị*



***Hình** 3.16. Activity quản lý người xem*



***Hình** 3.17. Activity yêu cầu ECG*



***Hình** 3.18. Activity xử lý cảnh báo*

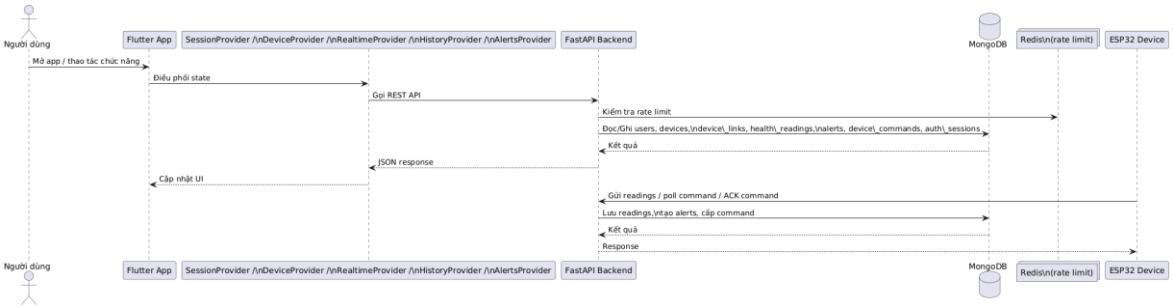
3.8 Kiến trúc giao tiếp và xác thực giữa các thành phần

Kiến trúc giao tiếp của hệ thống hiện tại được thiết kế theo hướng tách biệt rõ loại client và loại cơ chế xác thực. App và admin sử dụng HTTPS REST kết hợp `Authorization: Bearer <JWT>` và refresh token. ESP32 cũng sử dụng HTTPS REST nhưng xác thực bằng `X-Device-Token`. Backend giao tiếp với MongoDB bằng Motor async driver và với Redis bằng Redis async client để phục vụ rate limit.

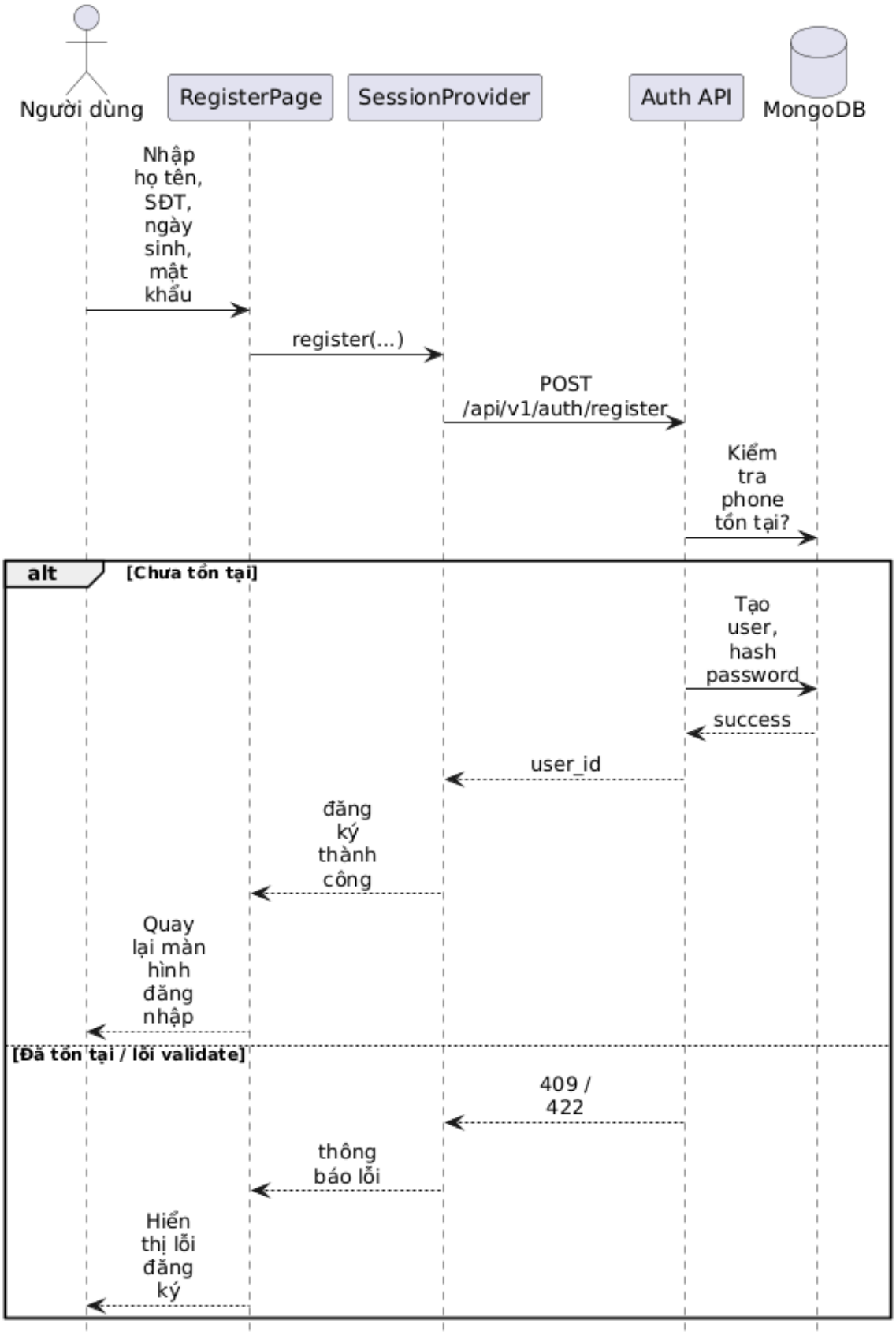
Bảng *3.4. Ma trận giao tiếp và xác thực giữa các thành phần*

Nguồn	Đích	Giao thức / kênh	Xác thực	Mục đích
ứng dụng Web	FastAPI Backend	HTTPS REST	JWT Bearer + refresh token	Đăng nhập, đồng bộ thiết bị, truy vấn latest/history/alerts/ECG, gửi yêu cầu ECG.
Web / Admin	FastAPI Backend	HTTPS REST	JWT Bearer hoặc quyền quản trị	Quản trị người dùng, thiết bị, token và giám sát dữ liệu.
ESP32	FastAPI Backend	HTTPS REST	X-Device-Token	Gửi readings, poll command, ACK command.
FastAPI Backend	MongoDB	Motor async driver	Kết nối DB nội bộ	Lưu dữ liệu sức khỏe, session, alert, command, user, device và audit.
FastAPI Backend	Redis	redis.asyncio	Kết nối hạ tầng nội bộ	Giữ trạng thái rate limit và bảo vệ API.

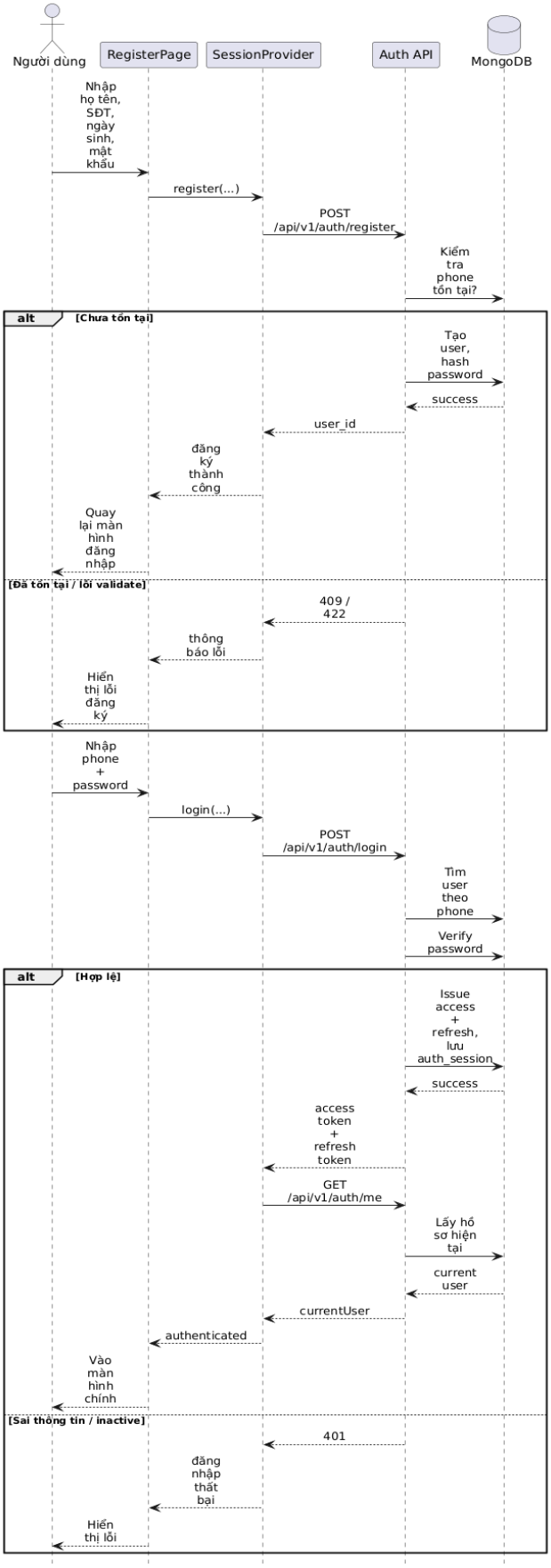
Điểm quan trọng của thiết kế này là hai lớp xác thực không bị trộn vào nhau. Người dùng cuối không cần biết token của thiết bị, còn firmware cũng không cần mang gánh nặng đăng nhập và refresh token. Đây là một quyết định kiến trúc hợp lý vì nó làm giảm độ phức tạp ở cả app lẫn thiết bị, đồng thời giữ cho backend là nơi điều phối thống nhất.



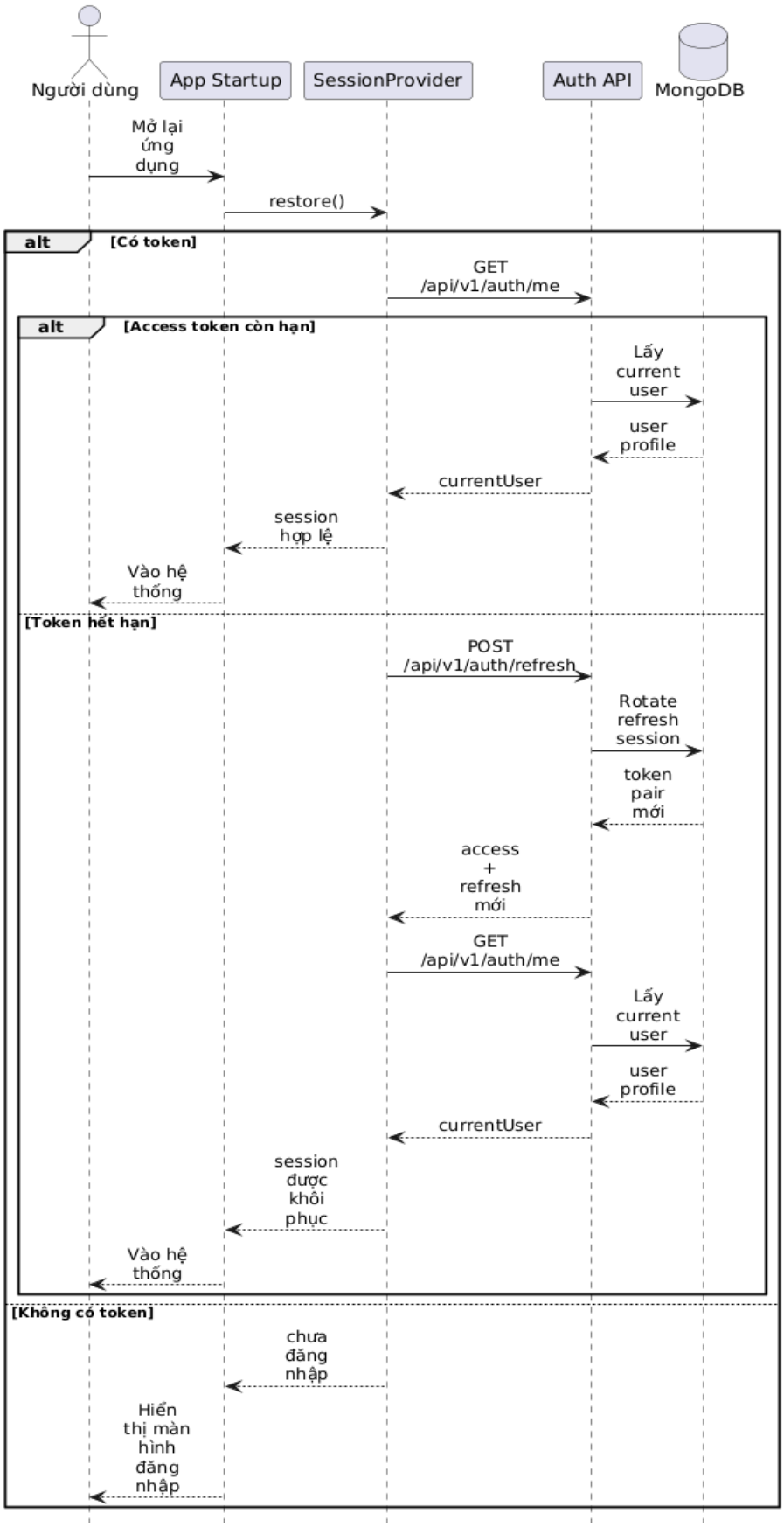
***Hình** 3.19. Sơ đồ Sequence tổng quan App ↔ API ↔ DB*



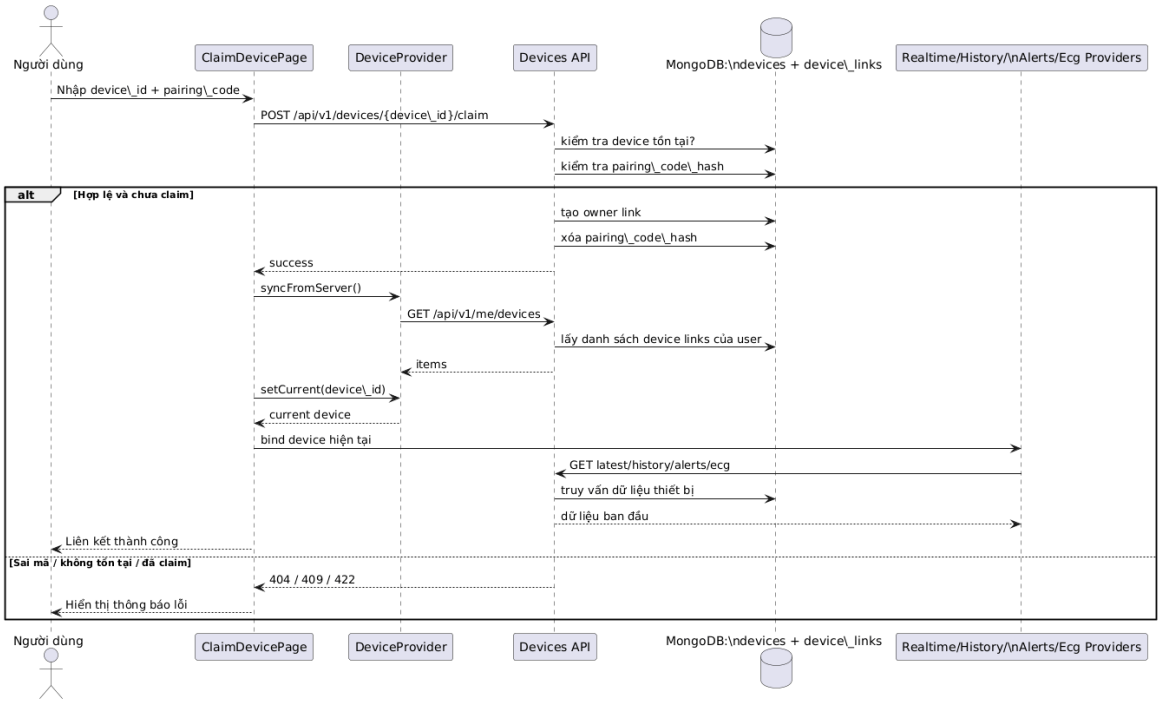
***Hình** 3.20. Sequence đăng ký*



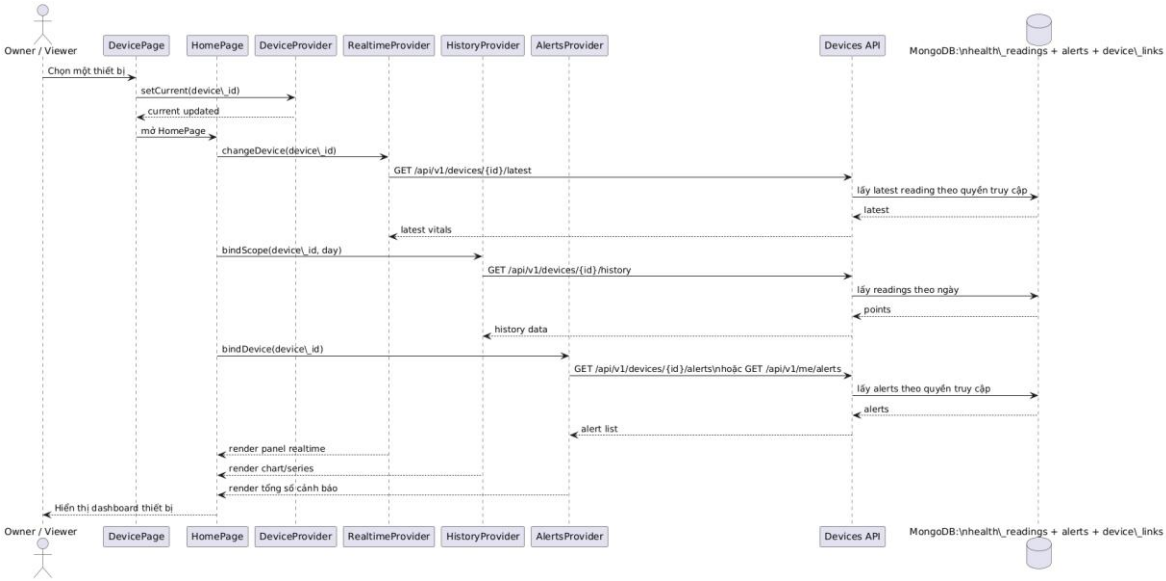
***Hình** 3.21. Sequence đăng nhập*



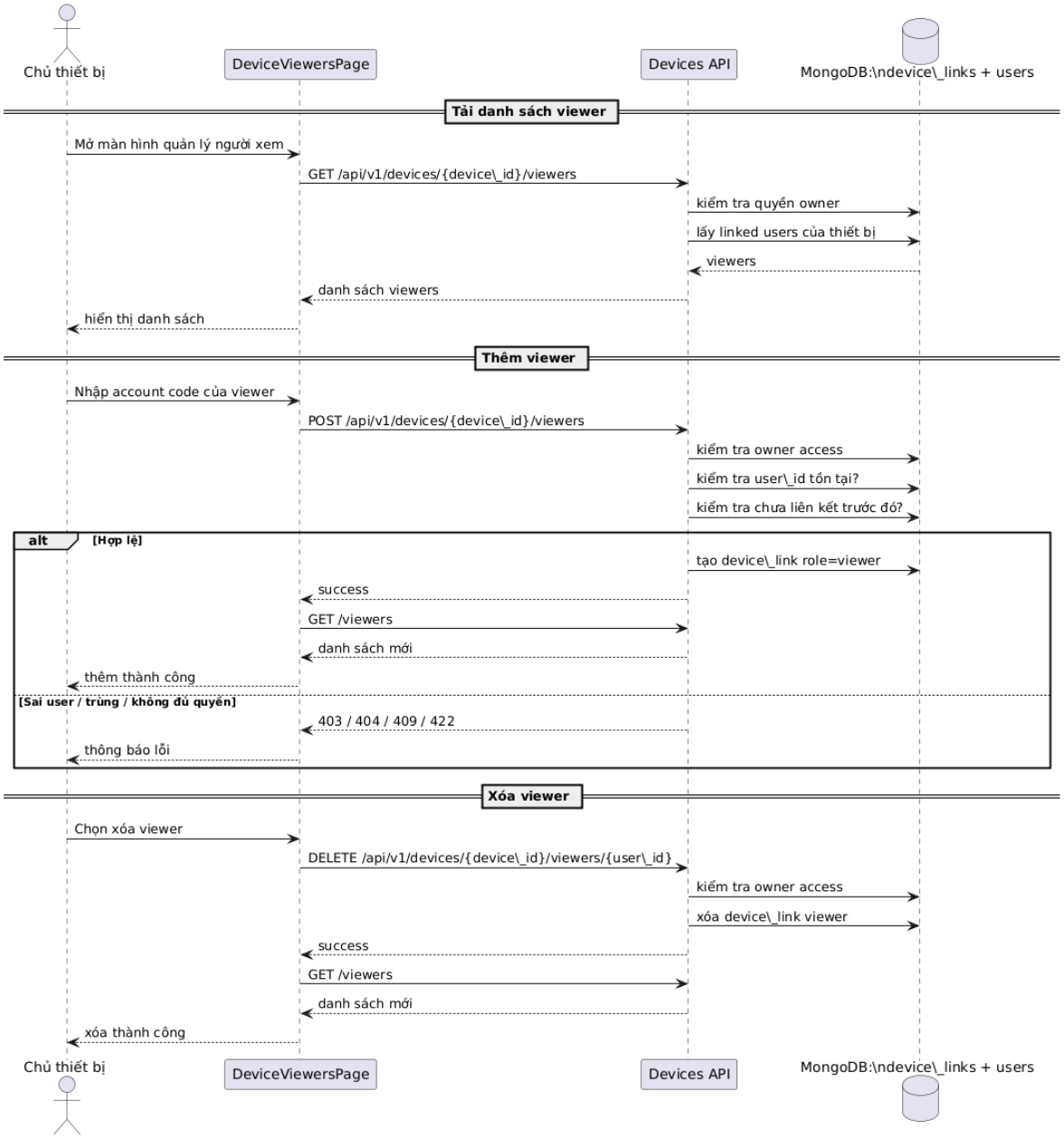
***Hình** 3.22. Sequence khôi phục phiên*



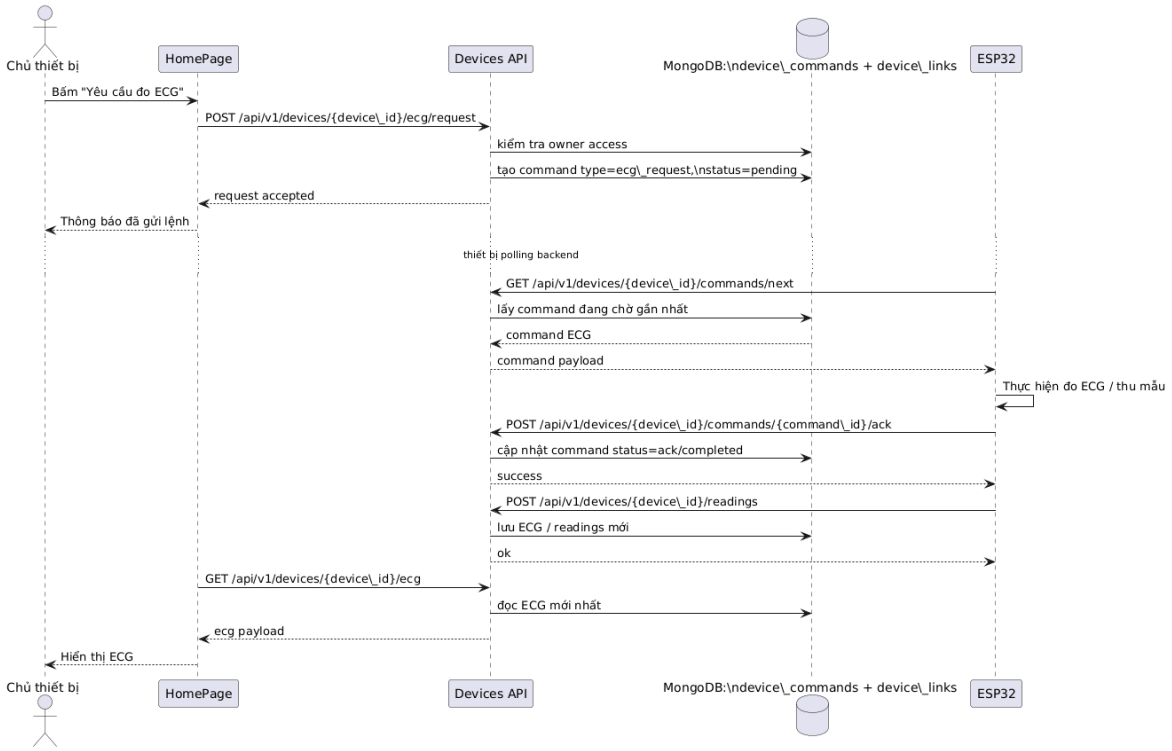
***Hình** 3.23. Sequence liên kết thiết bị bằng mã ghép nối*



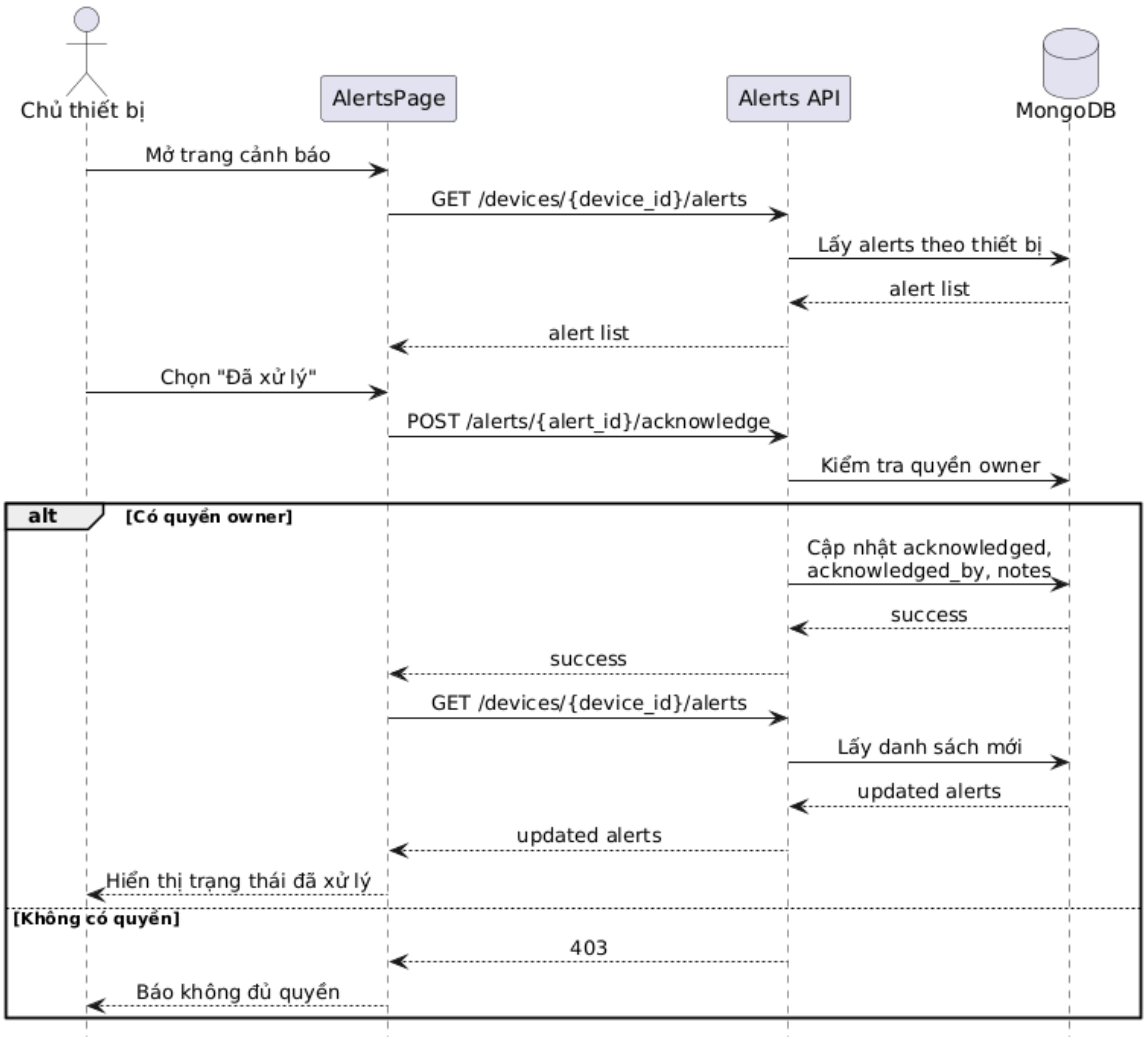
***Hình** 3.24. Sequence tải dashboard thiết bị (latest/history/alerts)*



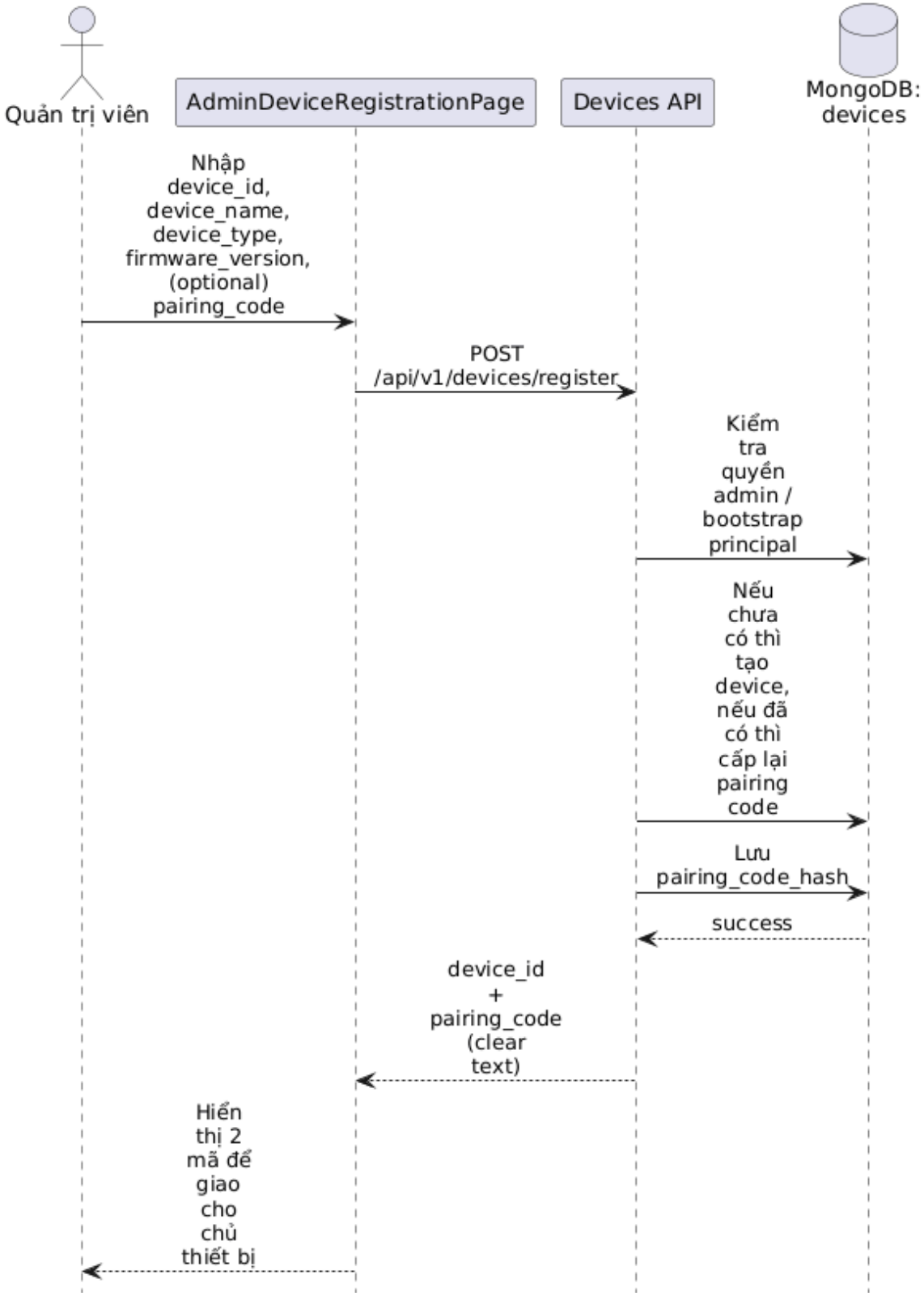
***Hình** 3.25. Sequence quản lý người xem*



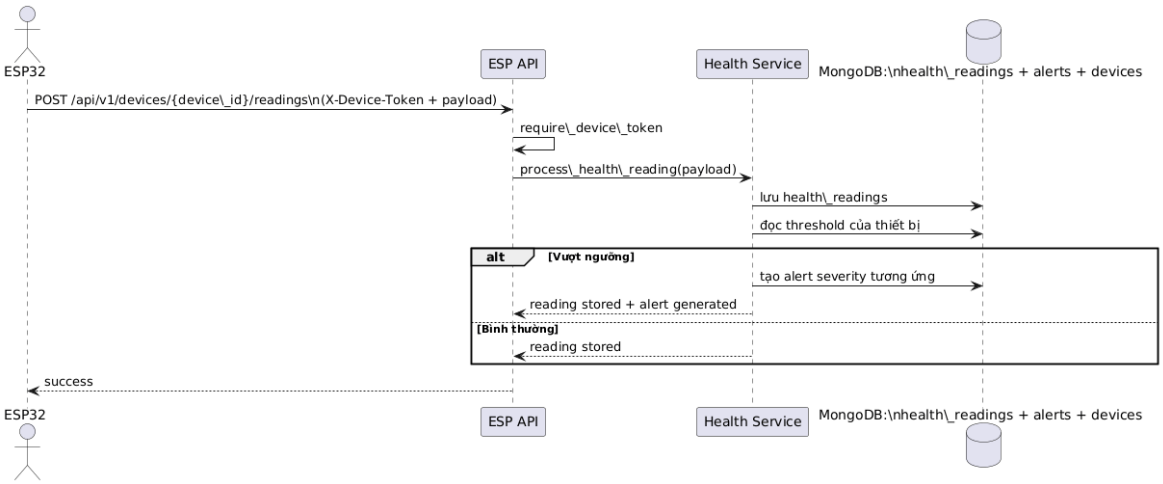
***Hình** 3.26. Sequence yêu cầu ECG theo lệnh từ app*



***Hình** 3.27. Sequence xác nhận xử lý cảnh báo*



***Hình** 3.28. Sequence admin đăng ký thiết bị / cấp lại pairing code*



***Hình** 3.29. Sequence ESP gửi readings và backend sinh cảnh báo*

3.9 Đánh giá kiến trúc hiện tại

Xét trên tổng thể, kiến trúc hiện tại có nhiều điểm mạnh đáng ghi nhận. Thứ nhất, hệ thống đã tách biệt rõ các loại client và cơ chế xác thực tương ứng. Thứ hai, backend được tổ chức theo hướng module hóa rõ ràng, bao gồm các lớp giao tiếp API, xử lý nghiệp vụ, truy cập dữ liệu, mô hình dữ liệu và các tiện ích hỗ trợ. Thứ ba, luồng ECG on-demand đã được triển khai với đầy đủ các thành phần như hàng đợi lệnh, cơ chế xác nhận (ACK), kiểm soát retry và cơ chế phục hồi trạng thái. Thứ tư, ứng dụng Web không chỉ đóng vai trò là lớp giao diện hiển thị mà còn được tổ chức với cơ chế quản lý trạng thái rõ ràng, gắn với phạm vi dữ liệu theo từng thiết bị, giúp bảo đảm tính nhất quán khi người dùng tương tác với hệ thống.

Bên cạnh đó, hệ thống cũng còn một số giới hạn cần nhìn nhận thẳng thắn. Command queue hiện dùng Mongo polling nên phù hợp hơn với quy mô nhỏ và trung bình. Phần firmware chưa được mô tả thành một repo riêng với mức chi tiết tương xứng với backend và app. Stack observability hiện đã có health check, metrics, logging và rate limit, nhưng chưa phải một bộ production hoàn chỉnh như kiến trúc lớn dùng Prometheus, Grafana và Alertmanager tích hợp sâu.

Dù vậy, với phạm vi một đồ án tốt nghiệp hoặc một nguyên mẫu nghiên cứu ứng dụng, kiến trúc hiện tại là phù hợp. Nó đủ rõ để chứng minh luồng hoạt động, đủ thực tế để phản ánh bài toán IoT + backend + ứng dụng web, và đủ mở để phát triển tiếp khi cần mở rộng phạm vi hoặc quy mô triển khai.

3.10 Kết luận chương

Chương 3 đã tái cấu trúc việc phân tích yêu cầu và kiến trúc hệ thống theo một hướng mạch lạc hơn. Thay vì chỉ liệt kê rời rạc từng mục nhỏ, chương này cho thấy hệ thống được hình thành từ một vòng kín gồm thiết bị ESP32, backend FastAPI, MongoDB, Redis và ứng dụng Web. Trên nền đó, các yêu cầu chức năng và phi chức

năng, các tác nhân tham gia, kiến trúc phân lớp, dữ liệu logic và luồng nghiệp vụ chính đều có thể được hiểu như các lát cắt khác nhau của cùng một hệ thống.

Một kết luận quan trọng được rút ra là backend đóng vai trò hạt nhân của toàn bộ giải pháp. Từ xác thực, quản lý phiên, ingest readings, sinh cảnh báo, điều phối command ECG cho đến cung cấp dữ liệu cho app, mọi luồng giá trị đều hội tụ về backend. Vì vậy, chương tiếp theo sẽ đi sâu vào phân tích và thiết kế backend ở mức module, API, bảo mật, pipeline dữ liệu, queue command và vận hành hệ thống.

CHƯƠNG 4
PHÂN TÍCH VÀ THIẾT KẾ BACKEND

4.1 Vai trò và mục tiêu thiết kế backend

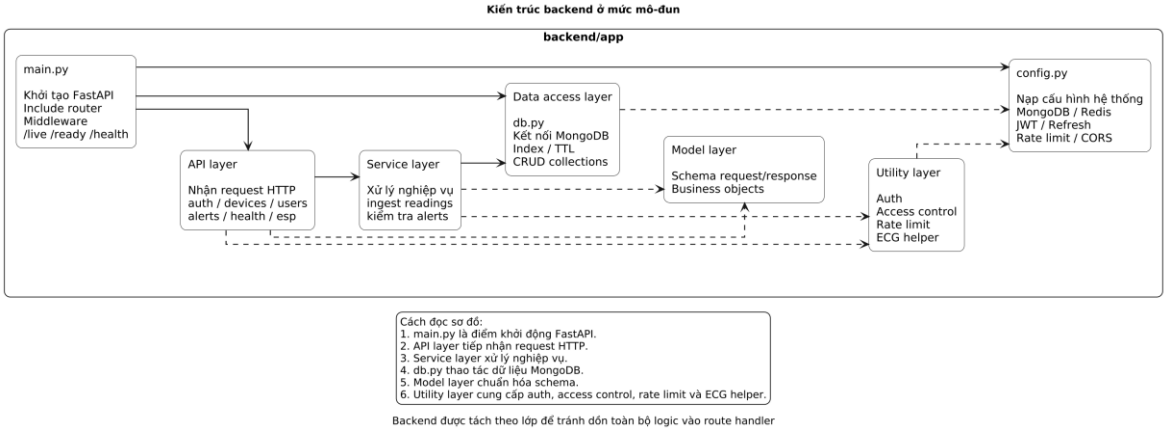
Trong hệ thống theo dõi sức khỏe từ xa, backend là thành phần chịu trách nhiệm gắn kết tất cả các lớp còn lại. Nếu ESP32 là nơi phát sinh dữ liệu và ứng dụng Web là nơi người dùng tương tác, thì backend là lớp trung gian xử lý nghiệp vụ, xác thực, lưu trữ, truy vấn và điều phối luồng dữ liệu. Nói cách khác, backend là bộ não trung tâm của toàn hệ thống.

Từ mã nguồn hiện tại, có thể thấy backend không chỉ là một tập hợp endpoint nhận và trả JSON. Nó đồng thời quản lý JWT access token, refresh session, xác thực thiết bị, quyền truy cập dữ liệu theo device, ingest readings, alerting, API truy vấn dữ liệu sức khỏe, command queue cho ECG on-demand, rate limit, health check và metrics. Đó là lý do chương này không thể chỉ mô tả route API mà cần phân tích backend như một kiến trúc hoàn chỉnh.

Mục tiêu thiết kế của backend hiện tại có thể tóm lược thành bốn hướng. Hướng thứ nhất là tiếp nhận và lưu readings một cách đúng đắn, có thể chống trùng lặp và hỗ trợ truy vấn theo thời gian. Hướng thứ hai là phục vụ nhiều loại client khác nhau trong cùng một hệ thống, gồm app-admin và ESP32. Hướng thứ ba là hỗ trợ tương tác hai chiều qua ECG on-demand. Hướng thứ tư là đủ khả năng vận hành thực tế ở mức nguyên mẫu hoàn chỉnh thông qua logging, metrics, health check và rate limit.

4.2 Kiến trúc module backend

Cấu trúc backend hiện tại thể hiện rõ tư duy phân lớp và mô-đun hóa. Phần `main.py` chịu trách nhiệm khởi động FastAPI và gắn các thành phần lại với nhau. `config.py` quản lý cấu hình runtime. Thư mục `api/` tiếp nhận request theo domain, `services/` xử lý nghiệp vụ, `db.py` là lớp truy cập dữ liệu tập trung, `models/` chuẩn hóa schema và `utils/` gom các chức năng dùng chung như auth, access control, rate limit hoặc xử lý hỗ trợ.



***Hình** 4.1. Kiến trúc backend ở mức mô-đun*

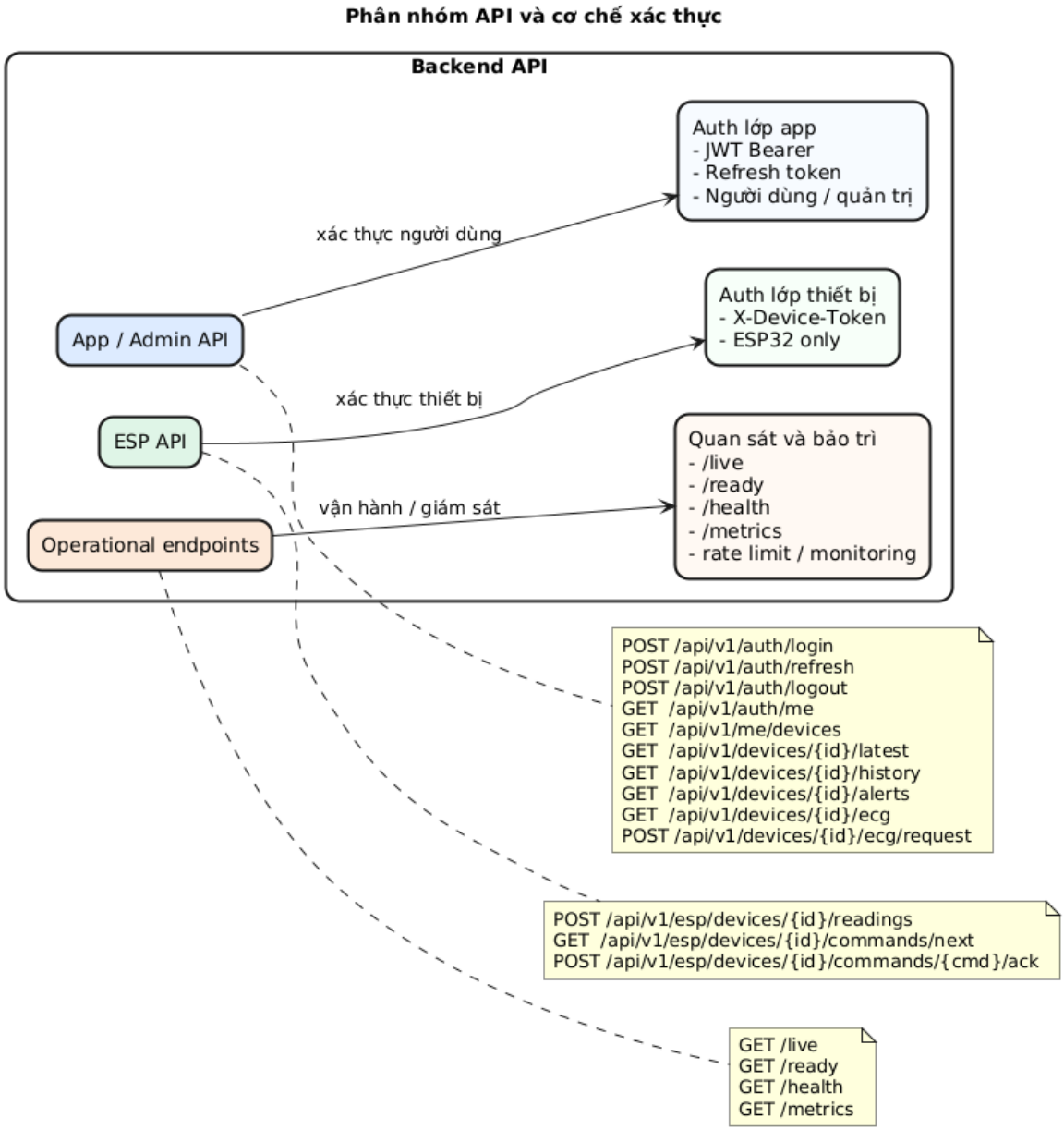
Bảng *4.1. Các mô-đun chính của backend*

Mô-đun	Vai trò	Ý nghĩa thiết kế
main.py	Khởi tạo FastAPI, đăng ký router, startup và operational endpoints.	Là điểm vào của hệ thống, nơi lắp ghép toàn bộ thành phần.
config.py	Đọc biến môi trường và các tham số vận hành.	Tách cấu hình khỏi logic xử lý để dễ thay đổi môi trường chạy.
api/	Nhận request HTTP theo domain nghiệp vụ.	Giữ route handler gọn, không dồn toàn bộ logic vào endpoint.
services/	Xử lý business logic như ingest và alerting.	Tạo lớp nghiệp vụ độc lập với HTTP và dữ liệu.
db.py	Kết nối MongoDB, tạo index, thao tác collections.	Hoạt động như repository/DAO trung tâm của backend.
models/	Schema request/response và business object.	Bảo đảm dữ liệu vào/ra có cấu trúc thống nhất.
utils/	Auth helper, access control, validator, rate limit, xử lý hỗ trợ.	Tránh lặp code và cung cấp các cơ chế chung cho nhiều lớp.

Ưu điểm của cách tổ chức này là giảm độ phức tạp trong từng file, tách logic HTTP khỏi logic nghiệp vụ, làm cho việc kiểm thử thuận lợi hơn và giữ cho code có thể tái sử dụng. Trong bối cảnh đồ án, đây là một điểm mạnh vì nó cho thấy backend không được viết theo kiểu dồn tất cả vào route handler, mà được xây dựng như một hệ thống có cấu trúc.

4.3 Thiết kế lớp giao tiếp API

Lớp API của backend được thiết kế theo phong cách REST, với các endpoint được chia nhóm theo loại client và theo domain nghiệp vụ. Sự phân nhóm này giúp client dễ sử dụng hơn và cũng giúp code phía server dễ tổ chức dependency xác thực, phân quyền và validate request.



***Hình** 4.2. Phân nhóm API và cơ chế xác thực*

Nhóm thứ nhất là App/Admin API dưới tiền tố `/api/v1/*`. Đây là nơi ứng dụng gọi các route đăng nhập, refresh token, lấy thông tin người dùng, lấy danh sách thiết bị, truy vấn latest/history/alerts/ECG và gửi yêu cầu ECG. Nhóm thứ hai là ESP API dưới tiền tố `/api/v1/esp/*`, tối giản hơn và dành riêng cho firmware nhúng. Nhóm thứ ba là operational endpoints như `/live`, `/ready`, `/health` và `/metrics` để phục vụ kiểm tra và giám sát hệ thống.

Bảng *4.2. Nhóm endpoint chính của backend*

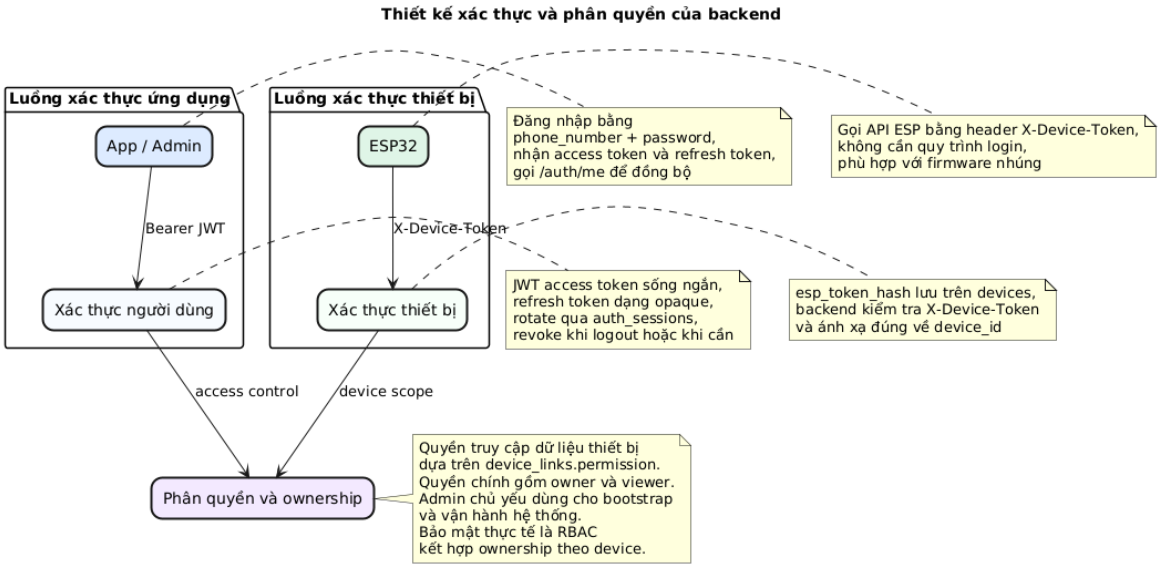
Nhóm API	Đường dẫn tiêu biểu	Lớp xác thực	Mục đích
Auth	`/api/v1/auth/login`, `/refresh`, `/logout`, `/me`	JWT + refresh token	Tạo, duy trì và hủy phiên người dùng.
Devices	`/api/v1/me/devices`, `/devices/{id}/latest`, `/history`, `/ecg`, `/ecg/request`	JWT + ownership theo device	Phục vụ mô hình device-centric của app.
ESP	`/api/v1/esp/devices/{id}/readings`, `/commands/next`, `/ack`	X-Device-Token	Ingest readings và thực thi command polling.
Operational	`/live`, `/ready`, `/health`, `/metrics`	Nội bộ / quan sát vận hành	Kiểm tra trạng thái và theo dõi runtime của backend.

Nhìn từ góc độ thiết kế, sự phân nhóm route hiện tại làm rõ ranh giới trách nhiệm giữa app, thiết bị và công cụ vận hành. Đây là một lựa chọn đúng vì nó giảm nguy cơ trộn lẫn logic người dùng với logic thiết bị, đồng thời cho phép backend áp dụng dependency xác thực và kiểm tra quyền riêng cho từng nhóm route.

4.4 Thiết kế xác thực và phân quyền

Backend duy trì hai cơ chế xác thực tách biệt. Đối với app và admin, hệ thống dùng access token JWT cho các request thường ngày và refresh token để làm mới phiên. Refresh token hiện là token dạng opaque, được hash và lưu trong `auth_sessions` để hỗ trợ rotate và revoke session. Thiết kế này vừa giảm rủi ro khi access token bị lộ, vừa duy trì trải nghiệm sử dụng tốt cho web.

Đối với thiết bị ESP32, backend sử dụng `X-Device-Token`. Cơ chế này đơn giản hơn nhiều so với đăng nhập bằng tài khoản người dùng, phù hợp với firmware nhúng vì thiết bị chỉ cần gắn token trong header của mỗi request. Token không được lưu thô mà chỉ lưu hash ở phía server, giúp tăng an toàn nếu database bị lộ hoặc cần rotate danh tính của thiết bị.



***Hình** 4.3. Mô hình xác thực và phân quyền của backend*

Về phân quyền, cần nhấn mạnh rằng mô hình hiện tại của code mang tính device-centric hơn là role-centric thuần túy. Ở mức nghiệp vụ, có thể diễn giải người sở hữu thiết bị và người theo dõi như patient và caregiver. Tuy nhiên trong code hiện tại, quyền truy cập dữ liệu hằng ngày chủ yếu được suy ra từ `device_links.permission`, với các giá trị chính là `owner` và `viewer`; còn `admin` chỉ đóng vai trò hệ thống cho bootstrap hoặc vận hành. Như vậy mô hình bảo mật đúng hơn khi được mô tả là RBAC kết hợp ownership theo đối tượng dữ liệu cụ thể.

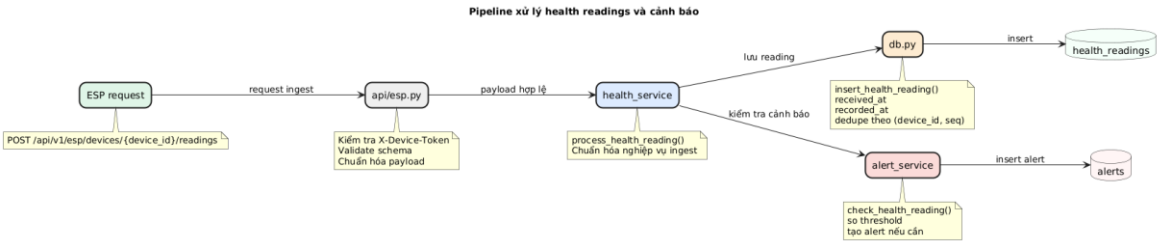
Bảng *4.3. So sánh hai lớp xác thực trong backend*

Khía cạnh	Người dùng / App	Thiết bị ESP32
Cơ chế	JWT access token + refresh token	X-Device-Token
Mục tiêu	Duy trì phiên và phân quyền cho người dùng	Xác thực danh tính thiết bị ở từng request
Lưu trạng thái	Có `auth_sessions` để rotate và revoke session	Hash token nằm trong `devices`
Độ phức tạp	Cao hơn vì cần đồng bộ phiên và quyền	Gọn hơn vì phù hợp cho firmware
Lợi ích chính	Trải nghiệm web tốt, access token sống ngắn	Đơn giản cho nhúng, tách biệt với auth của app

4.5 Thiết kế xử lý health readings và cảnh báo

Pipeline xử lý readings là trục chính của backend vì mọi chức năng khác như latest, history, summary, alert hay ECG đều phụ thuộc vào dữ liệu đi qua pipeline này. Khi ESP32 gửi request đến `/api/v1/esp/devices/{device_id}/readings`, route trong `api/esp.py` trước hết xác thực thiết bị, validate schema đầu vào và chuẩn hóa payload. Sau đó backend gọi `health_service.process_health_reading()` để xử lý logic ingest, rồi gọi `db.insert_health_reading()` để lưu document vào MongoDB.

Sau khi reading được lưu thành công, backend tiếp tục kích hoạt `alert_service.check_health_reading()` để so sánh readings với ngưỡng cấu hình. Nếu giá trị vượt ngưỡng, hệ thống sinh alert mới trong collection `alerts`. Thiết kế này rất quan trọng vì nó cho thấy cảnh báo không phải một bước xử lý rời rạc ở phía giao diện mà được gắn chặt ngay trong pipeline ingest.



***Hình** 4.4. Pipeline xử lý readings và sinh cảnh báo*

Ở mức dữ liệu, pipeline còn có một quyết định kỹ thuật rất tốt là dedupe readings bằng `(device_id, seq)`. Trong môi trường IoT, retransmission là điều dễ xảy ra khi mạng không ổn định. Nhờ unique partial index trên cặp này và logic xử lý `duplicate` trong `insert_health_reading()`, backend có thể xem việc resend như một tình huống hợp lệ thay vì coi đó là lỗi hệ thống.

Hệ quả của thiết kế trên là cùng một pipeline vừa phục vụ việc ghi dữ liệu liên tục từ thiết bị, vừa phục vụ các truy vấn realtime và lịch sử ở phía ứng dụng. Đây là một trong những lý do khiến backend hiện tại có mức hoàn thiện cao hơn nhiều hệ thống demo chỉ tập trung vào lưu dữ liệu thô.

4.6 Thiết kế quản lý thiết bị và liên kết người dùng

Trong backend hiện tại, thiết bị là một thực thể độc lập với vòng đời riêng. Collection `devices` lưu thông tin như `device_id`, `device_type`, `status`, `metadata`, `esp_token_hash`, `last_seen` và một số trường mô tả mở rộng khác. Các nghiệp vụ như đăng ký thiết bị, cập nhật metadata, xoay token, cập nhật `last_seen` hay đọc trạng thái thiết bị đều xoay quanh collection này.

Tuy nhiên, quan hệ giữa thiết bị và người dùng không còn được mô hình hóa đơn giản bằng một trường `user_id` cứng trên document thiết bị. Thay vào đó, `device_links` giữ vai trò lớp liên kết mềm giữa user và device. Nhờ collection này,

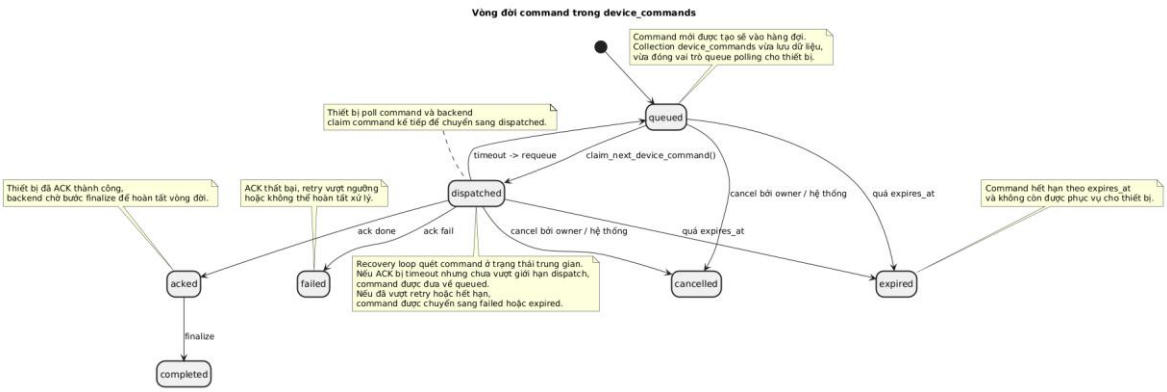
backend có thể hỗ trợ một thiết bị có nhiều user cùng truy cập theo các mức quyền khác nhau, đồng thời giữ cho app vận hành theo mô hình device-centric một cách tự nhiên.

Trường `last_seen` trong `devices` có ý nghĩa thực tiễn cao vì nó phản ánh thời điểm thiết bị tương tác gần nhất với backend. Trong hệ thống theo dõi sức khỏe, việc biết thiết bị còn online hay đã lâu không gửi dữ liệu quan trọng gần như ngang với bản thân dữ liệu readings, bởi nó ảnh hưởng trực tiếp đến cách app giải thích tình trạng 'không có dữ liệu mới' cho người dùng.

4.7 Thiết kế command queue cho ECG on-demand

Cơ chế ECG on-demand là một trong những phần nổi bật nhất của backend hiện tại. Thay vì chỉ chờ thiết bị gửi dữ liệu theo chu kỳ, hệ thống cho phép app chủ động yêu cầu một lần đo ECG mới. Để hiện thực điều này trong bối cảnh ESP32 không duy trì kết nối WebSocket hai chiều liên tục với server, backend sử dụng collection `device_commands` như một polling command queue.

Khi người dùng nhấn yêu cầu ECG trên app, backend tạo command mới trong `device_commands`. Thiết bị sẽ poll endpoint `commands/next` để lấy command phù hợp, thực thi tác vụ, rồi gửi ACK về backend. Command sau đó đi qua các trạng thái `queued`, `dispatched`, `acked`, `completed`, `failed`, `expired` hoặc `cancelled` tùy theo quá trình thực thi.



***Hình** 4.5. Vòng đời command trong collection device_commands*

Điểm mạnh của thiết kế này là backend không dừng ở mức có queue, mà còn có recovery loop để quét các command bị timeout, cơ chế giới hạn số command pending trên từng thiết bị, dedupe bằng `dedupe_key` và logic finalize từ `acked` sang `completed`. Nhờ đó, queue hiện tại đủ ổn định cho một nguyên mẫu hoàn chỉnh, đồng thời vẫn dễ triển khai vì tận dụng MongoDB sẵn có.

Tuy vậy, cũng cần nhìn nhận cân bằng rằng cách làm này phù hợp hơn cho quy mô nhỏ và trung bình. Khi tải tăng rất lớn, polling queue trên MongoDB sẽ không tối ưu bằng các giải pháp message broker chuyên dụng. Sự đánh đổi này là hợp lý trong

phạm vi hiện tại của đề tài vì ưu tiên lớn hơn là sự đơn giản, khả năng debug và tính nhất quán với kiến trúc đang có.

4.8 Thiết kế lớp truy cập dữ liệu và mô hình lưu trữ

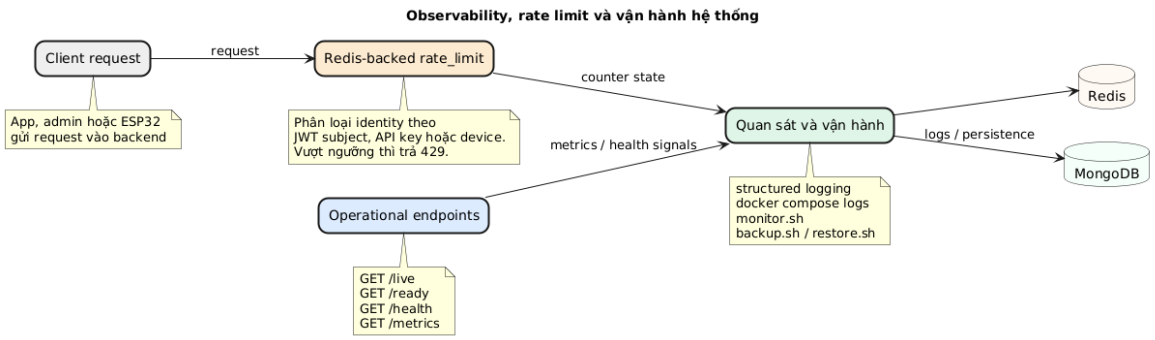
`db.py` là lớp truy cập dữ liệu trung tâm của backend. Toàn bộ logic kết nối MongoDB, tạo index, thiết lập TTL và thao tác trên các collection chính đều được gom về đây. Cách làm này giúp route và service không phải viết truy vấn trực tiếp, từ đó giảm trùng lặp và làm cho lớp repository của hệ thống rõ ràng hơn.

Trong `db.py`, backend hiện xử lý nhiều nhóm thao tác như user, device, readings, alerts, sessions, commands và cả các hàm utility phục vụ đồng bộ hóa quan hệ logic. Đây là một quyết định thiết kế hợp lý vì NoSQL cho phép tổ chức dữ liệu linh hoạt, nhưng để giữ được tính nhất quán nghiệp vụ thì vẫn cần một lớp trung tâm chịu trách nhiệm chuẩn hóa cách đọc và ghi dữ liệu.

Ở mức hiệu năng, lớp dữ liệu hiện tại đã được chuẩn bị tương đối tốt. Các index phục vụ latest và history theo device, unique partial index cho dedupe readings, TTL cho dữ liệu cũ và index cho queue command đều được tạo ngay từ code. Điều đó cho thấy thiết kế backend hiện tại không chỉ hướng đến việc 'chạy được' mà còn chú ý tới hành vi runtime và chi phí lưu trữ dài hạn.

4.9 Thiết kế observability, rate limit và vận hành

Một dấu hiệu rõ ràng cho thấy backend đã vượt qua mức của một API demo là sự hiện diện của lớp observability và vận hành. Backend hiện hỗ trợ `GET /live`, `GET /ready`, `GET /health` và `GET /metrics`, nhờ đó có thể được kiểm tra trong nhiều ngữ cảnh như health check của container, readiness cho orchestration hoặc thu thập metric khi cần giám sát hệ thống.



***Hình** 4.6. Lớp observability, rate limit và vận hành*

Cùng với đó, backend hiện có logging qua Docker Compose, các script monitor, backup và restore, cũng như `Redis-backed rate limit` để giới hạn request theo JWT subject, API key hoặc device. Cơ chế rate limit này đặc biệt quan trọng đối với hệ

thống sức khỏe vì cả app lẫn thiết bị đều có thể tạo ra lượng request lặp lại cao nếu gặp lỗi logic hoặc lỗi mạng.

Bảng *4.4. Các cơ chế quan sát và bảo vệ API*

Cơ chế	Hiện thực hiện tại	Giá trị mang lại
Liveness / readiness / health	`/live`, `/ready`, `/health`	Xác định service còn chạy, đã sẵn sàng nhận traffic hay chưa và có thể kết nối hạ tầng không.
Metrics	`/metrics` và các counter cho auth, ingest, queue, alert	Quan sát hành vi runtime, phát hiện điểm nghẽn hoặc tỷ lệ lỗi tăng bất thường.
Logging	Docker Compose logs, file log và monitor script	Phục vụ debug ingest, auth, queue command và truy vết sự cố.
Rate limit	Redis async client với identity theo user, API key hoặc device	Bảo vệ API khỏi spam request, giữ ổn định cho backend.

4.10 Đánh giá backend hiện tại

Nếu so với mặt bằng thông thường của một đồ án, backend hiện tại là một thiết kế khá đầy đủ. Nó có kiến trúc mô-đun rõ ràng, hỗ trợ hai cơ chế xác thực tách biệt, có refresh token và session store, có pipeline ingest và alerting rõ ràng, có command queue cho ECG on-demand, có index và TTL cho dữ liệu MongoDB, đồng thời có health check, metrics, logging và rate limit.

Những điểm mạnh này cho thấy backend không chỉ được xây dựng để minh họa về mặt học thuật mà còn được đặt trong bối cảnh vận hành thực tế. Điều này làm tăng đáng kể giá trị của đề tài, vì người đọc có thể thấy rõ hệ thống không dừng ở mức giao diện hoặc cảm biến đơn lẻ mà là một nền tảng API tích hợp tương đối hoàn chỉnh.

Dù vậy, cũng cần nhìn nhận một số hạn chế. Command queue hiện vẫn dùng Mongo polling nên chưa tối ưu cho tải rất lớn. Kiến trúc hiện tại chưa phải kiến trúc microservices; mọi domain vẫn tập trung trong một backend duy nhất. Stack observability dù đã khá tốt nhưng chưa đạt đến mức production stack đầy đủ với dashboard và alert manager tích hợp sâu. Ngoài ra, nếu muốn mở rộng triển khai sau này, integration test với MongoDB và Redis thật sẽ cần được đầu tư nhiều hơn.

Tổng thể mà nói, backend hiện tại là phù hợp với phạm vi của một hệ thống theo dõi sức khỏe từ xa ở mức nguyên mẫu hoàn chỉnh. Nó vừa thể hiện được tính kỹ

thuật, vừa đủ gần với thực tế để làm nền tảng cho những bước mở rộng trong tương lai.

4.11 Kết luận chương

Nhờ thiết kế backend tương đối đầy đủ và chặt chẽ, hệ thống hiện có thể hỗ trợ không chỉ bài toán ingest dữ liệu từ ESP32 mà còn cả bài toán session người dùng, phân quyền theo device, cảnh báo theo ngưỡng và điều phối tương tác hai chiều giữa app với thiết bị. Đây cũng là nền tảng trực tiếp để chương tiếp theo trình bày sâu hơn về thiết kế cơ sở dữ liệu MongoDB, nơi hiện thực hóa toàn bộ mô hình users, devices, health_readings, alerts, device_commands, auth_sessions và device_links.

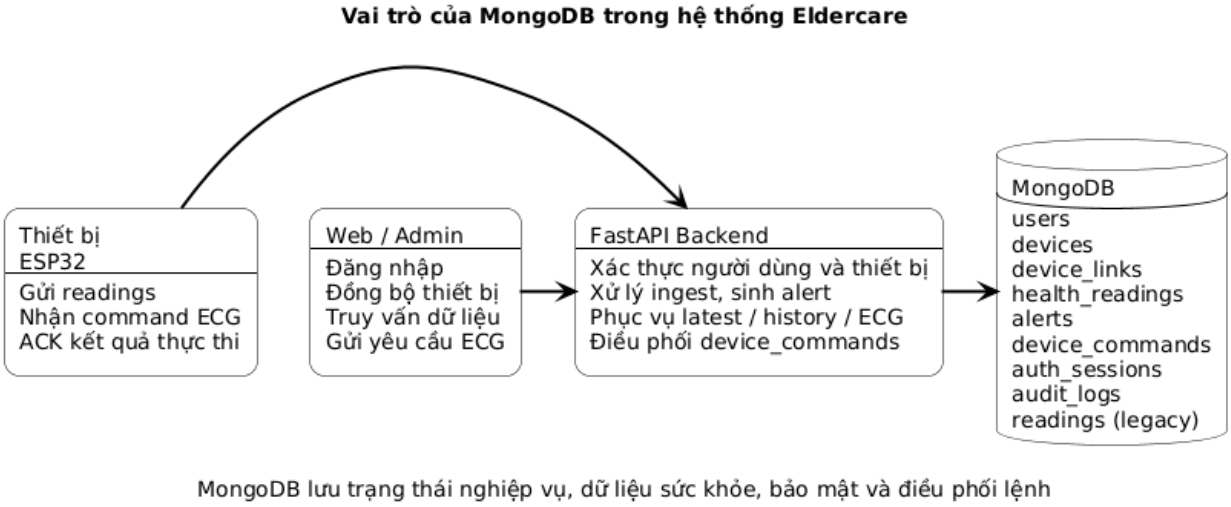
Nhìn ở góc độ kỹ thuật, điểm đáng giá nhất của backend không nằm ở một endpoint đơn lẻ, mà nằm ở việc các cơ chế nhỏ được nối với nhau thành một pipeline hoàn chỉnh: người dùng đăng nhập và duy trì session, thiết bị được xác thực riêng, readings được chuẩn hóa và lưu vào MongoDB, alerts được sinh ra từ chính pipeline ingest, còn ECG on-demand được điều phối qua command queue có khả năng phục hồi khi mạng không ổn định. Sự liên kết giữa các lớp này làm cho backend hiện tại trở thành một nền tảng thực sự có thể vận hành, thay vì chỉ là một API minh họa.

CHƯƠNG 5
THIẾT KẾ CƠ SỞ DỮ LIỆU

5.1 Vai trò của cơ sở dữ liệu và định hướng thiết kế

Trong hệ thống theo dõi sức khỏe từ xa, cơ sở dữ liệu không chỉ đóng vai trò là nơi lưu trữ thụ động mà còn là lớp duy trì toàn bộ trạng thái nghiệp vụ của hệ thống. Nếu backend là nơi xác thực, điều phối và xử lý logic, thì MongoDB là nơi bảo toàn các thực thể và sự kiện phát sinh theo thời gian, bao gồm thông tin người dùng, thiết bị, liên kết giữa người dùng với thiết bị, dữ liệu sức khỏe, cảnh báo, phiên đăng nhập, command ECG on-demand và dữ liệu audit. Cách tổ chức này khiến cơ sở dữ liệu trở thành nền móng cho cả hai chiều vận hành: chiều ghi dữ liệu liên tục từ thiết bị ESP32 và chiều đọc dữ liệu theo nhu cầu của ứng dụng Web.

Định hướng thiết kế dữ liệu của hệ thống được đặt trên ba yêu cầu lớn. Thứ nhất, mô hình dữ liệu phải đủ linh hoạt để chứa được readings có cấu trúc bán linh hoạt, trong đó một bản ghi có thể đồng thời mang vitals, ECG, metadata và các mốc thời gian khác nhau. Thứ hai, lớp dữ liệu phải hỗ trợ tốt các truy vấn thực tế như latest, history, ECG gần nhất, alert gần nhất và trạng thái command theo thiết bị. Thứ ba, dữ liệu phải có vòng đời rõ ràng để hệ thống không phình to theo thời gian; vì vậy TTL, dedupe, soft relation và retention là những quyết định thiết kế quan trọng chứ không phải phần bổ sung về sau.



Hình 5.1. Vai trò của MongoDB trong kiến trúc tổng thể hệ thống.

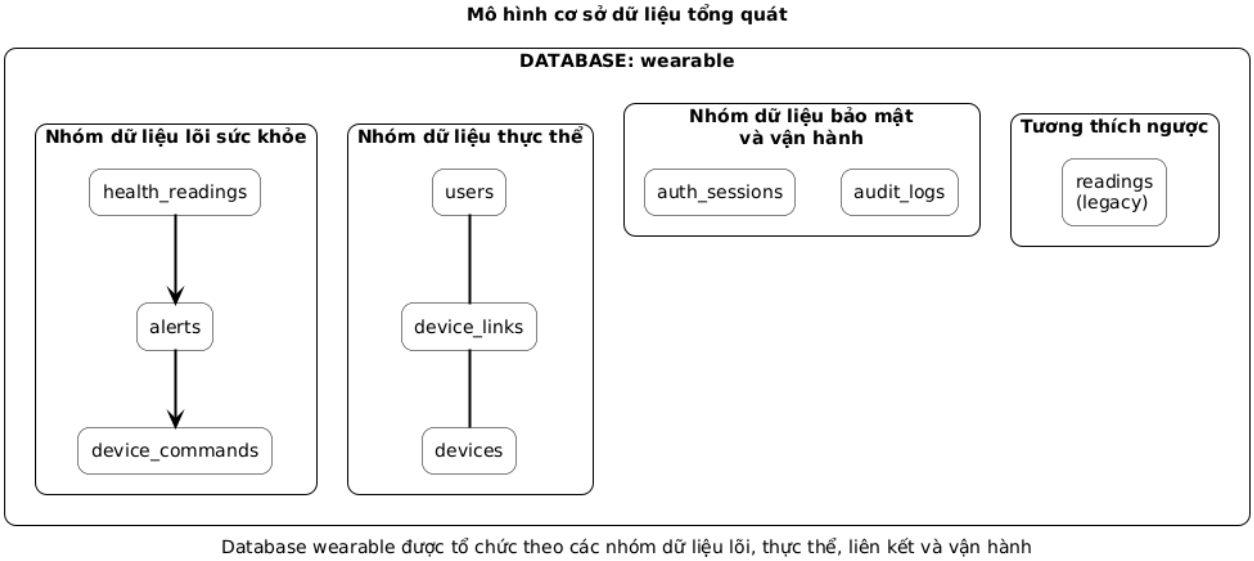
5.2 Lý do lựa chọn MongoDB và mô hình tổng quát của database

MongoDB được lựa chọn vì phù hợp với bản chất document của dữ liệu sức khỏe. Trong một reading, các trường như vitals và ECG thường có cấu trúc lồng nhau, không phải lúc nào cũng xuất hiện với cùng hình dạng và cũng không thuận tiện nếu phải chuẩn hóa thành quá nhiều bảng quan hệ. Mô hình document giúp backend lưu trực tiếp payload gần với bản chất dữ liệu đo, từ đó rút ngắn luồng ingest và giảm chi phí chuyển đổi dữ liệu ở tầng persistence.

Bên cạnh tính linh hoạt, MongoDB cũng phù hợp với mô hình truy vấn chủ đạo của hệ thống. Hệ thống không cần các join phức tạp theo kiểu giao dịch truyền thống, mà chủ yếu cần truy xuất document mới nhất, danh sách readings theo khoảng thời

gian, cảnh báo theo thiết bị, session theo refresh token hash và command theo trạng thái. Đây là loại truy vấn có thể tối ưu tốt bằng index ghép trên các collection độc lập. Quan trọng hơn, trong thiết kế hiện tại MongoDB còn được tận dụng để triển khai device_commands như một command queue polling cho ECG on-demand, nhờ đó hệ thống không cần thêm một message broker riêng ở giai đoạn nguyên mẫu.

Database mặc định của backend được tổ chức dưới tên wearable. Bên trong database này, các collection không nằm rời rạc mà được gom thành các nhóm dữ liệu có vai trò rõ ràng. Nhóm lõi sức khỏe tập trung vào health_readings, alerts và device_commands; nhóm thực thể gồm users và devices; nhóm liên kết logic được thể hiện qua device_links; nhóm bảo mật và vận hành gồm auth_sessions và audit_logs; ngoài ra còn có readings như một collection legacy để giữ tương thích ngược với mô hình cũ.



Hình 5.2. Mô hình tổng quát của database wearable và các nhóm collection chính.

Bảng 5.1. Tổng quan vai trò của các collection chính trong hệ thống.

Collection	Vai trò nghiệp vụ	Liên kết logic nổi bật	Vòng đời dữ liệu
health_readings	Lưu readings sức khỏe và ECG do thiết bị gửi lên	Gắn với device_id; được truy xuất theo user thông qua device_links	TTL khoảng 90 ngày
alerts	Lưu cảnh báo vượt ngưỡng và trạng thái acknowledged	Gắn với device_id và tập người nhận liên quan	TTL khoảng 180 ngày
device_commands	Hàng đợi command cho ECG on-demand và các trạng thái xử lý	Gắn với device_id, request_id, command và retry state	Tự dọn theo expires_at

users	Lưu hồ sơ người dùng và dữ liệu xác thực liên quan	Liên kết mềm với devices qua device_links	Duy trì lâu dài
devices	Lưu định danh thiết bị, token hash, metadata, last_seen	Liên kết mềm với users qua device_links	Duy trì lâu dài
device_links	Biểu diễn quan hệ owner/viewer giữa user và device	Là lớp nối trung tâm của mô hình device-centric	Duy trì lâu dài
auth_sessions	Lưu session và refresh token hash	Gắn với user_id và trạng thái revoke	Tự dọn theo expires_at
audit_logs	Lưu các sự kiện audit và truy vết vận hành	Gắn actor, action và target	Duy trì tùy chính sách
readings	Collection legacy cho tương thích ngược	Không phải hướng lưu trữ chính mới	Duy trì để chuyển tiếp hệ thống

5.3 Nhóm dữ liệu lõi sức khỏe và điều phối nghiệp vụ

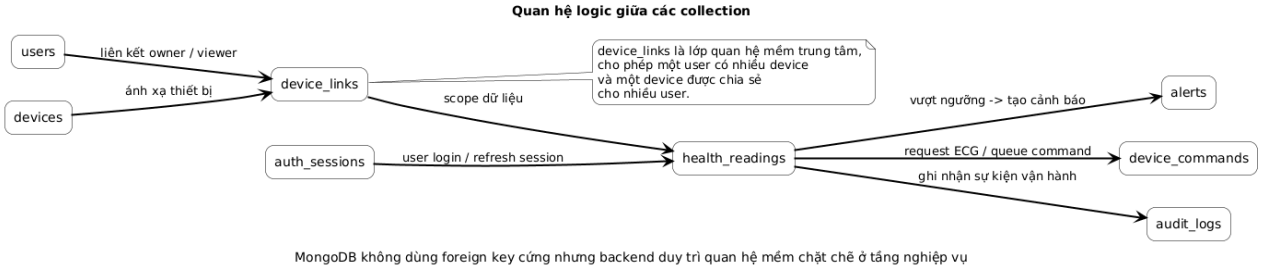
Trong nhóm dữ liệu lõi, health_readings là collection quan trọng nhất vì đây là nơi backend lưu mọi bản ghi sức khỏe được gửi từ thiết bị ESP32. Một document trong collection này có thể mang device_id, timestamp, vitals, ECG, metadata, recorded_at, received_at và seq. Cách thiết kế này cho phép một reading vừa phản ánh mốc thời gian logic của thiết bị, vừa mang mốc backend thực sự nhận dữ liệu, từ đó hỗ trợ tốt cho cả nghiệp vụ lẫn retention. Khi insert reading, backend tự bổ sung received_at nếu cần và chuyển timestamp sang recorded_at dạng datetime để phục vụ TTL.

Điểm mạnh của health_readings nằm ở chiến lược chống trùng lặp và tối ưu truy vấn. Backend tạo index theo device_id và timestamp để phục vụ latest, history; đồng thời tạo unique partial index trên cặp device_id và seq khi seq tồn tại dưới dạng số. Với thiết kế này, trường hợp thiết bị resend cùng một payload do mạng không ổn định sẽ được xem là duplicate QoS1 retransmission thay vì lỗi hệ thống. Đây là một quyết định rất phù hợp với môi trường IoT vì tính không ổn định của đường truyền được đưa vào thiết kế dữ liệu ngay từ đầu.

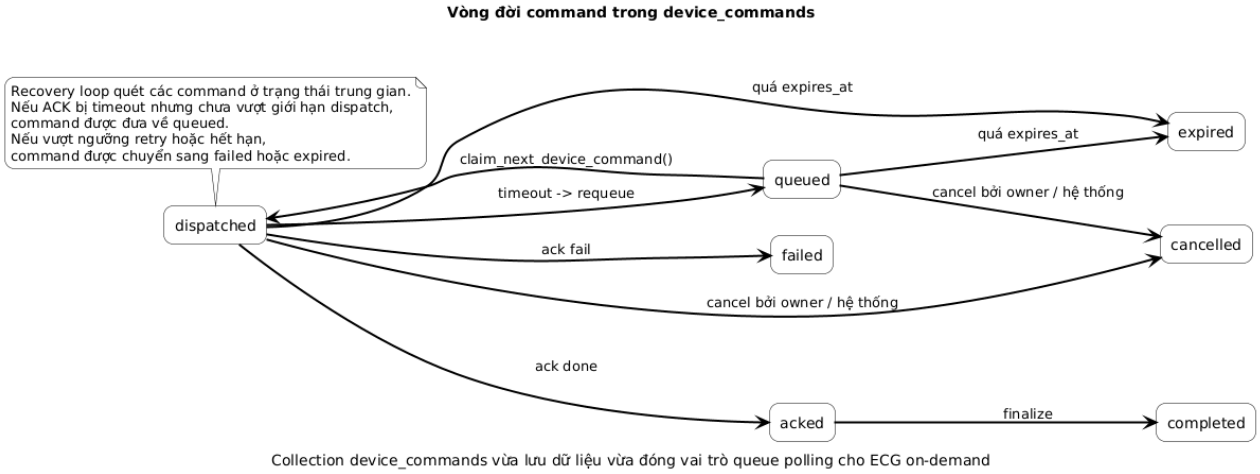
Song song với health_readings, collection alerts đảm nhiệm việc lưu lại các bất thường phát sinh sau bước ingest. Một alert không chỉ ghi lại giá trị vượt ngưỡng, mà còn có severity, trạng thái acknowledged, người xác nhận, thời điểm xác nhận và các thông tin bổ sung cần thiết cho vòng đời cảnh báo. Backend còn áp dụng cả hard dedupe bằng index và soft dedupe theo cửa sổ thời gian để tránh tình trạng một chỉ số bất thường tạo ra quá nhiều cảnh báo trùng lặp trong thời gian rất ngắn.

Collection device_commands là thành phần đưa mô hình cơ sở dữ liệu tiến thêm một bước: dữ liệu ở đây không chỉ được lưu để truy vấn, mà còn trực tiếp tham gia vào điều phối nghiệp vụ ECG on-demand. Mỗi document command mang trạng thái

queued, dispatched, acked, completed, failed, expired hoặc cancelled, cùng với dispatch_count, next_retry_at, expires_at, failure_reason và ack_message. Nhờ đó MongoDB đồng thời đóng vai trò persistence layer và queue state store cho một luồng polling giữa backend với ESP32.



Hình 5.3. Quan hệ logic giữa các collection lõi và nhóm thực thể.



Hình 5.4. Vòng đời command trong collection device_commands.

5.4 Nhóm thực thể, liên kết và dữ liệu bảo mật

Ở lớp thực thể, users và devices là hai collection nền tảng mô tả các đối tượng sống lâu của hệ thống. Users lưu định danh, thông tin hồ sơ, số điện thoại, email và dữ liệu xác thực liên quan; còn devices lưu device_id, device_type, trạng thái, esp_token_hash, metadata, registered_at và last_seen. last_seen là một trường đặc biệt quan trọng vì nó cho phép hệ thống suy ra thiết bị còn đang tương tác với backend hay đã im lặng trong một khoảng thời gian dài.

Tuy nhiên, mô hình dữ liệu hiện tại không còn dừng ở quan hệ đơn giản một user gắn với một device. Collection device_links được đưa vào như lớp liên kết mềm trung tâm, cho phép một user có nhiều thiết bị và một thiết bị được chia sẻ cho nhiều user theo quyền owner hoặc viewer. Chính device_links là thành phần giúp backend thực thi mô hình device-centric, nơi quyền dữ liệu hằng ngày không còn dựa chủ yếu vào role tĩnh trên user mà dựa vào quan hệ động giữa user với device cụ thể.

Ở lớp bảo mật, auth_sessions lưu session đăng nhập cùng refresh token hash, created_at, last_refreshed_at, expires_at và revoked_at. Thiết kế này cho phép backend kiểm soát vòng đời phiên ở mức database, hỗ trợ refresh token rotation và revoke session khi logout hoặc khi cần vô hiệu hóa một phiên. Song song với đó, audit_logs đóng vai trò nền tảng cho truy vết và quản trị, dù hiện tại đây chưa phải collection được khai thác mạnh ở phía giao diện. Việc duy trì audit_logs ngay từ giai

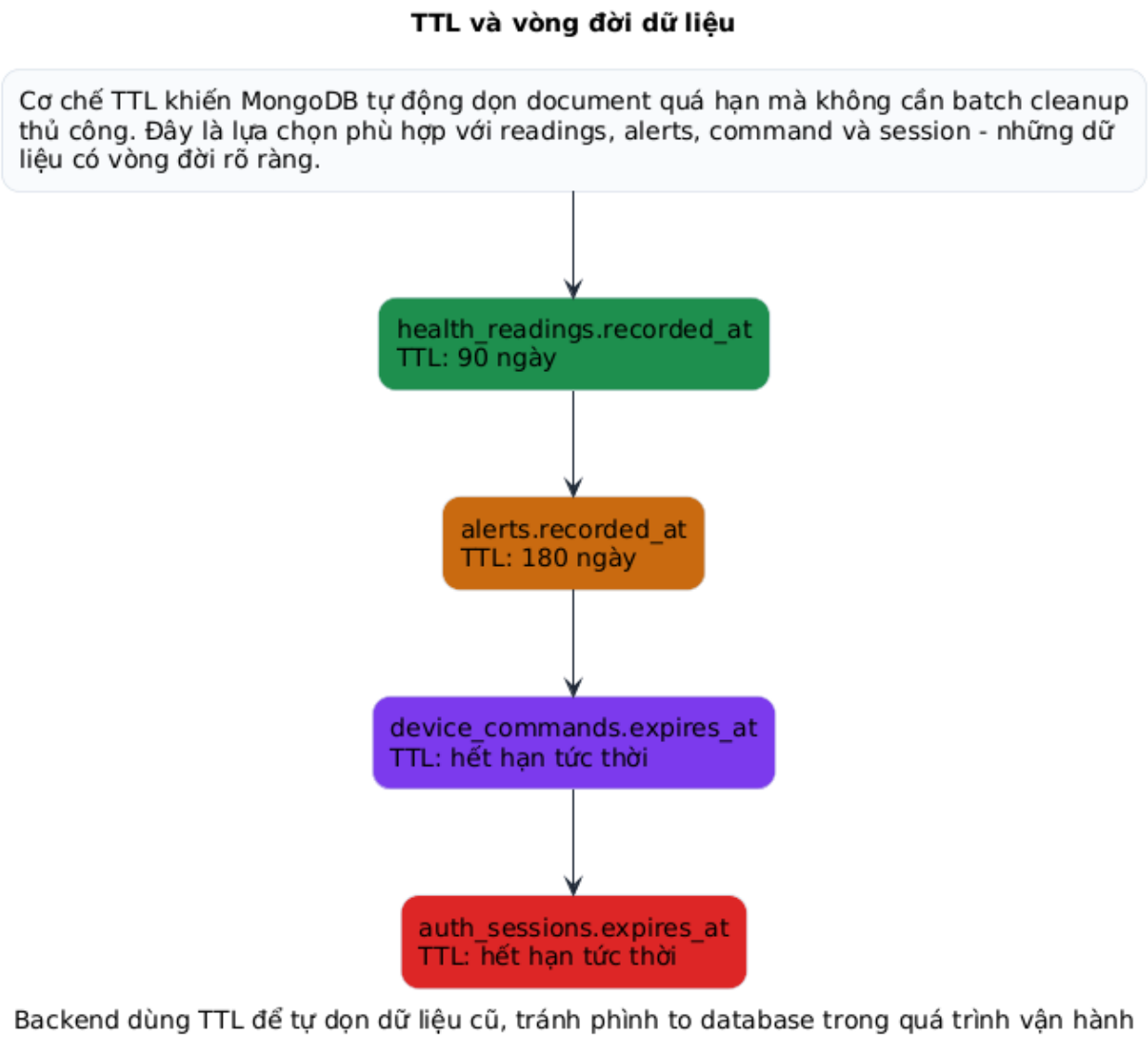
đoạn đầu là một quyết định tốt vì nó giúp hệ thống sẵn sàng cho việc mở rộng lớp giám sát và vận hành sau này.

5.5 Chỉ mục, TTL, tối ưu truy vấn và an toàn dữ liệu

Thiết kế MongoDB của hệ thống không tách rời khỏi nhu cầu truy vấn thực tế. Các hàm như get_latest_reading, get_latest_user_reading, get_readings_by_device, get_health_readings, get_alerts_by_device, claim_next_device_command hay get_auth_session_by_refresh_token_hash cho thấy lớp dữ liệu được xây dựng xoay quanh những luồng backend thật sự cần dùng. Nói cách khác, schema, index và hàm truy vấn được thiết kế đồng thời, thay vì tạo schema trước rồi tối ưu về sau.

TTL là một trong những quyết định quan trọng nhất của chương này. Health_readings được giữ với vòng đời khoảng 90 ngày, alerts khoảng 180 ngày, còn device_commands và auth_sessions tự dọn khi chạm expires_at. Cách làm này rất phù hợp với bài toán nguyên mẫu vì nó giữ database gọn, giảm nhu cầu dọn thủ công và khiến vòng đời dữ liệu phản ánh đúng bản chất nghiệp vụ: readings và alerts hữu ích nhất trong một khoảng thời gian hữu hạn, còn command và session chỉ nên tồn tại khi còn giá trị vận hành.

Ngoài hiệu năng, lớp dữ liệu còn chú trọng bảo vệ thông tin nhạy cảm khi serialize document trả về phía API. Hàm serialize ở db.py không chỉ chuyển ObjectId và datetime sang dạng phù hợp cho JSON, mà còn chủ động loại bỏ esp_token_hash, pairing_code_hash và password_hash. Điều này rất quan trọng vì nó cho thấy thiết kế cơ sở dữ liệu không chỉ tính đến việc lưu cái gì, mà còn tính đến việc tuyệt đối không đưa cái gì ra ngoài response.



Hình 5.5. TTL và vòng đời dữ liệu chính trong MongoDB.

Bảng 5.2. Tóm tắt chiến lược chỉ mục và retention của các collection chính.

Collection	Chỉ mục chính	Mục tiêu tối ưu	TTL / Dedupe
health_readings	(device_id, timestamp), (device_uid, timestamp), (device_id, seq) unique partial	latest, history, ECG theo thiết bị; chống resend trùng	TTL recorded_at ~ 90 ngày
alerts	(severity, acknowledged), (device_id, alert_type, timestamp), (device_id, seq, alert_type) unique partial	lọc severity, trạng thái xử lý, lịch sử cảnh báo theo thiết bị	TTL recorded_at ~ 180 ngày; soft dedupe cửa sổ thời gian

devices	device_id unique, status, esp_token_hash unique sparse	tra cứu device, trạng thái hoạt động, token hash	không TTL, lưu lâu dài
device_commands	(device_id, status, created_at), (device_id, dedupe_key, created_at), request_id unique sparse	claim command, chống enqueue trùng logic, điều phối retry	TTL expires_at = 0
auth_sessions	session_id unique, (user_id, created_at), refresh_token_hash unique sparse	quản lý refresh token rotation và revoke session	TTL expires_at = 0
device_links	(device_id, user_id) unique, (user_id, is_active, created_at), (device_id, permission, is_active)	list thiết bị của user, list user của thiết bị, kiểm tra quyền	không TTL, lưu lâu dài

5.6 Đánh giá thiết kế cơ sở dữ liệu

Nhìn tổng thể, thiết kế cơ sở dữ liệu của hệ thống là một thiết kế thực dụng và hợp lý. Điểm mạnh lớn nhất nằm ở chỗ mô hình document bám rất sát bản chất của bài toán sức khỏe từ xa, đồng thời hỗ trợ tốt cả chiều ingest từ thiết bị và chiều truy vấn từ ứng dụng. Health_readings, alerts và device_commands thể hiện rõ một lớp dữ liệu vừa phục vụ lưu trữ vừa phục vụ điều phối nghiệp vụ. Device_links và auth_sessions làm cho hệ thống linh hoạt hơn về phân quyền và an toàn hơn về xác thực. TTL, dedupe và serialize an toàn cho thấy lớp dữ liệu được chăm chút cả ở khía cạnh hiệu năng lẫn bảo mật.

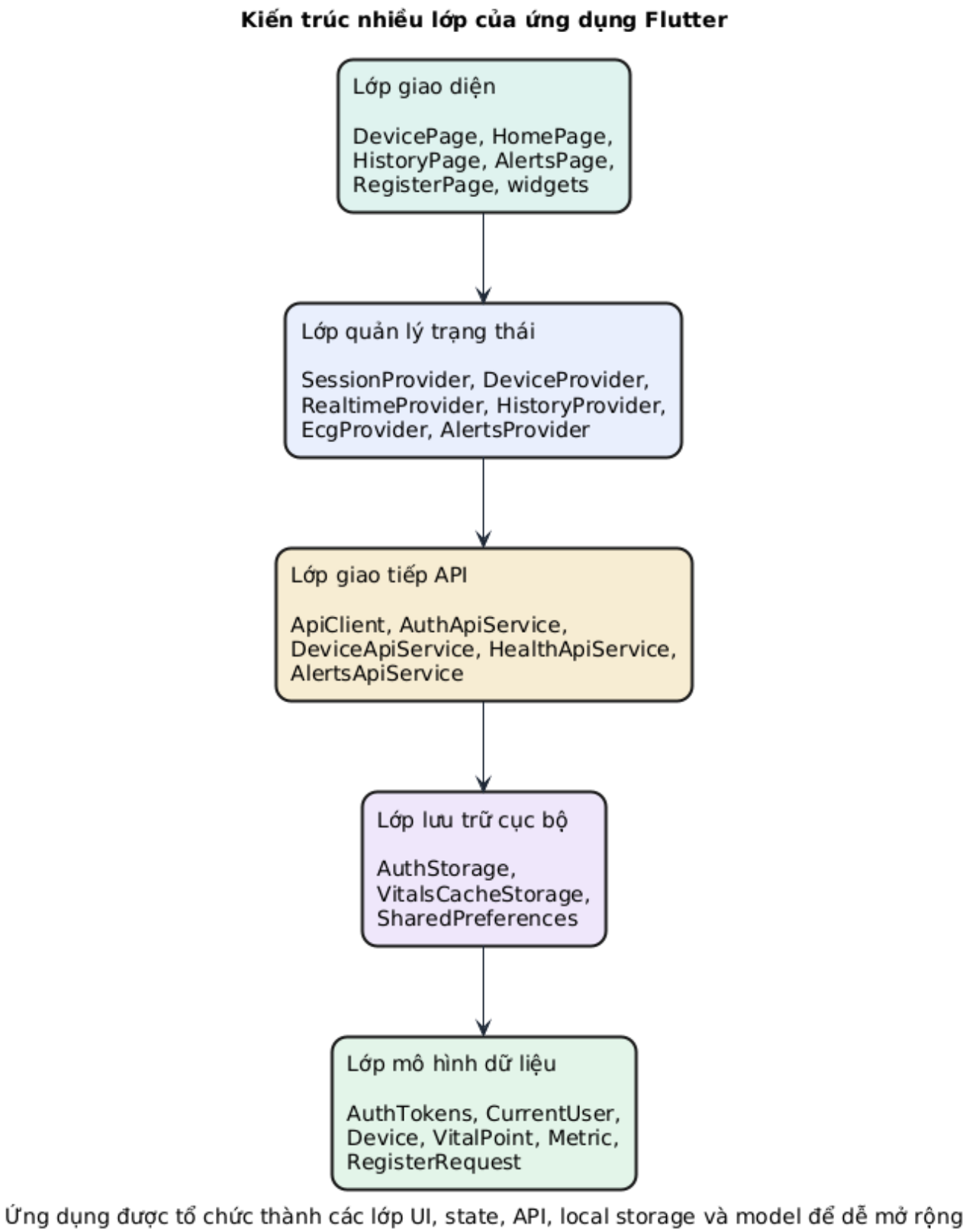
Hạn chế còn lại chủ yếu xuất hiện ở tính nhất quán và khả năng mở rộng. Vì MongoDB không dùng foreign key cứng, phần lớn tính nhất quán giữa users, devices, device_links, readings và alerts phụ thuộc vào tầng service và tầng DB logic. Bên cạnh đó, việc dùng MongoDB làm queue cho device_commands là rất phù hợp ở quy mô nguyên mẫu hoặc small-to-medium scale, nhưng khi số thiết bị và tần suất command tăng lớn, backend sẽ cần theo dõi kỹ hơn chỉ số queue latency, retry rate và dung lượng collection. Dù vậy, trong phạm vi đồ án tốt nghiệp, đây là một thiết kế cơ sở dữ liệu tốt, đủ rõ, đủ hiện đại và bám sát luồng hoạt động thực tế của hệ thống.

CHƯƠNG 6
THIẾT KẾ ỨNG DỤNG WEB

6.1 Vai trò của ứng dụng Web và mục tiêu thiết kế

Trong hệ thống Eldercare, ứng dụng web là lớp giao tiếp trực tiếp với người dùng cuối. Nếu backend chịu trách nhiệm xử lý nghiệp vụ và thiết bị ESP32 tạo ra dữ liệu, thì ứng dụng web là nơi người dùng đăng nhập, chọn thiết bị, xem chỉ số sức khỏe, tra cứu lịch sử, theo dõi cảnh báo và chủ động gửi yêu cầu ECG. Vai trò của app vì thế không chỉ là hiển thị dữ liệu, mà còn là tổ chức trải nghiệm người dùng trên nền một hệ thống có xác thực, có phân quyền theo thiết bị và có các trạng thái dữ liệu thay đổi liên tục.

Mục tiêu thiết kế của ứng dụng tập trung vào ba lớp yêu cầu. Lớp thứ nhất là yêu cầu sử dụng: giao diện phải đơn giản, dễ hiểu và đủ trực quan để người dùng không cần biết backend hay thiết bị vận hành ra sao vẫn có thể thao tác trọn vẹn. Lớp thứ hai là yêu cầu kỹ thuật: app phải bám chặt contract API hiện tại, đặc biệt ở luồng auth, me/devices, latest, history, alerts và ECG on-demand. Lớp thứ ba là yêu cầu ổn định: ứng dụng phải xử lý tốt nhiều trạng thái như chưa đăng nhập, phiên hết hạn, chưa có thiết bị, chưa có dữ liệu, lỗi quyền truy cập, dữ liệu cache và thay đổi device scope.



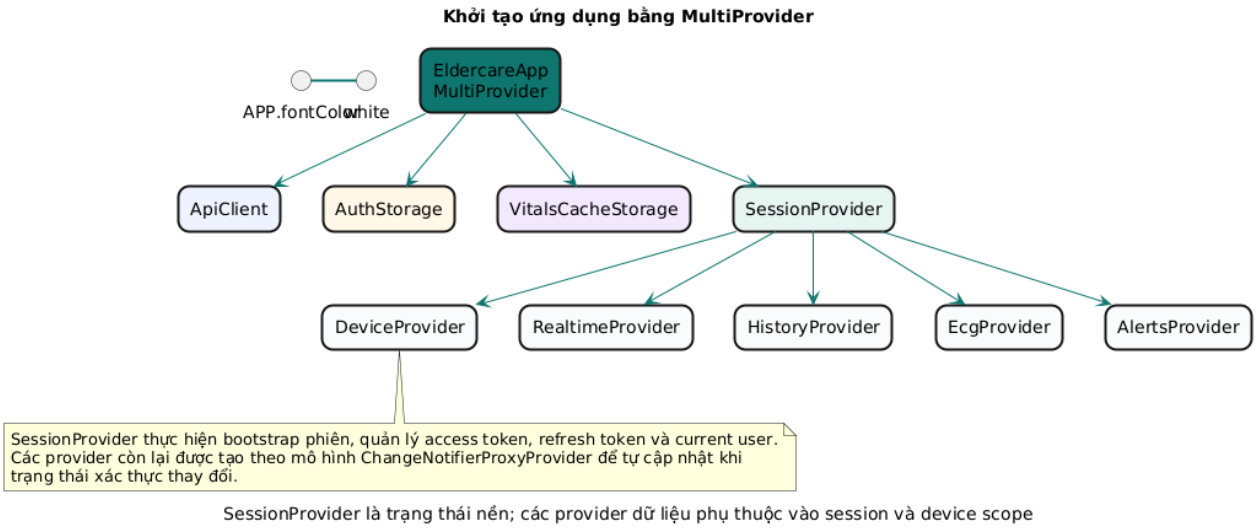
Hình 6.1. Kiến trúc nhiều lớp của ứng dụng web.

6.2 Kiến trúc tổng thể và cơ chế khởi tạo hệ thống frontend

Kiến trúc tổng thể của ứng dụng Web được xây dựng theo hướng phân tách nhiều lớp, trong đó giao diện người dùng, cơ chế quản lý trạng thái, tầng giao tiếp API, lưu trữ cục bộ và mô hình dữ liệu được tách biệt tương đối rõ ràng. Cách tổ chức này giúp hệ thống tránh việc dồn toàn bộ logic vào lớp giao diện, đồng thời cho phép tái sử dụng trạng thái và logic gọi API giữa nhiều trang khác nhau. Trên thực tế, đây là kiểu kiến trúc phù hợp với một ứng dụng có quản lý phiên, nhiều nguồn dữ liệu và nhiều trạng thái sử dụng như hệ thống hiện tại.

Ở bước khởi tạo, ứng dụng thực hiện việc cấu hình và liên kết các thành phần phụ thuộc của frontend. Một lớp giao tiếp HTTP chung được thiết lập để gửi request đến backend; các cơ chế lưu trữ cục bộ được sử dụng để duy trì dữ liệu phiên và cache; cơ chế quản lý session được khởi tạo trước và thực hiện bootstrap phiên ngay khi ứng dụng chạy. Các module quản lý dữ liệu như thiết bị, dữ liệu thời gian thực, lịch sử, ECG và cảnh báo được khởi tạo dựa trên trạng thái session hiện tại và có khả năng tự động cập nhật khi trạng thái phiên thay đổi.

Cách khởi tạo này biến trạng thái session trở thành nền tảng, trong khi các trạng thái còn lại phụ thuộc vào session và thiết bị đang được chọn.



Hình 6.2. Sơ đồ quan hệ phụ thuộc khi khởi tạo ứng dụng Web.

Bảng 6.1. Vai trò của các module quản lý trạng thái trong ứng dụng Web.

Thành phần	Trách nhiệm cốt lõi	Dữ liệu trạng thái nổi bật	Khi nào thay đổi
Quản lý session	Quản lý phiên đăng nhập, token, thông tin người dùng và trạng thái xác thực	access token, refresh token, người dùng hiện tại, lỗi	Đăng nhập, refresh, đăng xuất, khởi tạo lại phiên

Quản lý thiết bị	Đồng bộ danh sách thiết bị, lưu thiết bị hiện tại	danh sách thiết bị, thiết bị hiện tại	Sau đăng nhập, khi cập nhật danh sách, khi đổi thiết bị
Dữ liệu thời gian thực	Lấy dữ liệu mới nhất và trạng thái hoạt động	dữ liệu latest, trạng thái online/cache	Khi đổi thiết bị hoặc làm mới dữ liệu
Dữ liệu lịch sử	Quản lý dữ liệu theo thời gian	ngày được chọn, dữ liệu lịch sử	Khi đổi ngày hoặc thiết bị
Dữ liệu ECG	Quản lý yêu cầu ECG và kết quả	trạng thái, kết quả, lỗi	Khi gửi yêu cầu hoặc đổi thiết bị
Cảnh báo	Tải và lọc cảnh báo theo thiết bị	danh sách cảnh báo, trạng thái lọc	Khi truy cập hoặc làm mới

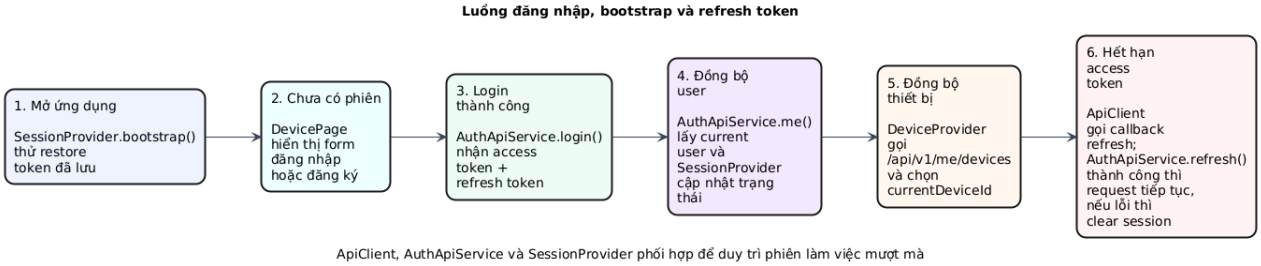
6.3 Tầng giao tiếp API và quản lý phiên người dùng

Lớp giao tiếp API đóng vai trò trung tâm trong việc gửi request và nhận phản hồi từ backend. Thành phần này chuẩn hóa cách thức giao tiếp mạng, tự động gắn access token vào header xác thực, xử lý cơ chế làm mới token và chuyển đổi lỗi HTTP thành các dạng lỗi có ý nghĩa đối với tầng quản lý trạng thái.

Nhờ đó, các module dịch vụ như xác thực, thiết bị, dữ liệu sức khỏe và cảnh báo không cần xử lý lại logic mạng, mà chỉ tập trung vào các endpoint và dữ liệu nghiệp vụ tương ứng.

Trong luồng xác thực, module quản lý session đóng vai trò điều phối chính ở phía frontend. Khi ứng dụng khởi động, hệ thống sẽ cố gắng khôi phục phiên từ dữ liệu lưu trữ cục bộ. Nếu thành công, ứng dụng tiếp tục lấy thông tin người dùng và đồng bộ danh sách thiết bị. Trong trường hợp access token hết hạn, hệ thống sẽ tự động thực hiện refresh token; nếu thất bại, phiên sẽ bị xóa và người dùng được đưa về trạng thái chưa đăng nhập.

Cách tổ chức này giúp logic phiên làm việc được tập trung, nhất quán và tách biệt khỏi giao diện. Các trang như trang chính hoặc trang thiết bị chỉ cần kiểm tra trạng thái xác thực, trong khi toàn bộ cơ chế xử lý token đã được xử lý ở tầng nền.



Hình 6.3. Luồng bootstrap, đăng nhập và refresh token của ứng dụng.

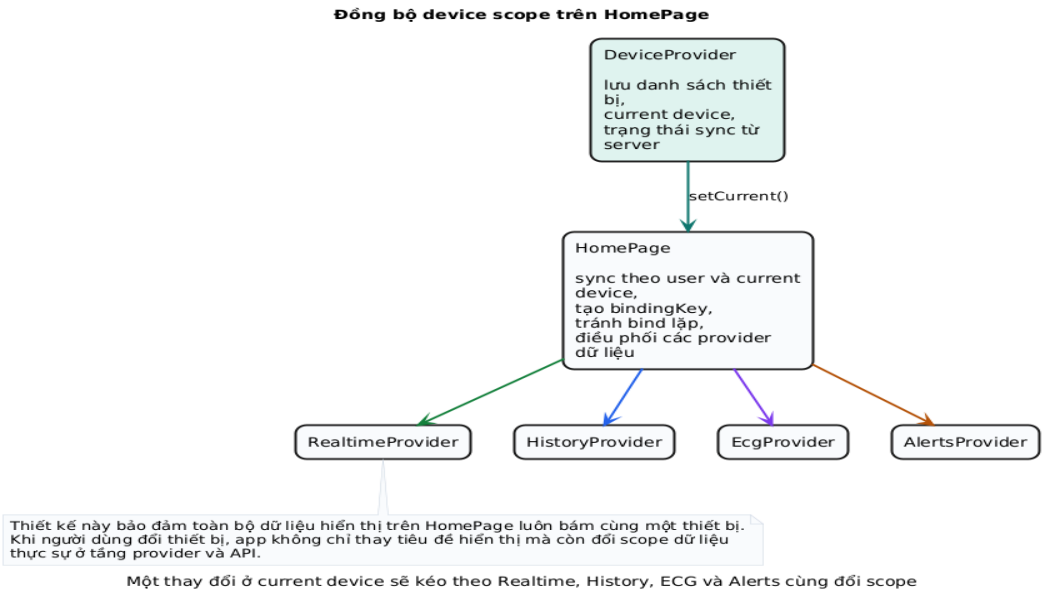
6.4 Quản lý device scope và dữ liệu sức khỏe theo thời gian thực

Một đặc điểm quan trọng của ứng dụng Web là không bị ràng buộc với một thiết bị cố định. Sau khi đăng nhập, hệ thống tải danh sách thiết bị liên kết, cho phép người dùng lựa chọn thiết bị đang theo dõi và lưu lại lựa chọn này trong bộ nhớ cục bộ.

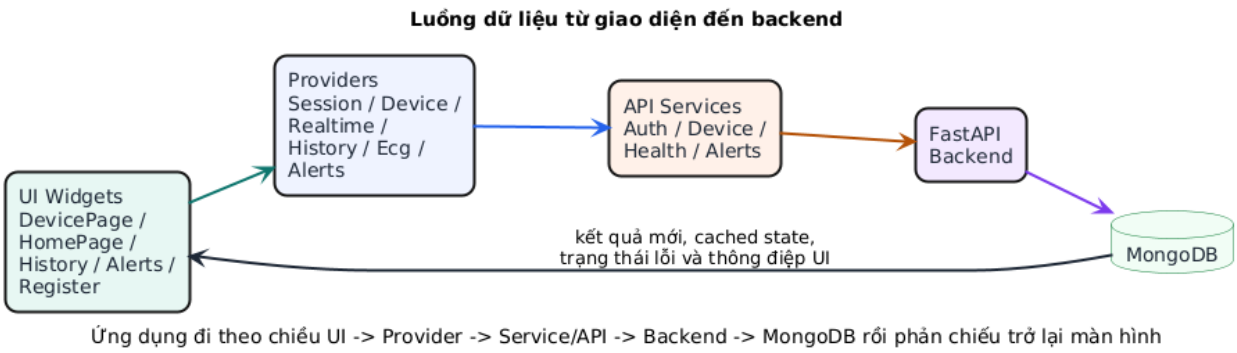
Từ đó, toàn bộ dữ liệu không còn gắn với người dùng một cách trừu tượng mà gắn với cặp người dùng – thiết bị cụ thể. Đây là biểu hiện của mô hình device-centric ở phía frontend.

Khi người dùng truy cập hoặc thay đổi thiết bị, hệ thống sẽ đồng bộ lại toàn bộ phạm vi dữ liệu. Các dữ liệu như thông tin mới nhất, lịch sử, cảnh báo và ECG đều được truy vấn dựa trên cùng một thiết bị, bảo đảm tính nhất quán và tránh hiển thị sai lệch giữa nhiều nguồn dữ liệu.

Về xử lý dữ liệu, hệ thống phân tách rõ các nhóm dữ liệu: dữ liệu thời gian thực, dữ liệu lịch sử, dữ liệu ECG và cảnh báo. Mỗi nhóm có vòng đời và trạng thái riêng, nhưng vẫn liên kết với nhau thông qua thiết bị hiện tại.



Hình 6.4. Cơ chế đồng bộ phạm vi thiết bị trong ứng dụng Web.

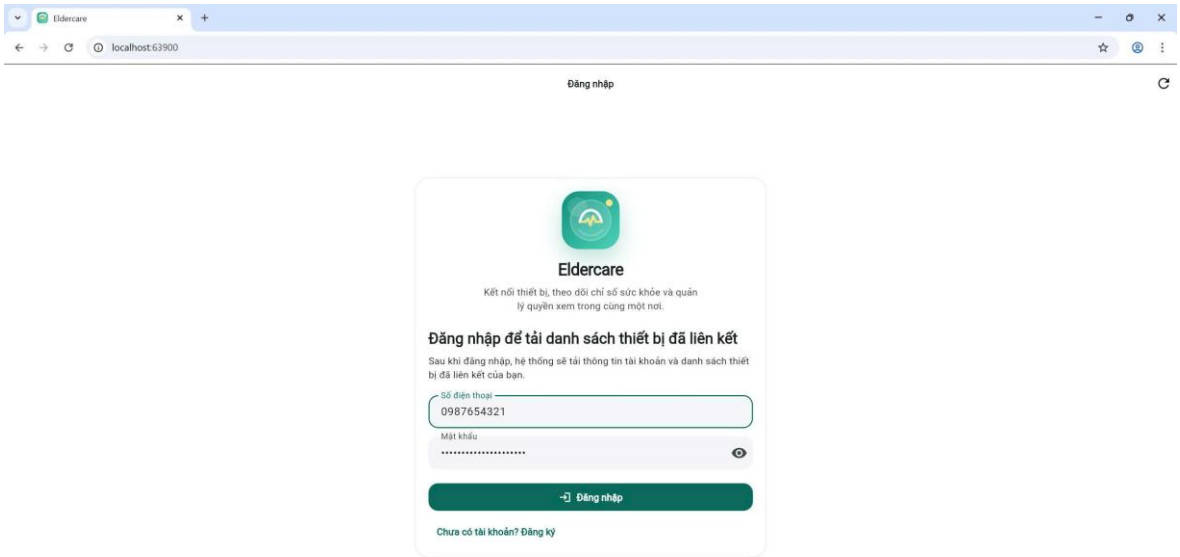


Hình 6.5. Luồng dữ liệu từ giao diện web đến backend và ngược lại.

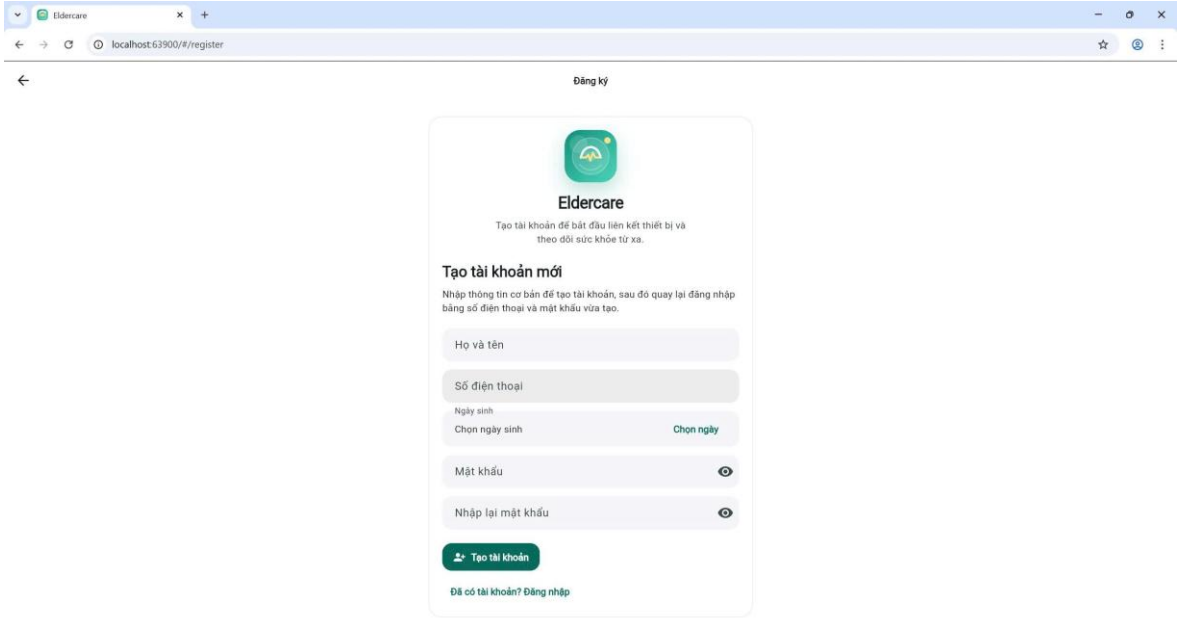
6.5 Tổ chức giao diện và luồng sử dụng chính

Về mặt giao diện, ứng dụng được tổ chức quanh một số màn hình chính có trách nhiệm rõ ràng. DevicePage là cửa vào của hệ thống, đồng thời đóng hai vai trò: khi chưa xác thực thì đây là màn hình đăng nhập, còn khi đã xác thực thì đây là màn hình quản lý thiết bị và điều hướng tiếp vào luồng theo dõi. HomePage là màn hình trung tâm để người dùng xem dữ liệu sức khỏe mới nhất, đổi thiết bị, mở lịch sử, xem cảnh báo và gửi yêu cầu ECG. RegisterPage tách riêng luồng tạo tài khoản mới, còn HistoryPage, AlertsPage và DeviceViewersPage là các màn hình chức năng mở rộng.

Một điểm mạnh của UI hiện tại là các trạng thái rỗng và lỗi được quan tâm khá kỹ. Nếu chưa đăng nhập, giao diện sẽ hiển thị đúng trạng thái chưa có session. Nếu đã đăng nhập nhưng chưa có thiết bị, DevicePage hiển thị empty state và hướng dẫn liên kết thiết bị. Nếu backend chưa có reading hoặc người dùng không có quyền xem thiết bị, HomePage và các màn hình dữ liệu sẽ hiển thị thông điệp phù hợp thay vì rơi vào khoảng trống vô nghĩa. Điều này giúp trải nghiệm sử dụng thực tế tốt hơn và cũng làm cho ứng dụng phù hợp hơn với môi trường thật, nơi dữ liệu và trạng thái không phải lúc nào cũng đầy đủ.

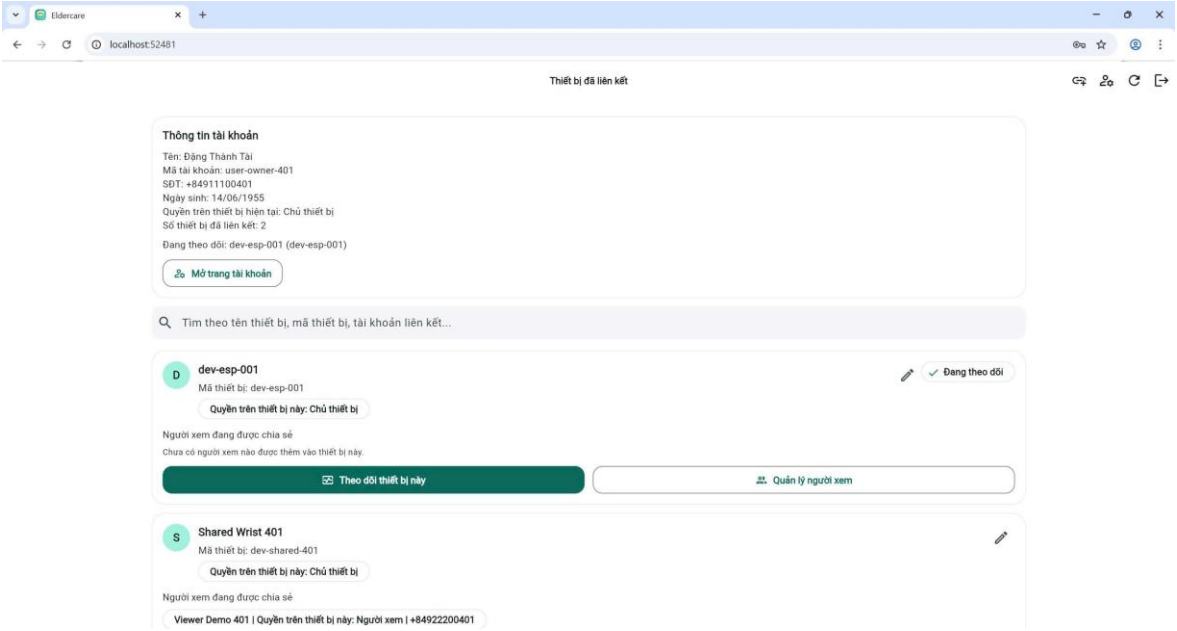


Hình 6.6. Cụm màn hình đầu vào của web: đăng nhập.

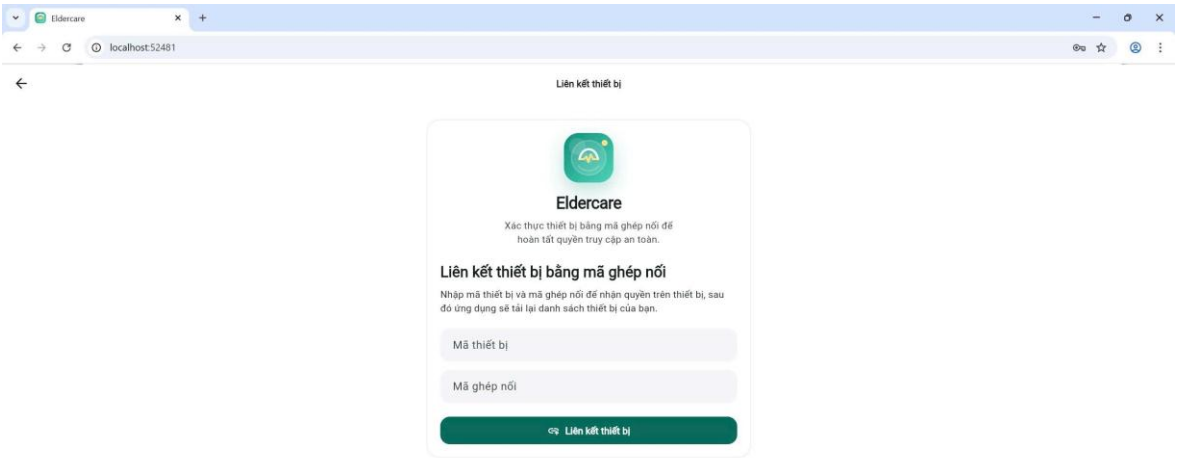


Hình 6.7. Màn hình giao diện đăng ký tài khoản.

Mô tả: Cụm màn hình đầu vào của ứng dụng được thiết kế theo phong cách tối giản, trực quan và đồng bộ với nhận diện thương hiệu Eldercare thông qua tông màu xanh ngọc chủ đạo. Giao diện gồm ba màn hình chính: đăng nhập, thông báo sai thông tin và đăng ký tài khoản. Ở màn hình đăng nhập, người dùng được yêu cầu nhập số điện thoại đã liên kết với danh sách thiết bị và mật khẩu để truy cập hệ thống; các trường nhập liệu được bố trí rõ ràng, nút chức năng nổi bật giúp thao tác nhanh chóng. Khi người dùng nhập sai thông tin xác thực, hệ thống hiển thị thông báo lỗi ngay bên dưới ô nhập, đồng thời làm nổi bật trường dữ liệu chưa hợp lệ để hỗ trợ nhận biết và chỉnh sửa kịp thời. Đối với màn hình đăng ký tài khoản, ứng dụng cung cấp biểu mẫu gồm các thông tin cơ bản như họ và tên, số điện thoại, ngày sinh, mật khẩu và xác nhận mật khẩu, giúp người dùng tạo tài khoản mới một cách đầy đủ và thuận tiện. Nhìn chung, cụm giao diện này thể hiện quy trình tiếp cận ban đầu của người dùng với hệ thống, đảm bảo tính dễ sử dụng, nhất quán và hỗ trợ tốt cho trải nghiệm đăng nhập, xử lý lỗi và đăng ký tài khoản.



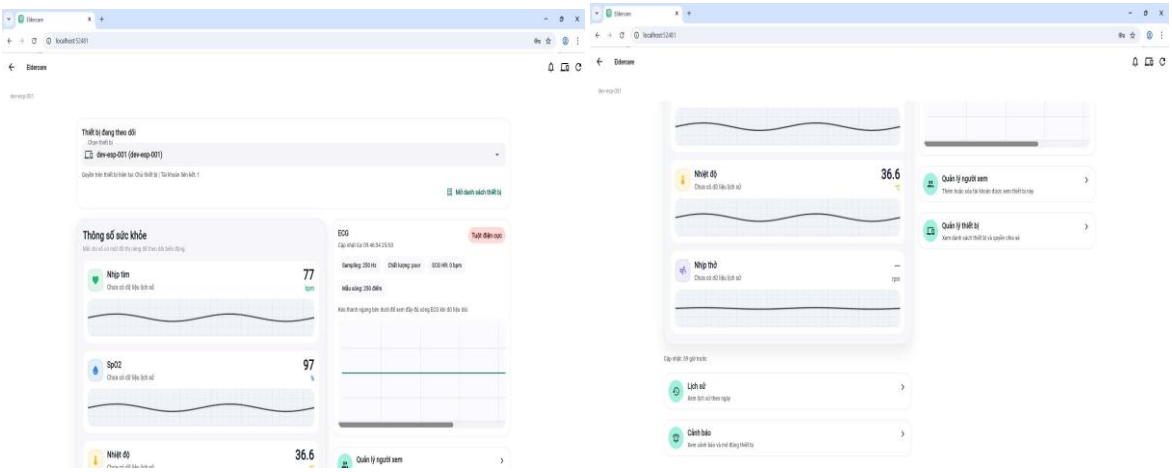
Hình 6.8. Màn hình giao diện quản lý thiết bị.



Hình 6.9. Màn hình giao diện liên kết thiết bị

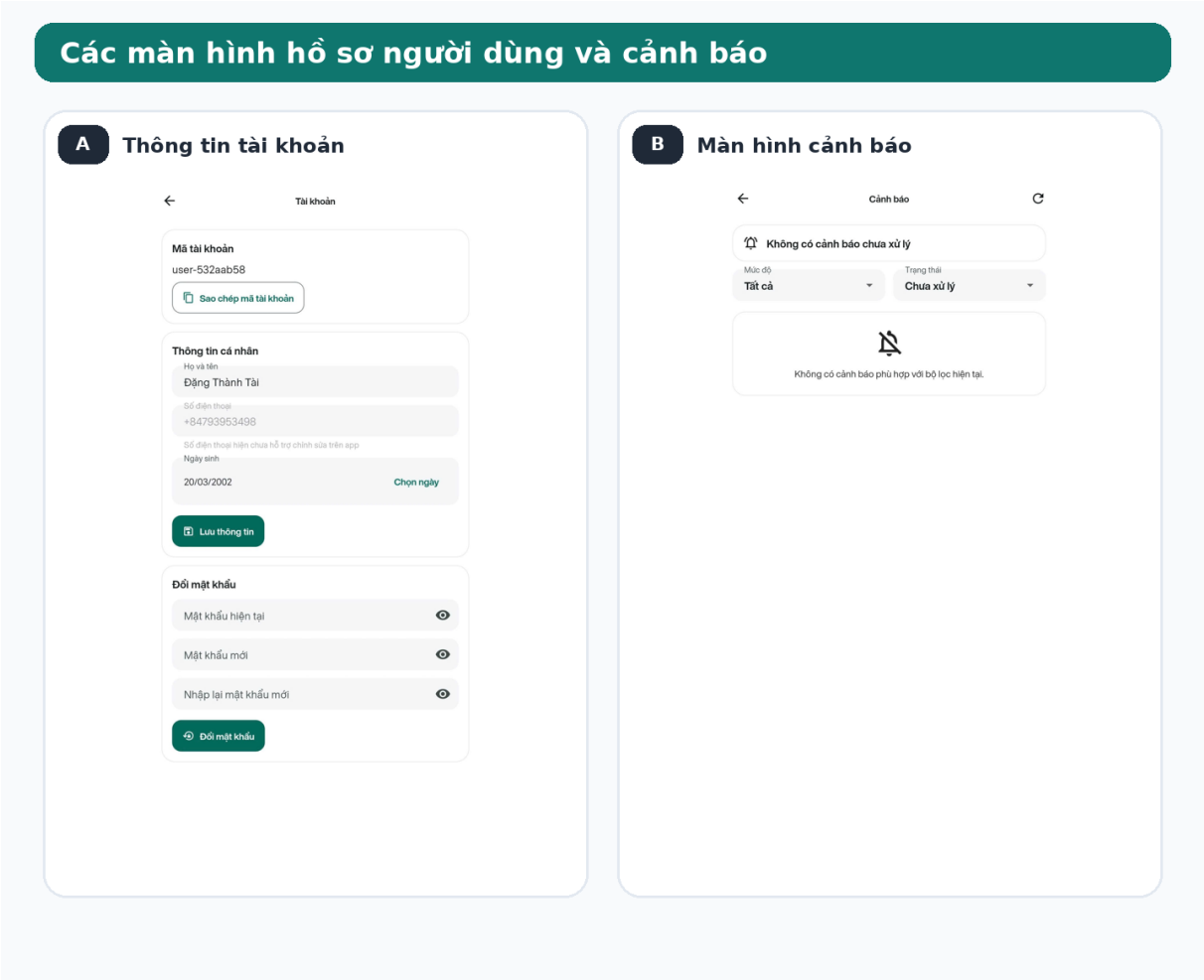
Hình 6.10. Màn hình giao diện phân quyền người dùng

Mô tả: Cụm màn hình quản lý thiết bị và phân quyền của ứng dụng được thiết kế nhằm hỗ trợ người dùng theo dõi trạng thái thiết bị, thực hiện liên kết và quản lý quyền truy cập một cách thuận tiện. Ở màn hình "Chưa có thiết bị", hệ thống hiển thị trạng thái chưa phát hiện thiết bị nào được kết nối, đồng thời cung cấp nút thao tác để người dùng bắt đầu quá trình liên kết thiết bị mới. Tại màn hình "Liên kết thiết bị", người dùng được yêu cầu nhập mã ghép nối để kết nối thiết bị với tài khoản, giao diện được bố trí đơn giản, tập trung vào thao tác chính nhằm giảm nhầm lẫn khi sử dụng. Khi quá trình ghép nối gặp sự cố, màn hình "Lỗi ghép nối" sẽ hiển thị thông báo lỗi ngay bên dưới ô nhập, giúp người dùng dễ dàng nhận biết nguyên nhân và thử lại. Bên cạnh đó, màn hình "Quản lý người xem" cho phép chủ tài khoản thêm và kiểm soát danh sách những người được chia sẻ quyền theo dõi thiết bị, qua đó hỗ trợ việc phân quyền truy cập rõ ràng và phù hợp với nhu cầu sử dụng thực tế. Tổng thể, cụm giao diện này thể hiện quy trình quản lý thiết bị từ khi chưa kết nối, thực hiện ghép nối, xử lý lỗi đến chia sẻ quyền truy cập, góp phần nâng cao tính trực quan và hiệu quả trong quá trình vận hành hệ thống.



Hình 6.11. Cụm màn hình theo dõi thiết bị, dashboard sức khỏe và lịch sử theo ngày.

Mô tả: Cụm màn hình theo dõi và điều hướng dữ liệu sức khỏe được thiết kế nhằm hỗ trợ người dùng quan sát nhanh tình trạng thiết bị và các chỉ số sức khỏe quan trọng của đối tượng theo dõi. Ở màn hình danh sách thiết bị, hệ thống hiển thị thông tin cơ bản của thiết bị đã liên kết, trạng thái chia sẻ và các nút chức năng để truy cập nhanh đến phần theo dõi hoặc quản lý quyền xem. Màn hình trang chủ đóng vai trò như một dashboard tổng quan, trình bày các chỉ số sức khỏe chính dưới dạng thẻ thông tin kết hợp đồ thị xu hướng, giúp người dùng dễ dàng nắm bắt sự thay đổi của dữ liệu theo thời gian. Bên cạnh đó, màn hình hành động nhanh cung cấp các lối tắt đến những chức năng thường dùng như xem thông tin liên quan, xử lý cảnh báo, theo dõi thiết bị hoặc quản lý người xem, từ đó rút ngắn thao tác và tăng tính thuận tiện khi sử dụng. Cuối cùng, màn hình lịch sử theo ngày cho phép người dùng lựa chọn từng loại chỉ số và xem biểu đồ biến thiên trong ngày, hỗ trợ việc theo dõi chi tiết, đối chiếu dữ liệu và phát hiện các bất thường sức khỏe một cách trực quan. Nhìn chung, cụm giao diện này thể hiện rõ vai trò của ứng dụng trong việc tập trung hóa dữ liệu, hỗ trợ điều hướng nhanh và nâng cao hiệu quả giám sát sức khỏe từ xa.



Hình 6.12. Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng.

Mô tả: Cụm màn hình hồ sơ người dùng và cảnh báo trong ứng dụng được xây dựng nhằm hỗ trợ người dùng quản lý thông tin cá nhân và theo dõi các thông báo bất thường một cách thuận tiện. Ở màn hình thông tin tài khoản, hệ thống hiển thị mã tài khoản cùng các thông tin cá nhân cơ bản như họ và tên, số điện thoại, ngày sinh, đồng thời cho phép người dùng cập nhật dữ liệu và thực hiện thay đổi mật khẩu ngay trong cùng một giao diện. Cách bố trí các trường thông tin rõ ràng, dễ thao tác, giúp nâng cao tính thuận tiện trong quá trình quản lý hồ sơ cá nhân. Bên cạnh đó, màn hình cảnh báo được thiết kế để tổng hợp các thông báo liên quan đến trạng thái sức khỏe hoặc thiết bị, cho phép người dùng lọc theo mức độ và tình trạng xử lý nhằm tra cứu thông tin nhanh chóng hơn. Trong trường hợp chưa có cảnh báo phù hợp, hệ thống hiển thị trạng thái rỗng để thông báo rằng hiện không có sự kiện cần xử lý. Nhìn chung, cụm giao diện này góp phần hoàn thiện chức năng quản lý tài khoản và hỗ trợ người dùng theo dõi cảnh báo một cách trực quan, từ đó nâng cao hiệu quả sử dụng và khả năng giám sát trong hệ thống.

6.6 Cache, xử lý lỗi, kiểm thử và đánh giá ứng dụng

Ứng dụng không chỉ làm việc với dữ liệu online mà còn có một lớp lưu trữ cục bộ tương đối rõ. AuthStorage giữ access token, refresh token và current user để phục vụ restore session; SharedPreferences giữ current device và một phần danh sách thiết bị; còn VitalsCacheStorage lưu latest và history theo từng scope. Nhờ lớp cache này, app vẫn có thể hiển thị dữ liệu gần nhất trong các tình huống mạng kém ổn định, đồng thời nói rõ cho người dùng khi nào dữ liệu đang là dữ liệu tạm lưu chứ không phải dữ liệu vừa lấy mới từ server.

Ở góc độ chất lượng phần mềm, thiết kế kiểm thử hiện tại cũng là một điểm cộng đáng ghi nhận. Ứng dụng đã xây dựng các kịch bản kiểm thử cho nhiều luồng giao diện quan trọng như hiển thị form đăng nhập, kiểm tra dữ liệu đầu vào, trạng thái chưa đăng nhập, trạng thái chưa có thiết bị và kiểm tra form đăng ký. Điều này cho thấy hệ thống không chỉ được phát triển để hoạt động chức năng mà còn được quan tâm đến khả năng kiểm thử.

Bên cạnh đó, việc tách riêng các thành phần giả lập (mock service) cho các chức năng như xác thực, quản lý thiết bị và dữ liệu sức khỏe trong môi trường kiểm thử cho thấy ứng dụng đã được thiết kế theo hướng có thể kiểm thử độc lập với backend thực tế. Đây là dấu hiệu của một kiến trúc frontend có mức độ hoàn thiện cao hơn so với các ứng dụng chỉ dừng lại ở mức giao diện cơ bản.

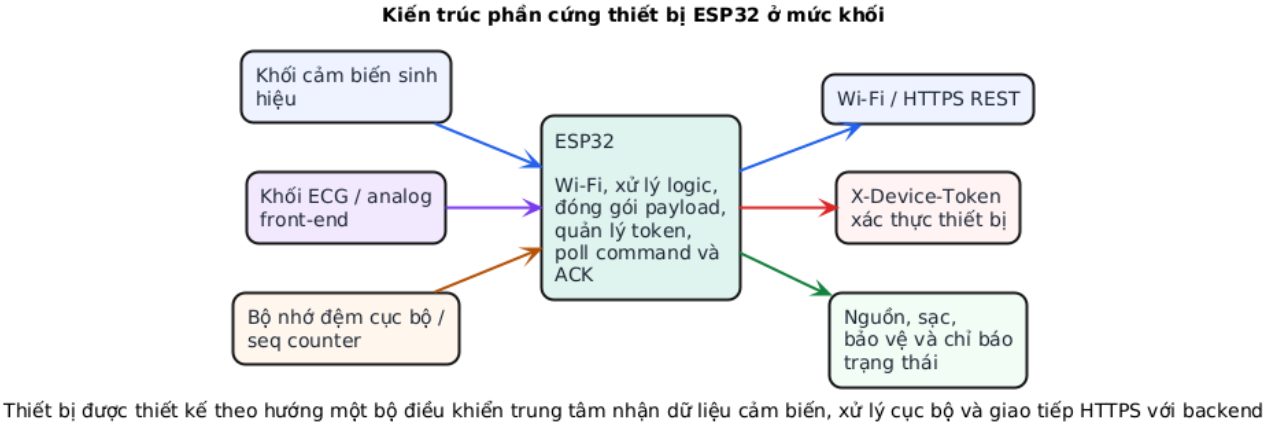
Nhìn tổng thể, thiết kế web của hệ thống là một thiết kế hợp lý và khá hoàn chỉnh trong phạm vi đồ án. Điểm mạnh nổi bật là kiến trúc tách lớp, quản lý session bài bản, hỗ trợ nhiều thiết bị liên kết, bám sát contract backend và xử lý được cả luồng ECG on-demand. Hạn chế chủ yếu nằm ở việc app hiện vẫn thiên về polling và refresh thủ công thay vì có cơ chế push realtime, đồng thời một số logic hỗ trợ debug như fallback device vẫn còn hiện diện. Dù vậy, xét trên mục tiêu của đề tài, đây là một ứng dụng client tốt, đủ rõ về cấu trúc và đủ thực tế về trải nghiệm sử dụng.

CHƯƠNG 7
THIẾT KẾ VÀ MÔ PHỎNG THIẾT BỊ ESP32

7.1 Vai trò của thiết bị và phạm vi thiết kế chương

Trong hệ thống Eldercare, thiết bị ESP32 là nguồn phát sinh dữ liệu sức khỏe và đồng thời là tác nhân thực thi các command do backend điều phối. Nếu ứng dụng web là giao diện cho người dùng và backend là bộ não nghiệp vụ, thì thiết bị là điểm chạm vật lý với môi trường đo. Nó đọc dữ liệu từ các khối cảm biến, chuẩn hóa readings, gửi dữ liệu lên backend, đồng thời tiếp nhận và xử lý các lệnh như đo ECG theo yêu cầu.

Cần lưu ý rằng Git hiện tại tập trung mạnh vào backend và ứng dụng web hơn là một repository firmware hay sơ đồ mạch chi tiết. Vì vậy, chương này được thiết kế lại theo hướng kiến trúc phần cứng mức khối và nguyên lý tích hợp với backend, bám theo contract thực tế của hệ thống như nhóm endpoint /api/v1/esp/*, header X-Device-Token, cơ chế polling command và ACK kết quả. Cách tiếp cận này phù hợp với báo cáo thiết kế vì nó mô tả được phần cứng theo hướng hệ thống, tránh suy đoán quá mức về BOM hoặc pin map khi không có hardware repo chuyên biệt đi kèm.



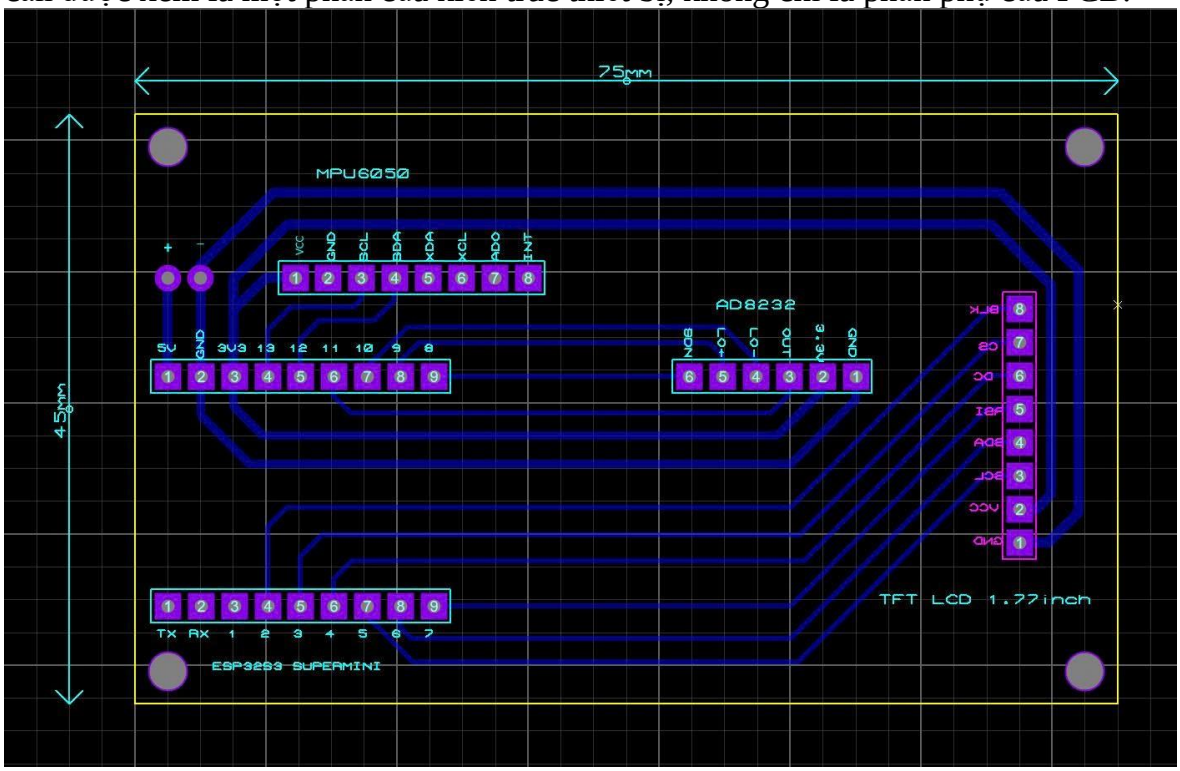
Hình 7.1. Kiến trúc phần cứng thiết bị ESP32 ở mức khối chức năng.

7.2 Kiến trúc phần cứng mức khối của thiết bị

Ở mức khối chức năng, ESP32 đóng vai trò bộ điều khiển trung tâm. Xung quanh nó là các khối cảm biến sinh hiệu, khối thu ECG, lớp lưu đệm cục bộ, lớp truyền thông Wi-Fi/HTTPS, lớp xác thực thiết bị bằng token và khối nguồn. Việc tổ chức theo kiểu một vi điều khiển trung tâm cùng nhiều khối ngoại vi là hướng phù hợp cho thiết bị đeo hoặc thiết bị theo dõi di động, vì nó giữ được chi phí phần cứng ở mức hợp lý mà vẫn đủ năng lực xử lý, mạng và bảo mật.

Khối cảm biến sinh hiệu được hiểu như nơi cung cấp dữ liệu đầu vào cho các chỉ số tim mạch, độ bão hòa oxy, nhiệt độ hoặc các biến sinh lý khác mà hệ thống muốn theo dõi. Khối ECG hoặc analog front-end phụ trách phần tín hiệu điện tim, thường đòi hỏi lấy mẫu có đặc trưng riêng và quy trình xử lý cẩn thận hơn so với các chỉ số rời rạc. Khối lưu đệm cục bộ và seq counter giúp thiết bị không phụ thuộc hoàn toàn vào trạng thái mạng ở từng thời điểm, trong khi khối bảo mật giữ token định danh của thiết bị để mọi request gửi đến backend đều được xác thực đúng cách.

Khối nguồn là thành phần không nên xem nhẹ trong thiết kế thiết bị. Một hệ thống theo dõi sức khỏe từ xa chỉ có ý nghĩa khi thiết bị duy trì được khả năng hoạt động ổn định, không gây tụt áp đột ngột trong quá trình đo, không mất trạng thái quá thường xuyên và có cơ chế chỉ báo trực quan để người dùng nhận biết tình trạng kết nối hoặc lỗi cơ bản. Vì vậy, thiết kế nguồn, sạc, bảo vệ và trạng thái online/offline cần được xem là một phần của kiến trúc thiết bị, không chỉ là phần phụ của PCB.



Hình 7.2. sơ đồ ESP32S3

Bảng 7.1. Các khối phần cứng chính và vai trò trong hệ thống thiết bị.

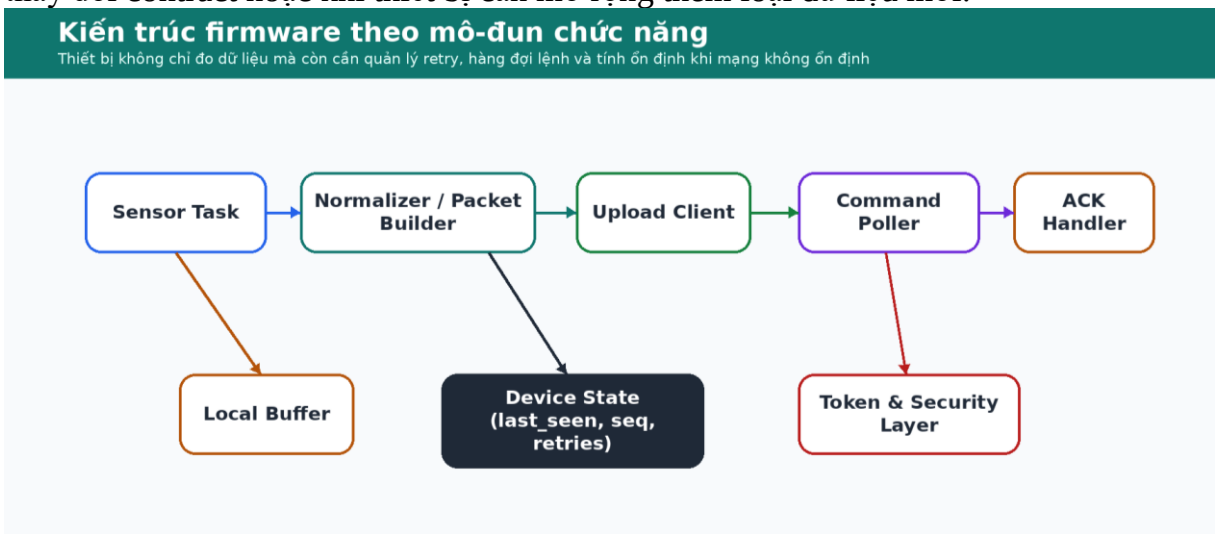
Khối	Vai trò trong thiết bị	Ý nghĩa với hệ thống
ESP32	Bộ điều khiển trung tâm, xử lý logic, kết nối Wi-Fi, giao tiếp với backend	Là lõi của phần cứng, quyết định khả năng tích hợp mạng và điều phối các mô-đun
Khối cảm biến sinh hiệu	Thu thập dữ liệu cho các chỉ số như nhịp tim, SpO2, nhiệt độ hoặc biến sinh lý khác	Tạo dữ liệu đầu vào cho health_readings và các API latest/history
Khối ECG / analog front-end	Thu tín hiệu ECG và chuyển cho vi điều khiển xử lý hoặc đóng gói	Cho phép hiện thực luồng ECG on-demand từ app qua backend đến thiết bị
Local buffer / seq counter	Lưu tạm readings, giữ số thứ tự gói tin và hỗ trợ resend	Tăng độ tin cậy khi mạng không ổn định và hỗ trợ dedupe ở backend

Wi-Fi / HTTPS REST	Gửi readings, poll command và ACK kết quả	Là cầu nối giữa thiết bị với backend FastAPI
Token & security layer	Quản lý X-Device-Token và thông tin định danh thiết bị	Bảo đảm chỉ thiết bị hợp lệ mới được ingest hoặc nhận command
Nguồn và chỉ báo trạng thái	Cấp nguồn ổn định, chỉ báo lỗi hoặc trạng thái hoạt động	Ảnh hưởng trực tiếp đến trải nghiệm sử dụng và độ tin cậy vận hành

7.3 Thiết kế firmware và luồng xử lý readings

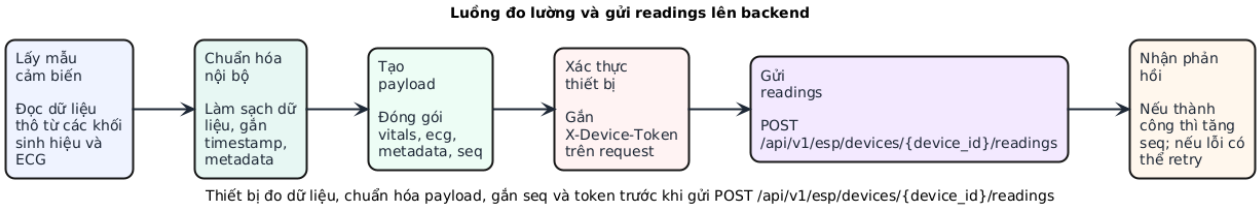
Nếu chỉ nhìn phần cứng mà bỏ qua cách firmware phân tách nhiệm vụ thì thiết kế thiết bị sẽ thiếu một nửa giá trị. Với hệ thống hiện tại, firmware nên được tổ chức thành các mô-đun chức năng tương đối tách biệt: một lớp lấy mẫu từ cảm biến, một lớp chuẩn hóa dữ liệu, một lớp đóng gói payload, một upload client gửi readings, một command poller, một ACK handler, cùng với local buffer và device state lưu các thông tin như seq, retry counter, last_seen cục bộ và trạng thái kết nối.

Cách chia mô-đun như vậy rất quan trọng vì luồng readings và luồng command có bản chất khác nhau. Luồng readings thiên về ghi dữ liệu định kỳ và đảm bảo không mất gói; luồng command thiên về polling, đổi trạng thái và ACK có kiểm soát. Nếu dồn tất cả vào một vòng lặp lớn, firmware sẽ khó bảo trì và khó thích nghi khi backend thay đổi contract hoặc khi thiết bị cần mở rộng thêm loại dữ liệu mới.



Hình 7.2. Kiến trúc firmware theo mô-đun chức năng trên thiết bị ESP32.

Ở chiều upload readings, thiết bị nên đi theo chuỗi xử lý rõ ràng: lấy mẫu dữ liệu thô, chuẩn hóa cục bộ, gắn timestamp và seq, đóng gói vitals hoặc ECG vào payload, gắn X-Device-Token rồi gửi đến endpoint /api/v1/esp/devices/{device_id}/readings. Sau khi backend phản hồi thành công, seq mới được tăng và local buffer được dọn tương ứng. Nếu gặp lỗi tạm thời, firmware có thể đưa payload vào hàng chờ resend. Cơ chế này rất phù hợp với thiết kế dedupe ở backend, nơi cặp device_id và seq được dùng để nhận biết retransmission thay vì coi đó là lỗi hệ thống.

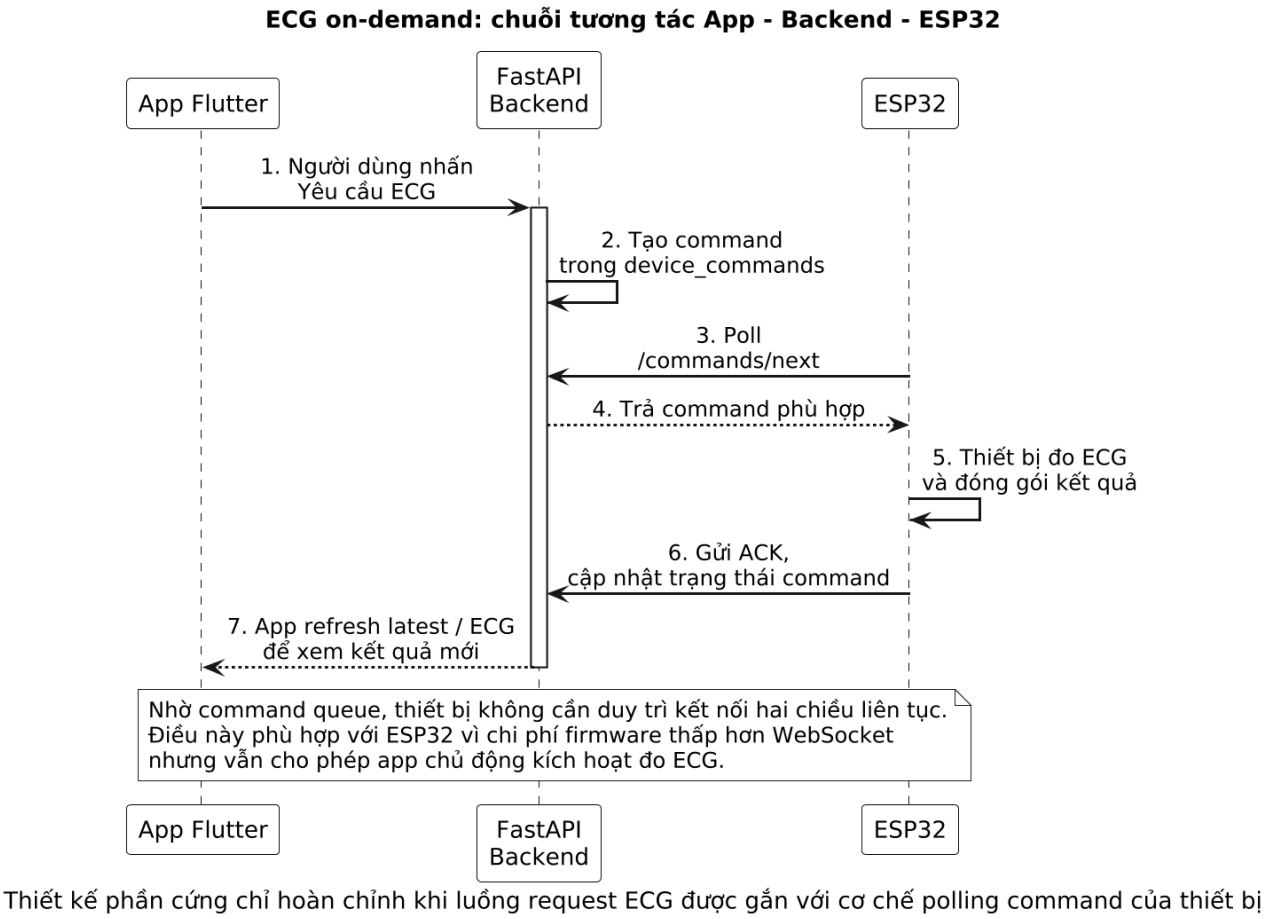


Hình 7.3. Luồng đo lường, đóng gói và gửi readings từ ESP32 lên backend.

7.4 Cơ chế ECG on-demand và sự phối hợp với backend

Điểm đặc biệt nhất của thiết bị trong hệ thống này là nó không chỉ gửi dữ liệu theo chu kỳ mà còn phải phản ứng với yêu cầu chủ động từ ứng dụng. Khi người dùng nhấn nút Yêu cầu ECG trên app, backend không thể đẩy lệnh xuống thiết bị theo kiểu socket hai chiều liên tục. Thay vào đó, backend lưu command vào device_commands, còn thiết bị định kỳ poll endpoint /commands/next để hỏi xem có việc mới hay không. Vì vậy, thiết kế firmware phải hỗ trợ command poller như một thành phần chính thức, chứ không phải logic phụ thêm vào sau.

Khi nhận được command đo ECG, thiết bị chuyển sang chế độ thực thi: khởi tạo phiên đo, thu tín hiệu, lưu tạm kết quả và gửi ACK về backend qua endpoint /commands/{command_id}/ack. Sau đó app sẽ refresh latest hoặc truy vấn ECG để lấy dữ liệu mới. Từ góc độ thiết bị, điều quan trọng nhất là phân tách rõ hai giai đoạn: nhận lệnh và thực thi lệnh. Nếu thiết bị trộn lẫn hai giai đoạn này vào một request-response duy nhất, hệ thống sẽ kém ổn định hơn khi backend chậm, mạng kém hoặc phép đo cần nhiều thời gian.



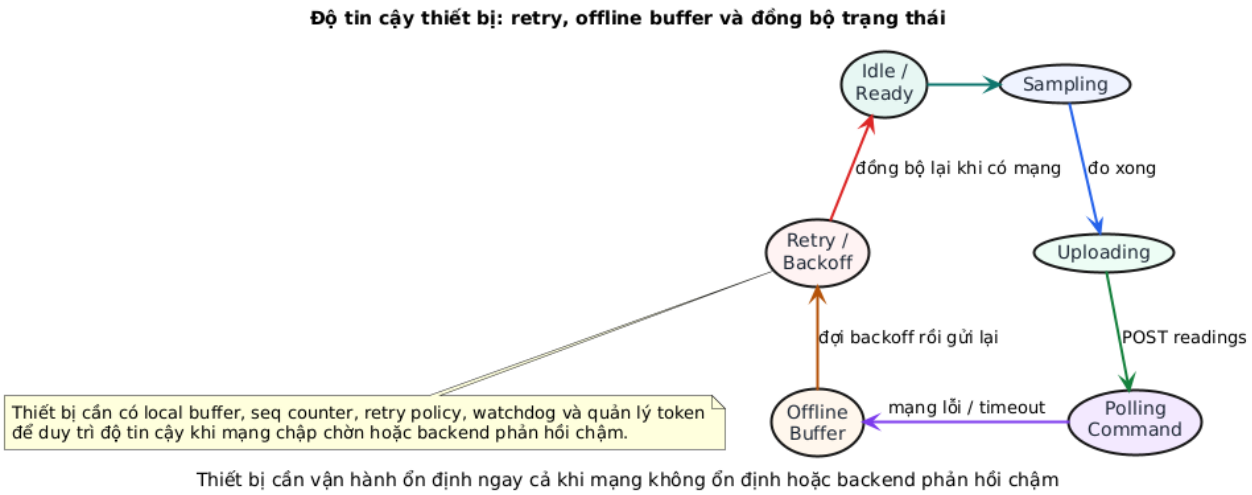
Thiết kế phần cứng chỉ hoàn chỉnh khi luồng request ECG được gắn với cơ chế polling command của thiết bị

Hình 7.4. Chuỗi tương tác giữa app, backend và ESP32 trong cơ chế ECG on-demand.

7.5 Độ tin cậy vận hành, nguồn và khả năng mở rộng

Một thiết bị theo dõi sức khỏe muốn dùng được trong thực tế phải có chiến lược tin cậy rõ ràng. Ở thiết kế hiện tại, local buffer, seq counter, retry policy, backoff, last_seen và token management là những thành phần bắt buộc. Chúng giúp readings không bị mất khi Wi-Fi chập chờn, giúp command không bị bỏ sót khi backend phản hồi chậm và giúp backend luôn có tín hiệu tương đối chính xác về lần hoạt động gần nhất của thiết bị.

Thiết kế nguồn cũng liên quan trực tiếp đến tính tin cậy đó. Thiết bị cần bảo đảm nguồn đủ ổn định cho cả các pha đo, pha truyền dữ liệu và pha chờ polling. Trong trường hợp dùng pin, logic tiết kiệm năng lượng có thể được tổ chức theo các trạng thái idle, sampling, uploading, polling và retry/backoff như một state machine có kiểm soát. Từ góc nhìn kiến trúc, đây là cách tiếp cận hợp lý vì nó buộc firmware và phần cứng phải cùng phục vụ mục tiêu độ bền vận hành thay vì chỉ tối ưu từng mô-đun riêng lẻ.



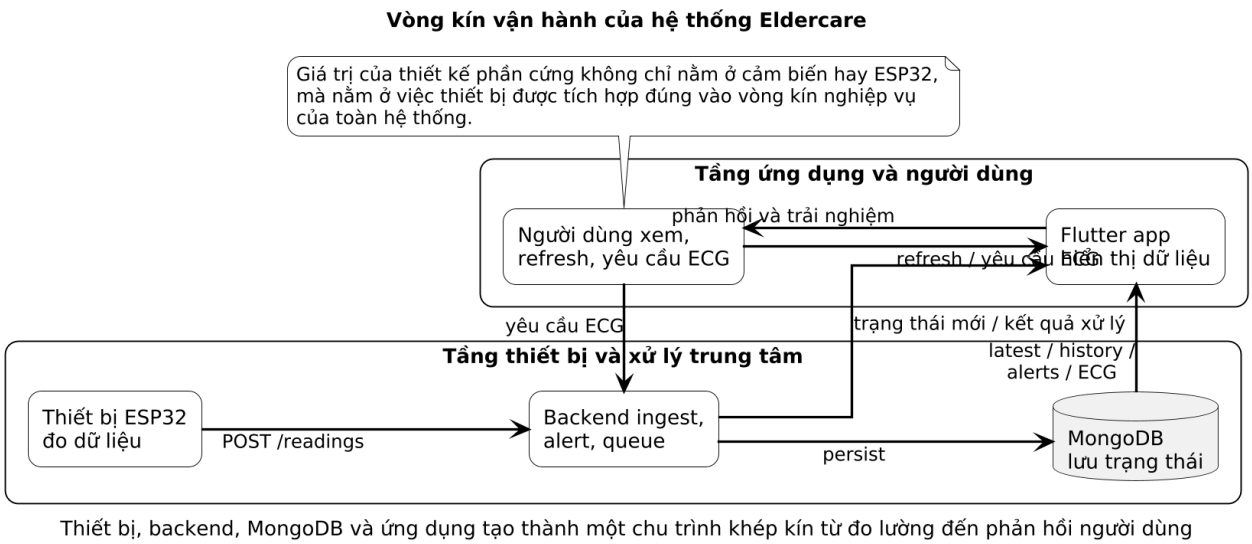
Hình 7.5. Gợi ý state machine cho độ tin cậy thiết bị: sampling, upload, polling, retry và offline buffer.

Xét về khả năng mở rộng, thiết kế mức khối hiện tại cho phép hệ thống phát triển theo hai hướng. Hướng thứ nhất là mở rộng loại dữ liệu đo, ví dụ bổ sung thêm chỉ số hoặc metadata từ các mô-đun cảm biến khác mà không phá vỡ khung kiến trúc hiện hữu. Hướng thứ hai là mở rộng mức độ trưởng thành của firmware, chẳng hạn bổ sung watchdog tốt hơn, logging thiết bị, chẩn đoán lỗi cục bộ hoặc tối ưu quản lý nguồn. Điểm cần nhìn nhận thẳng thắn là current Git chưa cung cấp một hardware repo riêng với sơ đồ mạch, BOM hoặc pin map cụ thể; vì vậy chương này được thiết kế lại ở mức kiến trúc và tích hợp hệ thống. Dù vậy, mức thiết kế này vẫn đủ để làm rõ vai trò của ESP32 trong toàn bộ đề tài và đủ để định hướng cho giai đoạn hiện thực phần cứng chi tiết về sau.

7.6 Đánh giá thiết kế phần cứng ESP32

Điểm mạnh của thiết kế thiết bị nằm ở chỗ nó bám rất sát yêu cầu hệ thống. ESP32 không bị xem như một cảm biến đơn lẻ mà được đặt đúng vị trí của một thiết bị IoT hoàn chỉnh: có lớp đo lường, lớp xử lý, lớp xác thực, lớp truyền thông, lớp poll command và lớp bảo toàn trạng thái cục bộ. Cấu trúc này phù hợp với backend hiện tại vì nó tương thích trực tiếp với nhóm API /api/v1/esp/*, cơ chế X-Device-Token và command queue ECG on-demand.

Hạn chế chủ yếu của chương này đến từ phạm vi nguồn hiện có. Khi chưa có firmware repo và sơ đồ mạch chuyên biệt, các chi tiết như sensor cụ thể, map chân, lọc nhiễu tín hiệu ECG hay thông số nguồn chưa thể trình bày ở mức phần cứng chi tiết. Tuy nhiên, đối với một đồ án theo hướng hệ thống, việc mô tả phần cứng theo kiến trúc mức khối và theo contract tích hợp với backend vẫn là hướng tiếp cận đúng. Nó cho thấy rõ thiết bị được thiết kế để làm gì, giao tiếp ra sao và cần những cơ chế nào để vận hành ổn định trong hệ sinh thái Eldercare.



Hình 7.6. Vòng kín vận hành giữa thiết bị, backend, cơ sở dữ liệu và ứng dụng người dùng.

Nhìn từ sơ đồ tổng hợp này có thể thấy thiết bị ESP32 không tồn tại như một mô-đun tách rời, mà là mắt xích đầu vào của một vòng kín dữ liệu và điều khiển. Chính vì vậy, một thiết kế phần cứng tốt trong đề tài không chỉ là thiết kế đo được dữ liệu, mà còn là thiết kế có thể gửi readings ổn định, nhận command an toàn, cộng tác chặt chẽ với backend và tạo ra trải nghiệm theo dõi nhất quán cho người dùng cuối.

CHƯƠNG 8
TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

8.1 Môi trường triển khai hệ thống

8.1.1 Môi trường chạy backend

Hệ thống được triển khai trong một môi trường có cấu trúc tương đối rõ ràng và phù hợp với định hướng phát triển của một hệ thống theo dõi sức khỏe từ xa có nhiều thành phần phối hợp với nhau. Ở phía backend, môi trường khuyến nghị là Linux hoặc WSL2 kết hợp với Docker Engine. Cách lựa chọn này có ý nghĩa thực tiễn vì nó giúp toàn bộ hệ thống được chạy trong một môi trường đồng nhất, hạn chế sự khác biệt giữa máy phát triển, máy kiểm thử và môi trường trình diễn. Trong trường hợp cần chạy backend theo cách truyền thống, hệ thống vẫn có thể hoạt động với Python 3.11 và pip, tuy nhiên phương án chuẩn của repo vẫn là dựng toàn bộ stack bằng Docker Compose.

Backend hiện được xây dựng trên nền FastAPI và sử dụng image cơ sở python:3.11-slim. Các dịch vụ hạ tầng đi kèm như MongoDB và Redis cũng được tách thành các container riêng. Điều này cho thấy hệ thống không được tổ chức theo kiểu chạy mọi thứ trên một tiến trình duy nhất, mà đã có sự phân tách rõ giữa lớp ứng dụng và lớp dữ liệu hỗ trợ. Trong cấu hình triển khai, backend sử dụng nhiều biến môi trường quan trọng như MONGO_URI, MONGO_DB, DEVICE_TOKEN_SECRET, JWT_SECRET, REFRESH_TOKEN_SECRET và REDIS_URL. Việc cấu hình theo hướng này giúp hệ thống có thể thay đổi môi trường chạy mà không cần can thiệp trực tiếp vào mã nguồn, đồng thời cũng phản ánh tư duy triển khai gần với thực tế hơn so với cách hard-code giá trị trong chương trình.

Một điểm đáng chú ý là backend không khởi động một cách độc lập hoàn toàn, mà có kiểm soát phụ thuộc đối với MongoDB và Redis. Chỉ khi hai thành phần này đạt trạng thái healthy thì backend mới được đưa lên. Bản thân backend cũng có healthcheck thông qua endpoint /ready, nhờ đó Docker có thể nhận biết chính xác service đã sẵn sàng hay chưa. Điều này rất quan trọng trong môi trường nhiều thành phần vì nó làm giảm khả năng backend khởi động khi database hoặc Redis chưa sẵn sàng, từ đó hạn chế lỗi do phụ thuộc hạ tầng.

8.1.2 Thành phần hạ tầng đi kèm

Ngoài backend, hệ thống còn được triển khai cùng với các thành phần hạ tầng hỗ trợ khác như MongoDB, Redis, Cloudflare Tunnel và một số service phụ trợ. MongoDB đóng vai trò là lớp lưu trữ dữ liệu chính, Redis phục vụ cơ chế rate limit, còn Cloudflare Tunnel được sử dụng để công bố hệ thống ra Internet khi cần thử nghiệm hoặc trình diễn từ xa. Đây là một điểm thể hiện khá rõ rằng hệ thống đã được chuẩn bị cho các tình huống vận hành thực tế chứ không chỉ để chạy nội bộ trên một máy cá nhân.

Trong cấu hình hiện tại, mỗi service quan trọng đều được gắn healthcheck riêng. MongoDB được kiểm tra bằng lệnh ping nội bộ, Redis được kiểm tra bằng redis-cli ping, còn backend được kiểm tra qua endpoint HTTP. Việc tổ chức như vậy giúp toàn bộ stack có khả năng tự phản ánh tình trạng hoạt động của từng lớp thành phần. Nếu một service gặp sự cố, trạng thái unhealthy có thể được phát hiện sớm ngay từ Docker Compose mà không cần chờ đến khi người dùng thấy lỗi ở tầng giao diện.

8.1.3 Các lệnh triển khai cơ bản

Quy trình triển khai của hệ thống hiện tại tương đối rõ ràng và có thể tái lập được từ mã nguồn. Nếu chạy local không dùng Docker, người triển khai cần tạo môi trường ảo Python, cài dependency và khởi động backend bằng uvicorn. Tuy nhiên, cách triển khai chính thức của hệ thống là sao chép file cấu hình môi trường, chỉnh lại các biến cần thiết rồi dùng Docker Compose để dựng toàn bộ stack. Sau khi dựng xong, hệ thống có thể được dừng, khởi động lại hoặc xóa môi trường chạy bằng các lệnh Docker Compose tương ứng.

Việc có sẵn các bước triển khai rõ ràng giúp hệ thống không rơi vào tình trạng "chỉ chạy được trên máy người phát triển". Đây là một điểm rất quan trọng trong luận văn, vì nó chứng minh rằng sản phẩm đã có khả năng được dựng lại một cách nhất quán từ repo nguồn.

8.2 Cấu hình và khởi tạo backend

8.2.1 Biến môi trường cần thiết

Một phần quan trọng trong quá trình triển khai backend là cấu hình biến môi trường. Ở trạng thái hiện tại, hệ thống đã tách khá rõ các nhóm cấu hình liên quan đến MongoDB, Redis, JWT, refresh token, device token, rate limit, metrics, command queue và các tham số triển khai khác. Cách tổ chức này cho thấy backend được thiết kế theo hướng linh hoạt, trong đó mã nguồn giữ phần logic còn các thông số môi trường được quản lý bên ngoài.

Điều này đặc biệt có ý nghĩa với một hệ thống có nhiều lớp xác thực như hệ thống hiện tại. Access token, refresh token và device token đều là các thành phần nhạy cảm và không nên được ghi trực tiếp vào mã nguồn. Việc tách các secret thành biến môi trường không chỉ giúp an toàn hơn mà còn làm cho hệ thống dễ triển khai trên nhiều môi trường khác nhau.

8.2.2 Trình tự khởi động hệ thống

Khi khởi động hệ thống, MongoDB và Redis được đưa lên trước, sau đó backend mới khởi động. Đây không phải chỉ là một trình tự ngẫu nhiên mà là một yêu cầu kiến trúc, vì backend phụ thuộc trực tiếp vào database và Redis trong quá trình vận hành. Chỉ khi các dịch vụ phụ thuộc sẵn sàng, backend mới

có thể tạo kết nối, khởi tạo index, thiết lập rate limiter và đưa các router vào trạng thái hoạt động.

Sau khi khởi động xong, có thể kiểm tra tình trạng hệ thống bằng /ready, /health, danh sách container đang chạy và log của backend. Việc có một trình tự khởi động rõ ràng giúp giảm nhiều lỗi khó đoán trong triển khai, đặc biệt là khi hệ thống được dựng lại nhiều lần trong môi trường thử nghiệm.

8.2.3 Kiểm tra sau triển khai

Sau khi dựng xong hệ thống, một bước cần thiết là kiểm tra nhanh các chức năng nền tảng để xác nhận backend không chỉ chạy về mặt tiến trình mà còn hoạt động đúng ở mức API. Hiện tại repo đã có cơ chế smoke test để hỗ trợ bước này. Sự có mặt của smoke test cho thấy dự án đã vượt ra khỏi mức "viết code xong" và đã bắt đầu có tư duy kiểm tra nhanh sau triển khai, điều rất cần thiết cho một hệ thống có nhiều thành phần phụ thuộc lẫn nhau.

8.3 Triển khai và kiểm tra lớp dữ liệu MongoDB – Redis

8.3.1 MongoDB trong môi trường triển khai

MongoDB trong hệ thống hiện tại không chỉ là nơi lưu readings mà là kho lưu trữ trạng thái trung tâm của gần như toàn bộ backend. Database chính của hệ thống là wearable, trong đó có các collection phục vụ cho nhiều lớp nghiệp vụ khác nhau như readings, devices, users, alerts, device_commands, auth_sessions và device_links. Điều này cho thấy lớp dữ liệu đã được tổ chức có chiều sâu, thay vì chỉ tập trung vào một bảng readings đơn giản như nhiều hệ thống thử nghiệm ban đầu.

Việc MongoDB được triển khai bằng container riêng và có volume để duy trì dữ liệu lâu dài cũng là một điểm quan trọng. Nó giúp backend có thể được dựng lại nhiều lần mà không làm mất dữ liệu hiện có, đồng thời phản ánh một cách tổ chức gần hơn với môi trường vận hành thực tế.

8.3.2 Redis trong môi trường triển khai

Redis hiện được triển khai như một thành phần phục vụ trực tiếp cho lớp bảo vệ API. Trong hệ thống này, Redis không giữ dữ liệu nghiệp vụ chính mà lưu trạng thái ngắn hạn cho rate limit. Điều này có nghĩa là Redis đang đóng vai trò thực tế trong vận hành, chứ không chỉ tồn tại ở mức cấu hình "cho có". Khi hệ thống phải tiếp nhận request từ app, từ thiết bị và từ các kịch bản kiểm thử, rate limit bằng Redis là một lớp đệm cần thiết để tránh quá tải hoặc lạm dụng tài nguyên.

8.3.3 Kiểm tra kết nối backend với MongoDB và Redis

Một nội dung quan trọng trong triển khai là khả năng xác nhận backend có thực sự kết nối được với MongoDB và Redis hay không. Nếu backend báo lỗi ở /health hoặc /ready, việc kiểm tra đầu tiên phải hướng tới URI kết nối, credentials, trạng thái container và log của dịch vụ. Đây là loại lỗi rất điển hình trong môi trường nhiều service, và việc hệ thống đã có sẵn các hướng kiểm tra cơ bản cho thấy phần triển khai đã được nghĩ đến ở góc độ thực tế vận hành.

8.4 Triển khai và kiểm thử API backend

8.4.1 Nhóm API chính được triển khai

Backend hiện tại đã triển khai tương đối đầy đủ các nhóm API chính phục vụ cho cả người dùng và thiết bị. Ở phía app và admin, hệ thống hỗ trợ các nhóm API xác thực, người dùng, thiết bị, dữ liệu sức khỏe, cảnh báo và ECG. Ở phía thiết bị, hệ thống hỗ trợ nhóm API riêng cho readings, polling command và ACK command. Điều này cho thấy backend không còn là một dịch vụ "nhận dữ liệu rồi lưu" đơn thuần, mà đã trở thành lớp điều phối trung tâm cho nhiều tác nhân khác nhau trong hệ thống.

8.4.2 Kiểm thử health, live, ready và metrics

Việc backend có các endpoint như /health, /live, /ready và /metrics là một điểm rất có giá trị trong triển khai. Chúng không trực tiếp phục vụ nghiệp vụ người dùng nhưng lại đóng vai trò rất lớn trong debug, giám sát và xác định trạng thái hệ thống sau khi triển khai. Đây là dấu hiệu cho thấy backend đã được phát triển với nhận thức rằng một hệ thống tốt không chỉ cần chạy đúng chức năng mà còn phải dễ quan sát khi có sự cố.

8.4.3 Kiểm thử nhóm API ESP

Nhóm API dành cho ESP32 có thể được kiểm thử bằng curl hoặc trực tiếp bằng firmware. Điều này rất quan trọng vì nó cho phép kiểm tra độc lập lớp giao tiếp thiết bị – server mà không cần phụ thuộc hoàn toàn vào giao diện người dùng. Trong hệ thống hiện tại, việc có contract rõ ràng cho readings, command polling và ACK giúp lớp giao tiếp thiết bị có thể được xác minh một cách thực tế.

8.5 Triển khai và kiểm thử lớp thiết bị ESP32

8.5.1 Kiến trúc triển khai thiết bị thực tế

Điểm cần nhấn mạnh trong phiên bản hiện tại của hệ thống là lớp thiết bị không còn chỉ tồn tại dưới dạng ý tưởng hay contract giao thức, mà đã được hiện thực hóa bằng hai firmware cụ thể. Firmware thứ nhất chạy trên **ESP32-S3** và đóng vai trò thiết bị/gateway chính, chịu trách nhiệm thu thập dữ liệu thực

tế, xử lý cục bộ, hiển thị trạng thái và gửi dữ liệu lên backend. Firmware thứ hai chạy trên **ESP32-C3** và đóng vai trò nguồn BLE mô phỏng, phát ra dữ liệu sinh hiệu để S3 tiếp nhận. Cách tổ chức này cho thấy phần cứng và firmware của hệ thống đã đạt mức hoàn thiện cao hơn đáng kể so với một mô hình nguyên mẫu chỉ có contract trên giấy.

8.5.2 Chức năng của firmware ESP32-S3 practical

Firmware ESP32-S3 practical cho thấy thiết bị đã tích hợp tương đối đầy đủ các lớp chức năng cần thiết cho một node/gateway theo dõi sức khỏe. Thiết bị hỗ trợ cấu hình Wi-Fi linh hoạt, đồng bộ thời gian bằng NTP, đọc tín hiệu ECG từ AD8232, đọc chuyển động từ MPU6050, kết nối BLE để nhận dữ liệu sinh hiệu từ nguồn ngoài, hiển thị trạng thái trên màn hình TFT và gửi dữ liệu lên backend qua HTTP client. Điều đó có nghĩa là lớp thiết bị hiện không chỉ là bộ gửi readings đơn giản, mà đã có năng lực xử lý và hiển thị cục bộ tương đối rõ.

Một điểm rất quan trọng là firmware này đã tổ chức dữ liệu theo dạng gói có cấu trúc, trong đó readings không chỉ gồm giá trị đo đơn lẻ mà còn kèm timestamp, seq, metadata và các trường hỗ trợ khác. Nhờ đó, lớp thiết bị đã phù hợp hơn với backend vốn đang xử lý dữ liệu theo hướng có timestamp, quality, metadata và dedupe theo seq.

8.5.3 Vai trò của firmware ESP32-C3 simulated

ESP32-C3 simulated đóng vai trò là nguồn BLE mô phỏng, phát ra các gói dữ liệu như nhịp tim, SpO2 và nhiệt độ. Mặc dù đây chưa phải một thiết bị đo y sinh độc lập hoàn chỉnh, nó có giá trị thực nghiệm rất lớn vì cho phép kiểm thử toàn bộ chuỗi BLE – gateway – backend mà không phải phụ thuộc hoàn toàn vào phần cứng cảm biến thật ngay từ đầu. Đây là một giải pháp hợp lý trong giai đoạn phát triển hệ thống, đồng thời cũng là bằng chứng rằng phần thiết bị hiện tại đã có tư duy kiểm thử theo tầng thay vì chỉ đợi phần cứng cuối cùng hoàn thiện mới tiến hành tích hợp.

8.5.4 Kiểm thử thu thập và tổng hợp dữ liệu trên thiết bị

Ở mức xử lý cục bộ, ESP32-S3 không chỉ tiếp nhận dữ liệu từ các nguồn khác nhau mà còn tổng hợp chúng lại thành các gói dữ liệu có cấu trúc. Dữ liệu từ ECG, chuyển động và BLE được hợp nhất thành các summary packet trước khi được đưa vào hàng đợi gửi lên backend. Điều này cho thấy thiết bị hiện không hoạt động theo kiểu "đọc đâu gửi đó" mà đã có một lớp xử lý trung gian nhằm chuẩn hóa dữ liệu trước khi đồng bộ với hệ thống trung tâm.

8.5.5 Kiểm thử gửi readings lên backend

Một kết quả quan trọng là firmware ESP32-S3 đã thực hiện rõ cơ chế gửi readings lên backend thông qua endpoint dành cho thiết bị. Request được gắn

X-Device-Token và payload có cấu trúc tương thích với backend. Điều này chứng minh lớp thiết bị hiện đã thật sự tham gia vào hệ thống theo đúng contract API, thay vì chỉ dừng ở mức mô phỏng logic nội bộ.

Bên cạnh đó, việc có hàng đợi cục bộ và cơ chế đóng gói readings giúp tăng độ ổn định khi mạng không thật sự lý tưởng. Đây là một bước tiến đáng kể về mặt thực nghiệm, vì nó cho thấy thiết bị đã được thiết kế tính đến khả năng vận hành ngoài môi trường phát triển lý tưởng.

8.5.6 Kiểm thử đồng bộ thời gian và trạng thái mạng

Việc firmware hỗ trợ đồng bộ thời gian bằng NTP giúp readings có timestamp đáng tin cậy hơn, đồng thời hỗ trợ tốt hơn cho truy vấn lịch sử ở backend. Hệ thống cũng có cơ chế cấu hình Wi-Fi linh hoạt, qua đó giúp thiết bị dễ vận hành hơn trong các môi trường khác nhau. Những yếu tố này tuy không trực tiếp tạo ra chức năng nghiệp vụ nhưng có vai trò rất quan trọng trong việc giúp thiết bị có thể vận hành thật.

8.5.7 Giới hạn hiện tại của lớp thiết bị

Mặc dù phần thiết bị đã hoàn thiện hơn đáng kể, vẫn cần ghi nhận một cách trung thực rằng một số phần nâng cao vẫn chưa hoàn tất ở mức production. Chẳng hạn, lớp upload ECG chuyên biệt hiện chưa phải trọng tâm vận hành chính trong firmware đang dùng, và phần C3 hiện vẫn là nguồn BLE mô phỏng. Tuy nhiên, đây không còn là hạn chế kiểu "chưa có lớp thiết bị", mà là giới hạn trong việc hoàn thiện sâu hơn các chức năng nâng cao. Xét trong phạm vi đồ án, mức độ hoàn thiện của thiết bị hiện tại đã đủ mạnh để nâng giá trị thực nghiệm của toàn bộ hệ thống lên đáng kể.

8.6 Kiểm thử và đánh giá ứng dụng web

Ứng dụng Web hiện tại không chỉ dừng lại ở lớp giao diện mà còn được xây dựng với một hệ thống kiểm thử tương đối có tổ chức. Các kịch bản kiểm thử giao diện cho thấy ứng dụng đã được kiểm tra trong những tình huống quan trọng như trạng thái chưa đăng nhập, chưa có thiết bị, không đủ quyền thao tác và kiểm tra dữ liệu đầu vào của các form đăng nhập hoặc đăng ký.

Điều này là minh chứng cho việc lớp ứng dụng được phát triển hướng đến các tình huống sử dụng thực tế, thay vì chỉ đóng vai trò trình diễn dữ liệu trong điều kiện lý tưởng.

Điểm đáng chú ý là app xử lý khá rõ các trạng thái của người dùng. Khi chưa đăng nhập, app hiển thị đúng thông điệp và hướng đi tiếp theo. Khi đã đăng nhập nhưng chưa có thiết bị, giao diện phản hồi bằng empty state phù hợp. Khi người dùng không đủ quyền, các hành động như yêu cầu ECG được ẩn đi. Cách tổ chức này giúp lớp client phản ánh tốt logic phân quyền và phạm vi dữ liệu từ backend.

8.7 Đánh giá mức độ hoàn thiện của hệ thống

Nếu xét tổng thể, hệ thống hiện đã đạt một mức hoàn thiện khá tốt trong phạm vi đồ án. Backend có kiến trúc rõ ràng, có auth, rate limit, healthcheck, observability và luồng ECG on-demand. MongoDB và Redis được triển khai hợp lý để phục vụ lưu trữ và bảo vệ API. web đã có logic session, device scope và kiểm thử giao diện. Quan trọng hơn, lớp thiết bị hiện đã được hiện thực hóa ở mức thực nghiệm bằng firmware ESP32-S3 practical và ESP32-C3 simulated, chứ không còn dừng ở mức mô tả giao thức.

Hạn chế còn lại chủ yếu nằm ở mức độ mở rộng và hoàn thiện sâu hơn. Một số phần vẫn phù hợp hơn với nguyên mẫu hoặc quy mô nhỏ đến trung bình, và firmware vẫn còn không gian để hoàn thiện thêm ở các hướng nâng cao. Tuy nhiên, những giới hạn này không phủ nhận giá trị của hệ thống, mà trái lại cho thấy đề tài đã phát triển được một nguyên mẫu có nền tảng đủ vững để tiếp tục mở rộng.

8.8 Hướng cải tiến sau triển khai

Từ trạng thái hiện tại, hệ thống có thể tiếp tục được phát triển theo nhiều hướng. Ở phía backend, có thể mở rộng integration test với MongoDB và Redis thật, nâng cấp observability, hoặc tối ưu command queue nếu quy mô hệ thống tăng. Ở phía ứng dụng, có thể bổ sung kiểm thử end-to-end với backend thật. Ở phía thiết bị, các hướng quan trọng nhất là hoàn thiện sâu hơn lớp ECG, nâng cấp giao tiếp bảo mật và thay thế nguồn BLE mô phỏng bằng thiết bị đo thực tế hoàn chỉnh hơn nếu cần.

8.9 Kết luận chương

Chương 8 đã trình bày quá trình triển khai, kiểm thử và đánh giá hệ thống theo dõi sức khỏe từ xa trên cả ba lớp chính là backend, ứng dụng web và thiết bị ESP32. So với trạng thái nguyên mẫu ban đầu, phiên bản hiện tại cho thấy mức độ hoàn thiện thực nghiệm đã tăng lên rõ rệt, đặc biệt ở lớp thiết bị. Việc có firmware ESP32-S3 practical và ESP32-C3 simulated giúp hệ thống không chỉ có backend và app hoàn chỉnh mà còn có lớp thiết bị thực sự tham gia vào chuỗi xử lý dữ liệu.

Có thể kết luận rằng hệ thống hiện đã đạt được giá trị thực nghiệm rõ rệt trong phạm vi đồ án. Backend có thể triển khai và vận hành, app có thể sử dụng với nhiều trạng thái thực tế, còn thiết bị đã có khả năng thu thập, tổng hợp và gửi dữ liệu về backend theo đúng kiến trúc thiết kế. Đây là cơ sở vững chắc để khẳng định đề tài không chỉ có giá trị học thuật mà còn có tính ứng dụng và khả năng phát triển tiếp trong tương lai.

CHƯƠNG 9
KẾT LUẬN VÀ KIẾN NGHỊ

9.1 Kết quả đạt được của đề tài

9.1.1 Xây dựng được hệ thống theo dõi sức khỏe từ xa hoàn chỉnh ở mức nguyên mẫu tích hợp

Kết quả quan trọng nhất của đề tài là đã xây dựng được một hệ thống theo dõi sức khỏe từ xa có kiến trúc tích hợp tương đối hoàn chỉnh, trong đó các thành phần thiết bị, backend và ứng dụng người dùng không còn tồn tại rời rạc mà đã được kết nối thành một chu trình nghiệp vụ thống nhất. Theo README backend hiện tại, hệ thống gồm ba nhóm client chính là ESP32 wearable, ứng dụng web/admin và backend FastAPI; phía backend tiếp tục kết nối MongoDB để lưu trữ dữ liệu và Redis để thực hiện rate limit. Kiến trúc này đồng thời hỗ trợ xác thực riêng cho thiết bị bằng X-Device-Token và xác thực cho app/admin bằng Bearer JWT, cho thấy đây không còn là một mô hình thử nghiệm rời rạc mà đã đạt đến mức một nguyên mẫu hệ thống thực thụ.

Ý nghĩa của kết quả này nằm ở chỗ đề tài không chỉ giải quyết một bài toán phần mềm đơn lẻ, mà đã triển khai được một mô hình hoàn chỉnh bao gồm lớp thiết bị đầu cuối, lớp xử lý trung tâm, lớp lưu trữ dữ liệu và lớp tương tác với người dùng. Với phạm vi của một đồ án tốt nghiệp, đây là mức độ hoàn thiện tốt, vì nó thể hiện được năng lực thiết kế và tích hợp hệ thống theo hướng gần với thực tế triển khai.

9.1.2 Xây dựng được backend FastAPI có chức năng tương đối đầy đủ

Một kết quả nổi bật của đề tài là backend hiện tại đã đạt mức hoàn thiện cao so với phạm vi nguyên mẫu. README backend cho thấy hệ thống đã có đầy đủ các nhóm API cho xác thực, quản lý user, quản lý device, ingest readings, truy vấn latest, history, summary, ECG, cảnh báo và command queue cho ECG on-demand. Backend cũng đã chuyển sang mô hình **device-centric**, trong đó dữ liệu được đọc theo device_id, còn quyền truy cập thiết bị được suy ra từ device_links với hai mức chính là owner và viewer; admin chỉ còn là vai trò ở mức hệ thống và bootstrap. Điều này phản ánh một kiến trúc chặt chẽ hơn và sát với cách hệ thống thực sự vận hành.

Ở tầng bảo mật và kiểm soát truy cập, backend không chỉ dùng Bearer JWT cho ứng dụng mà còn có refresh token dạng opaque được lưu hash trong auth_sessions, đồng thời dùng X-Device-Token riêng cho ESP32. Ngoài ra, hệ thống cũng đã có health endpoint, metrics, logging, script backup/restore/monitor và rate limit dùng Redis, cho thấy backend không chỉ dừng ở mức xử lý nghiệp vụ mà còn được chuẩn bị cho triển khai và giám sát vận hành.

9.1.3 Thiết kế được cơ sở dữ liệu MongoDB phù hợp với bài toán

Đề tài đã xây dựng được mô hình dữ liệu MongoDB phù hợp với bản chất của hệ thống theo dõi sức khỏe. Theo README backend, database chính hiện dùng các collection như users, devices, device_links, health_readings, alerts, device_commands, auth_sessions, audit_logs và readings legacy. Hệ thống cũng đã có retention và chỉ mục rõ ràng, chẳng hạn TTL cho health_readings và alerts, unique partial index cho dedupe readings, unique sparse index cho device_commands.request_id, cũng như các ràng buộc hợp lý cho users, devices và device_links.

Điểm mạnh của thiết kế dữ liệu hiện tại là nó không chỉ "lưu được dữ liệu", mà còn phục vụ trực tiếp cho các luồng nghiệp vụ quan trọng. health_readings cho phép lưu linh hoạt vitals và ECG; alerts phục vụ cảnh báo theo ngưỡng; auth_sessions hỗ trợ refresh token rotation; còn device_commands giữ vai trò như command queue cho ECG on-demand. Đây là nền tảng rất quan trọng, vì tính ổn định của latest, history, alerts hay ECG đều phụ thuộc trực tiếp vào chất lượng thiết kế dữ liệu.

9.1.4 Xây dựng được ứng dụng web đóng vai trò client chính cho hệ thống

Ở phía người dùng, đề tài đã xây dựng được ứng dụng Web với cơ chế quản lý trạng thái tương đối rõ ràng. Trong quá trình khởi tạo, ứng dụng thiết lập các module quản lý trạng thái cho các thành phần như session, thiết bị, dữ liệu thời gian thực, dữ liệu lịch sử, ECG và cảnh báo. Điều này cho thấy ứng dụng không chỉ đóng vai trò là lớp giao diện đơn giản mà đã được tổ chức theo hướng có quản lý trạng thái bài bản, hỗ trợ điều phối dữ liệu giữa các phần của hệ thống.

Trong HomePage, ứng dụng thực hiện đồng bộ thiết bị hiện tại, bind realtime, history và ECG theo deviceId, cho phép chọn thiết bị, làm mới dữ liệu, xem cảnh báo và gửi yêu cầu ECG nếu tài khoản là chủ thiết bị. Trang này cũng xử lý rõ trạng thái chỉ xem, trong đó viewer không được quyền gửi yêu cầu đo ECG. Điều đó cho thấy app đã phản ánh đúng logic quyền hạn và device scope của backend.

Ngoài lớp giao diện, hệ thống kiểm thử cũng là một kết quả đáng ghi nhận. Các kịch bản kiểm thử hiện tại đã xác nhận được nhiều trạng thái quan trọng như hiển thị form đăng nhập khi chưa có phiên làm việc, kiểm tra dữ liệu đầu vào của form đăng nhập, trạng thái chưa đăng nhập trên trang chính, trạng thái chưa có thiết bị đối với người dùng đã xác thực, trường hợp người dùng không có quyền thao tác với chức năng yêu cầu đo ECG, cũng như kiểm tra dữ liệu của form đăng ký.

Điều này cho thấy phần frontend đã được kiểm thử ở mức hành vi người dùng, thay vì chỉ dừng lại ở việc kiểm tra hiển thị tĩnh của giao diện.

9.1.5 Hoàn thiện được lớp thiết bị ở mức phần cứng – firmware thực nghiệm

So với trạng thái trước đây chỉ mới có contract tích hợp và gateway mẫu, phần thiết bị hiện nay đã tiến thêm một bước rất quan trọng. Từ hai firmware bạn cung cấp, có thể thấy lớp thiết bị đã được hiện thực hóa theo hướng rõ ràng hơn và gần thực tế hơn. Ở nhánh ESP32-S3 thực nghiệm, thiết bị đã tích hợp mạch ECG thực với AD8232, lấy mẫu ECG ở 250 Hz, suy ra nhịp tim từ tín hiệu ECG, ước lượng nhịp thở từ MPU6050, hiển thị waveform trên TFT, quản lý ring buffer ECG, hàng đợi summary upload và chuẩn bị block ECG chờ gửi. Thiết bị cũng đã có Wi-Fi portal, đồng bộ thời gian, cơ chế upload readings và lớp trạng thái cục bộ để phục vụ retry và vận hành ổn định.

Ở nhánh ESP32-C3 mô phỏng, thiết bị có vai trò nguồn BLE mô phỏng cho các chỉ số wrist như nhịp tim, SpO2 và nhiệt độ, giúp tạo nguồn dữ liệu đầu vào rõ ràng cho gateway thực nghiệm. Điều này cho thấy lớp thiết bị không còn nằm ở mức ý tưởng hay tài liệu tích hợp, mà đã được hiện thực hóa thành mã nguồn có thể chạy, đo, đóng gói và đưa dữ liệu vào hệ thống. Với việc phần cứng và firmware thực nghiệm đã được hoàn thiện ở mức chức năng cốt lõi, đề tài hiện có giá trị thực nghiệm cao hơn đáng kể.

9.2 Đánh giá mức độ hoàn thành mục tiêu đề tài

9.2.1 Mức độ hoàn thành về mặt kiến trúc

Nếu đối chiếu với mục tiêu tổng quát của đề tài, có thể đánh giá rằng hệ thống hiện đã đạt mức hoàn thành tốt về mặt kiến trúc. Ba lớp chính của hệ thống là thiết bị, backend và ứng dụng đã được xác lập rõ ràng, đồng thời có contract giao tiếp nhất quán giữa nhau. Backend giữ vai trò trung tâm xử lý, thiết bị thực hiện thu thập và gửi dữ liệu, còn app đảm nhiệm lớp tương tác với người dùng. Về mặt kiến trúc tích hợp, mục tiêu xây dựng một hệ thống theo dõi sức khỏe từ xa nhiều lớp có thể xem là đã đạt được.

9.2.2 Mức độ hoàn thành về mặt chức năng

Về chức năng, backend hiện đã hỗ trợ tốt các luồng cốt lõi như đăng nhập, refresh token, quản lý thiết bị, ingest readings, cảnh báo, latest, history, summary, ECG và command queue. ứng dụng web đã xử lý được các trạng thái quan trọng như chưa đăng nhập, chưa có thiết bị, chọn thiết bị hiện tại, chế độ chỉ xem và yêu cầu ECG theo quyền owner. Thiết bị hiện cũng đã có lớp đo lường thực nghiệm, xử lý tín hiệu và upload dữ liệu. Vì vậy, có thể nói phần lớn các luồng chức năng chính của hệ thống đã được xây dựng tương đối đầy đủ.

Tuy nhiên, để đánh giá trung thực, vẫn nên nhìn nhận rằng mức hoàn thiện hiện tại là mức **nguyên mẫu tích hợp mạnh**, chưa phải mức sản phẩm production hoàn chỉnh. Hệ thống đã có đầy đủ chiều sâu kỹ thuật để chứng minh

hướng đi của đề tài, nhưng vẫn còn một số khía cạnh cần mở rộng thêm nếu muốn vận hành ở quy mô lớn hoặc trong môi trường thương mại.

9.2.3 Mức độ hoàn thành về mặt triển khai và kiểm thử

Về triển khai, hệ thống backend đã được tổ chức với môi trường Docker Compose, kết hợp với các cơ chế kiểm tra trạng thái (health check), các script hỗ trợ sao lưu và khôi phục dữ liệu, cũng như các kịch bản kiểm thử nhanh (smoke test). Điều này cho thấy hệ thống đã có quy trình khởi tạo và kiểm tra vận hành tương đối rõ ràng.

Ở góc độ kiểm thử, ứng dụng Web đã xây dựng các kịch bản kiểm thử cho các trạng thái quan trọng của giao diện, trong khi backend được kiểm tra thông qua các kịch bản smoke test. Đối với lớp thiết bị, hệ thống sử dụng cả firmware mô phỏng và firmware thực nghiệm nhằm kiểm tra hoạt động ở từng lớp khác nhau.

Đây là một điểm tích cực của đề tài, vì hệ thống không chỉ dừng lại ở việc xây dựng chức năng mà còn chú trọng đến việc kiểm tra tính đúng đắn và khả năng vận hành trong thực tế.

9.3 Những điểm mạnh nổi bật của hệ thống

9.3.1 Kiến trúc rõ ràng, bám sát bài toán thực tế

Điểm mạnh đầu tiên của hệ thống là kiến trúc rất rõ ràng. ESP32, FastAPI, MongoDB, Redis và web được kết nối thành một hệ thống nhiều lớp có vai trò cụ thể, không chồng chéo trách nhiệm. Mô hình device-centric của backend cũng giúp làm rõ phạm vi truy cập và cách tổ chức dữ liệu theo thiết bị thay vì mô hình người dùng thuần túy. Đây là một ưu điểm lớn khi trình bày luận văn, vì nó làm cho hệ thống dễ giải thích, dễ bảo vệ và dễ mở rộng.

9.3.2 Có lớp bảo mật và kiểm soát truy cập tương đối tốt

So với nhiều đồ án chỉ dừng ở xác thực đơn giản, hệ thống hiện tại đã có JWT Bearer cho app/admin, refresh token với auth_sessions, X-Device-Token cho thiết bị, và phân quyền theo owner/viewer ở mức thiết bị, trong khi admin được tách thành vai trò quản trị hệ thống. Cách tổ chức này cho thấy đề tài đã quan tâm nghiêm túc đến tính an toàn dữ liệu và quản lý truy cập.

9.3.3 Có command queue ECG on-demand mang tính đặc trưng

device_commands là một thành phần đặc trưng và tạo chiều sâu kỹ thuật cho dự án. Nó cho thấy hệ thống không chỉ ingest dữ liệu một chiều từ thiết bị, mà còn hỗ trợ điều khiển ngược từ app tới thiết bị thông qua backend. Cơ chế owner gửi yêu cầu ECG, backend enqueue command, ESP poll command, gửi

ACK, và backend dùng recovery loop để xử lý retry hoặc fail là một điểm mạnh rất rõ của hệ thống so với một API CRUD thông thường.

9.3.4 Có nền tảng vận hành khá tốt ở mức nguyên mẫu

Việc repo hiện có health check, readiness, metrics, logging, backup/restore/monitor scripts và Docker Compose là một lợi thế rất lớn. Điều này cho thấy hệ thống được xây dựng theo hướng có thể vận hành và theo dõi, chứ không chỉ là một chương trình chạy được trên máy phát triển. Đây cũng là một điểm tăng độ tin cậy cho luận văn vì nó chứng minh nhóm thực hiện đã nghĩ đến bài toán triển khai thực tế.

9.3.5 Lớp thiết bị đã có giá trị thực nghiệm rõ rệt

Điểm mạnh nổi bật mới của đề tài là lớp thiết bị hiện không còn dừng ở mức mô phỏng giao tiếp. Với firmware bạn cung cấp, hệ thống đã có một gateway ESP32-S3 thực nghiệm có khả năng lấy mẫu ECG thực, ước lượng hô hấp, hiển thị dữ liệu cục bộ và tổ chức upload readings theo chu kỳ; bên cạnh đó còn có một nguồn BLE mô phỏng cho thiết bị đeo phụ trợ. Điều này làm cho đề tài mạnh hơn đáng kể ở phương diện thực nghiệm, vì thiết bị giờ đây không chỉ là "đầu phát dữ liệu giả lập" mà đã trở thành một phần cứng có logic đo lường và xử lý cụ thể.

9.4 Hạn chế hiện tại của hệ thống

9.4.1 Command queue vẫn dùng Mongo polling

Một hạn chế quan trọng vẫn còn giữ nguyên là command queue hiện dựa trên MongoDB polling. Theo README backend, đây là lựa chọn phù hợp với nguyên mẫu small/medium scale, nhưng nếu số lượng thiết bị tăng mạnh thì sẽ không tối ưu bằng các broker chuyên dụng hơn. Đây không phải lỗi thiết kế, mà là một sự đánh đổi hợp lý theo phạm vi đồ án.

9.4.2 Quan hệ thiết bị – backend – app đã mạnh, nhưng pipeline ECG waveform thật vẫn còn dư địa hoàn thiện thêm ở mức end-to-end

Mặc dù lớp thiết bị hiện đã hoàn thiện rất nhiều về đo lường và firmware thực nghiệm, pipeline ECG waveform thật từ thiết bị tới backend rồi hiển thị sâu trên app vẫn còn là vùng có thể tiếp tục chuẩn hóa. Trong firmware thực nghiệm, khối ECG đã được lấy mẫu, lưu ring buffer và chuẩn bị block dữ liệu ECG chờ gửi, nhưng việc chuẩn hóa hoàn toàn endpoint upload ECG waveform, cách đồng bộ với command on-demand và cách khai thác kết quả ở app vẫn còn có thể nâng cấp thêm để đạt mức khép kín sâu hơn. Đây là hạn chế mang tính phát triển tiếp, không phủ nhận việc thiết bị đã đạt được bước tiến lớn.

9.4.3 Hạ tầng observability chưa phải mức production đầy đủ

Dù hệ thống đã có metrics, health check và monitor script, observability hiện tại vẫn phù hợp hơn với nguyên mẫu vận hành tốt chứ chưa phải một stack production hoàn chỉnh theo kiểu Prometheus + Grafana + Alertmanager được tích hợp đầy đủ. Nếu đi xa hơn về quy mô và độ tin cậy, đây sẽ là phần cần đầu tư thêm.

9.4.4 Integration test end-to-end còn có thể mở rộng thêm

Hệ thống hiện đã có các kịch bản kiểm thử giao diện ở phía ứng dụng Web và kiểm thử nhanh (smoke test) ở phía backend. Tuy nhiên, vẫn còn dư địa đáng kể để mở rộng kiểm thử tích hợp end-to-end giữa các thành phần như frontend, backend, MongoDB, Redis và firmware thiết bị trong cùng một pipeline thống nhất.

Đây là một hạn chế hợp lý trong phạm vi của đồ án, nhưng cũng là điểm cần được ghi nhận một cách trung thực nhằm phản ánh đầy đủ mức độ hoàn thiện của hệ thống và định hướng cho các bước phát triển tiếp theo.

9.5 Bài học kinh nghiệm rút ra từ đề tài

9.5.1 Tính thống nhất giữa báo cáo và mã nguồn là yếu tố quyết định

Một bài học rất rõ từ quá trình thực hiện đề tài là nội dung luận văn chỉ thật sự vững khi bám sát sản phẩm đang có. Khi kiến trúc, API, quyền truy cập, dữ liệu và trạng thái thiết bị đều được mô tả dựa trên README, mã nguồn app, backend và firmware hiện có, nội dung báo cáo sẽ chặt chẽ và khó bị mâu thuẫn khi bảo vệ. Ngược lại, nếu giữ mô tả theo ý tưởng ban đầu mà không cập nhật theo trạng thái code thực tế thì rất dễ dẫn đến chênh lệch giữa sản phẩm và báo cáo.

9.5.2 Hệ thống nhiều lớp đòi hỏi tư duy tích hợp

Một bài học khác là hệ thống gồm backend, app và thiết bị không thể phát triển hiệu quả nếu mỗi phần được làm hoàn toàn tách biệt. Các contract dữ liệu, mô hình quyền, vòng đời command, cách hiển thị trạng thái trên app và logic thiết bị đều phải bám vào cùng một kiến trúc thống nhất. Chính dự án hiện tại cho thấy giá trị của việc thiết kế đồng bộ giữa dữ liệu, API và giao diện.

9.5.3 Quan sát và vận hành quan trọng không kém chức năng nghiệp vụ

Một hệ thống chạy được là chưa đủ. Trong quá trình thực hiện đề tài, các yếu tố như health check, metrics, logs, backup, restore, monitor script và smoke test cho thấy vai trò rất lớn trong việc nâng chất lượng hệ thống. Chúng giúp người phát triển phát hiện vấn đề sớm hơn và làm cho sản phẩm đáng tin cậy hơn.

9.6 Kiến nghị và hướng phát triển

9.6.1 Chuẩn hóa sâu hơn luồng ECG waveform thật theo hướng end-to-end

Vì phần thiết bị hiện đã tiến được đến mức đo ECG thực và quản lý dữ liệu cục bộ, hướng phát triển có giá trị nhất tiếp theo là chuẩn hóa hoàn toàn pipeline ECG waveform thật từ thiết bị lên backend và từ backend xuống app ở mức hiển thị và phân tích. Khi đó, luồng ECG của hệ thống sẽ không chỉ mạnh ở mức command queue và request on-demand, mà còn mạnh ở mức khai thác dữ liệu waveform hoàn chỉnh.

9.6.2 Nâng cấp command queue khi quy mô tăng

Nếu hệ thống cần phục vụ quy mô lớn hơn, có thể cân nhắc thay device_commands Mongo polling bằng một broker chuyên dụng như Redis Streams, RabbitMQ hoặc một nền tảng message queue khác. Điều này sẽ giúp cải thiện khả năng scale và giảm độ trễ của command dispatch.

9.6.3 Mở rộng kiểm thử end-to-end

Hệ thống có thể được nâng cấp bằng cách bổ sung integration test với MongoDB và Redis thật trong pipeline, đồng thời mở rộng kiểm thử end-to-end giữa app, backend và thiết bị. Đây là hướng rất phù hợp nếu muốn nâng dự án từ mức đồ án lên mức sản phẩm thử nghiệm nghiêm túc hơn.

9.6.4 Hoàn thiện observability và dashboard vận hành

Backend hiện đã có các thành phần nền như metrics, logs và health check. Bước tiếp theo hợp lý là phát triển dashboard và cơ chế alerting trực quan hơn để theo dõi hiệu năng, tỷ lệ lỗi, trạng thái queue và tình trạng thiết bị theo thời gian thực.

9.6.5 Mở rộng tính năng quản trị và bảo mật

Một số hướng như thu hồi toàn bộ session, quản lý sâu hơn vòng đời thiết bị, tăng cường công cụ admin hoặc hoàn thiện runbook vận hành cho backup/restore/disaster recovery đều là những hướng phát triển có giá trị. Nếu thực hiện các bước này, hệ thống sẽ tiến gần hơn tới mức sản phẩm có thể vận hành lâu dài thay vì dừng ở mức nguyên mẫu.

9.7 Kết luận chương

Chương 9 đã tổng kết toàn bộ kết quả đạt được, những điểm mạnh, những hạn chế và hướng phát triển của hệ thống theo dõi sức khỏe từ xa trên cơ sở đối chiếu trực tiếp với backend, ứng dụng web và phần firmware thiết bị hiện có. Có thể kết luận rằng đề tài đã xây dựng được một nguyên mẫu tích hợp tương đối hoàn chỉnh, trong đó backend đóng vai trò trung tâm xử lý, MongoDB và Redis hỗ trợ lưu trữ và bảo

vệ API, ứng dụng web đóng vai trò client chính cho người dùng, còn lớp thiết bị hiện đã tiến đến mức phần cứng – firmware thực nghiệm có giá trị rõ rệt.

Kết luận quan trọng nhất là dự án hiện không chỉ có giá trị học thuật mà còn có giá trị thực nghiệm rõ rệt, vì các thành phần chính đều đã được hiện thực hóa thành mã nguồn, cấu hình triển khai và cơ chế kiểm tra cụ thể. Dù vẫn còn một số giới hạn như command queue theo kiểu Mongo polling hay nhu cầu chuẩn hóa sâu hơn pipeline ECG waveform end-to-end, các hạn chế này đã được nhận diện rõ ràng và hoàn toàn có thể trở thành hướng phát triển tiếp theo. Chính điều đó làm cho đề tài có tính trung thực, có chiều sâu kỹ thuật và đủ sức thuyết phục trong phạm vi một báo cáo tốt nghiệp.

TÀI LIỆU THAM KHẢO

Docker (2026), "Docker Compose". Trực tuyến. Có tại: https://docs.docker.com/guides/docker-compose/. [Truy cập: 22/03/2026].

Espressif Systems (2026), "ESP-IDF Programming Guide". Trực tuyến. Có tại: https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32/. [Truy cập: 22/03/2026].

FastAPI (2026), "FastAPI documentation". Trực tuyến. Có tại: https://fastapi.tiangolo.com/. [Truy cập: 22/03/2026].

Fielding, R. T. (2000), "Architectural Styles and the Design of Network-based Software Architectures". Ph.D. dissertation, University of California, Irvine. Trực tuyến. Có tại: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. [Truy cập: 22/03/2026].

Hardt, D. (2012), "RFC 6749: The OAuth 2.0 Authorization Framework". IETF. Trực tuyến. Có tại: https://www.rfc-editor.org/rfc/rfc6749. [Truy cập: 22/03/2026].

Jones, M., Bradley, J. và Sakimura, N. (2015), "RFC 7519: JSON Web Token (JWT)". IETF. Trực tuyến. Có tại: https://www.rfc-editor.org/rfc/rfc7519. [Truy cập: 22/03/2026].

MongoDB (2026), "Documents - Database Manual". Trực tuyến. Có tại: https://www.mongodb.com/docs/manual/core/document/. [Truy cập: 22/03/2026].

MongoDB (2026), "TTL Indexes - Database Manual". Trực tuyến. Có tại: https://www.mongodb.com/docs/v7.0/core/index-ttl/. [Truy cập: 22/03/2026].

Redis (2026), "INCR - Pattern: Rate limiter". Trực tuyến. Có tại: https://redis.io/docs/latest/commands/incr/. [Truy cập: 22/03/2026].

Sandhu, R., Ferraiolo, D. F. và Kuhn, D. R. (2000), "The NIST Model for Role-Based Access Control: Towards a Unified Standard". NIST. Trực tuyến. Có tại: https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard. [Truy cập: 22/03/2026].

Sheffer, Y., Hardt, D. và Jones, M. (2020), "RFC 8725: JSON Web Token Best Current Practices". IETF. Trực tuyến. Có tại: https://www.rfc-editor.org/rfc/rfc8725. [Truy cập: 22/03/2026].

U.S. Department of Health and Human Services (2024). *Telehealth and remote patient monitoring: Getting started*. Trực tuyến: https://telehealth.hhs.gov/providers/best-practice-guides/telehealth-and-remote-patient-monitoring/getting-started (truy cập ngày 22/03/2026).

World Health Organization (2025), "Global strategy on digital health 2020-2027". WHO. Trực tuyến. Có tại: https://www.who.int/publications/i/item/9789240116870. [Truy cập: 22/03/2026].

PHỤ LỤC A. TỔNG HỢP CÔNG NGHỆ SỬ DỤNG

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Thành phần	Công nghệ / Nền tảng	Vai trò chính
Thiết bị đầu cuối	ESP32	Thu thập dữ liệu sức khỏe, gửi readings và nhận lệnh ECG on-demand.
Backend	FastAPI	Xử lý nghiệp vụ, xác thực, cung cấp REST API và điều phối command.
Cơ sở dữ liệu	MongoDB	Lưu trữ users, devices, health_readings, alerts, device_commands và auth_sessions.
Bộ nhớ đệm / bảo vệ API	Redis	Thực hiện rate limiting và lưu trạng thái ngắn hạn phục vụ vận hành.
Ứng dụng web	ReactJS	Đăng nhập, đồng bộ thiết bị, hiển thị latest, history, alerts và ECG thông qua trình duyệt.
Triển khai	Docker Compose	Tổ chức, khởi tạo và vận hành đồng thời nhiều dịch vụ của hệ thống.

PHỤ LỤC B. TỔNG HỢP NHÓM API CHÍNH CỦA HỆ THỐNG

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Nhóm API	Endpoint tiêu biểu	Chức năng chính
Auth	/api/v1/auth/login, /refresh, /logout, /me	Xác thực người dùng và quản lý phiên làm việc.
Devices	/api/v1/me/devices	Đồng bộ danh sách thiết bị đã liên kết với tài khoản.
Health	/devices/{id}/latest, /history, /ecg	Truy vấn dữ liệu sức khỏe hiện tại, lịch sử và ECG.
Alerts	/devices/{id}/alerts	Xem và theo dõi các cảnh báo phát sinh theo thiết bị.
ECG on-demand	/devices/{id}/ecg/request	Gửi yêu cầu đo ECG theo yêu cầu từ ứng dụng.
ESP	/api/v1/esp/devices/{id}/readings, /commands/next, /ack	Giao tiếp giữa thiết bị ESP32 và backend.
Operational	/live, /ready, /health, /metrics	Kiểm tra trạng thái hoạt động và giám sát hệ thống.

PHỤ LỤC C. TỔNG HỢP COLLECTION DỮ LIỆU CHÍNH

Phụ lục này tổng hợp nhanh các nội dung chính đã được trình bày trong báo cáo nhằm hỗ trợ tra cứu và đối chiếu khi xem tài liệu.

Collection	Vai trò chính	Trường / đặc điểm nổi bật
users	Lưu hồ sơ người dùng	Thông tin tài khoản, email, số điện thoại và dữ liệu xác thực liên quan.
devices	Lưu hồ sơ thiết bị	device_id, status, metadata, esp_token_hash, last_seen.
device_links	Liên kết người dùng - thiết bị	Xác định owner / viewer và trạng thái truy cập theo thiết bị.
health_readings	Lưu dữ liệu sức khỏe	Chứa vitals, ECG, recorded_at, received_at và seq.
alerts	Lưu cảnh báo	Theo dõi severity, acknowledged và timestamp của từng cảnh báo.
device_commands	Hàng đợi command	Lưu request_id, status, expires_at và thông tin ACK / retry.
auth_sessions	Quản lý refresh token	Lưu refresh_token_hash, expires_at, revoked_at và thông tin phiên.